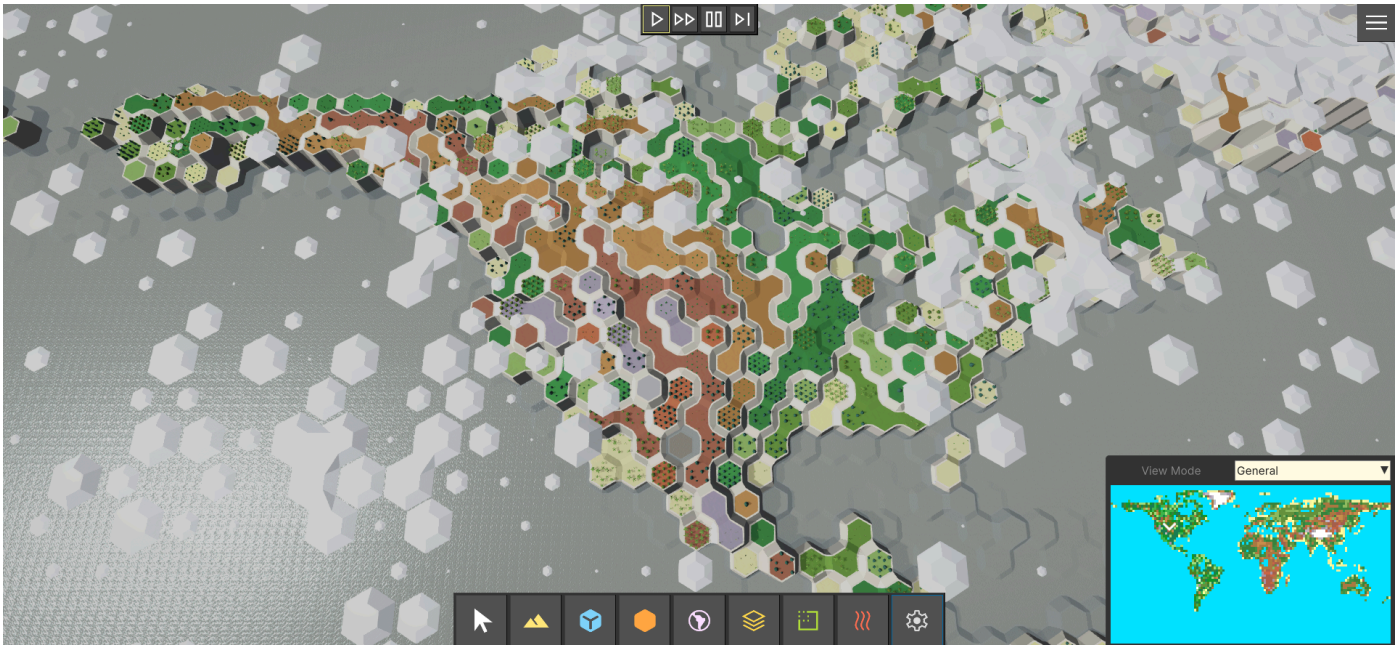
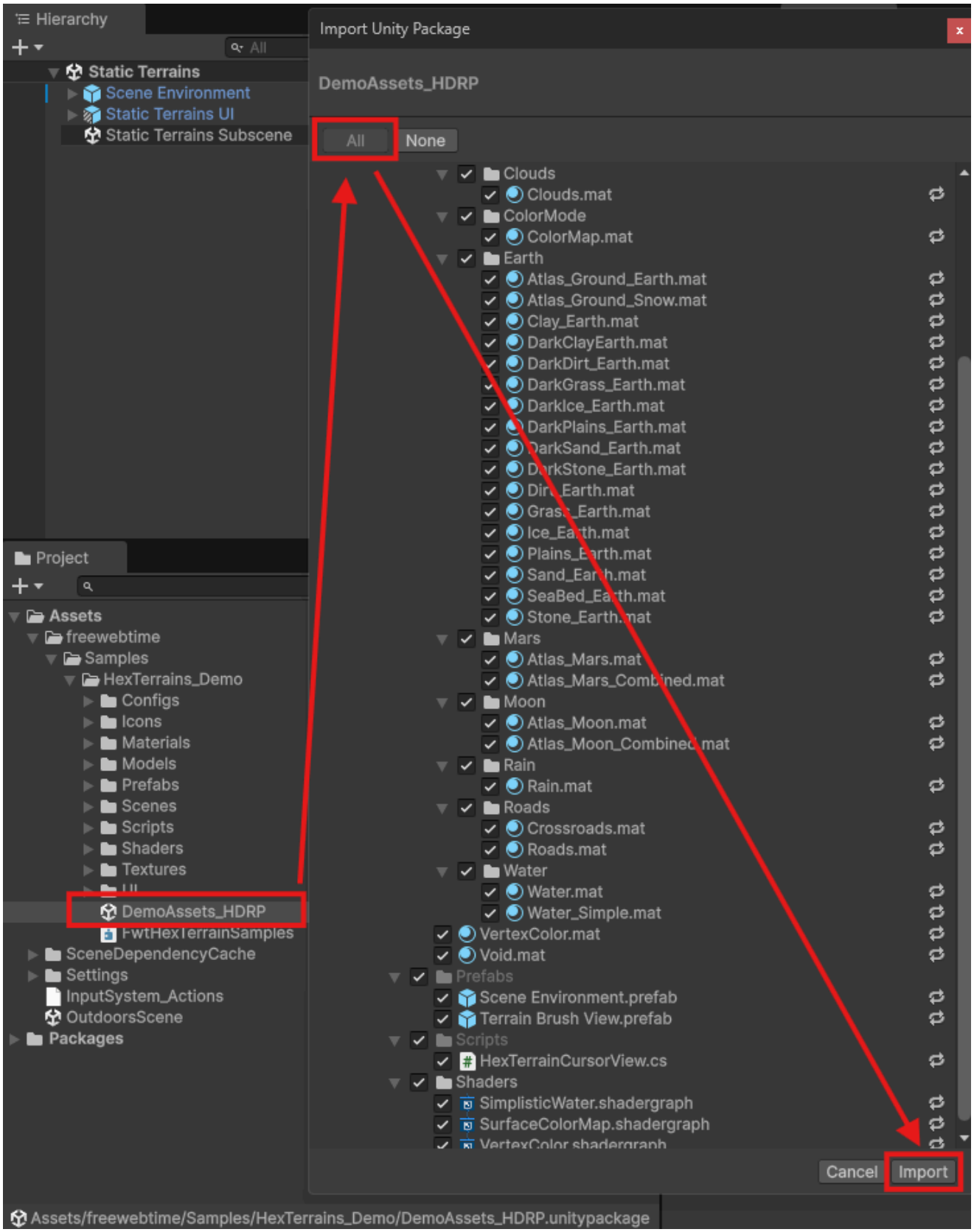
Getting Started



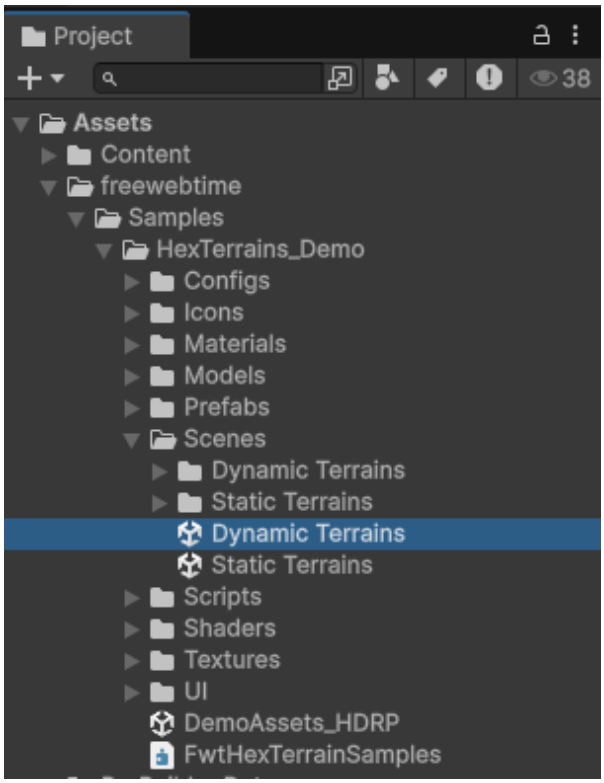
Demo Scenes

1. Install package from asset store.
2. If you use a Universal Render Pipeline, you are good to go. But if you use a High Definition Render Pipeline, extract HDRP assets from included DemoAssets_HDRP.unitypackage (`Assets/freewebtime/Samples/HexTerrains_Demo/DemoAssets_HDRP.unitypackage`). Double click on the DemoAssets_HDRP.unitypackage, in the popup ensure that all assets are selected and press "Import" button. This will overwrite URP assets with HDRP ones in your demo scenes.



3. Navigate to folder `Assets/freewebtime/Samples/HexTerrains_Demo/Scenes`

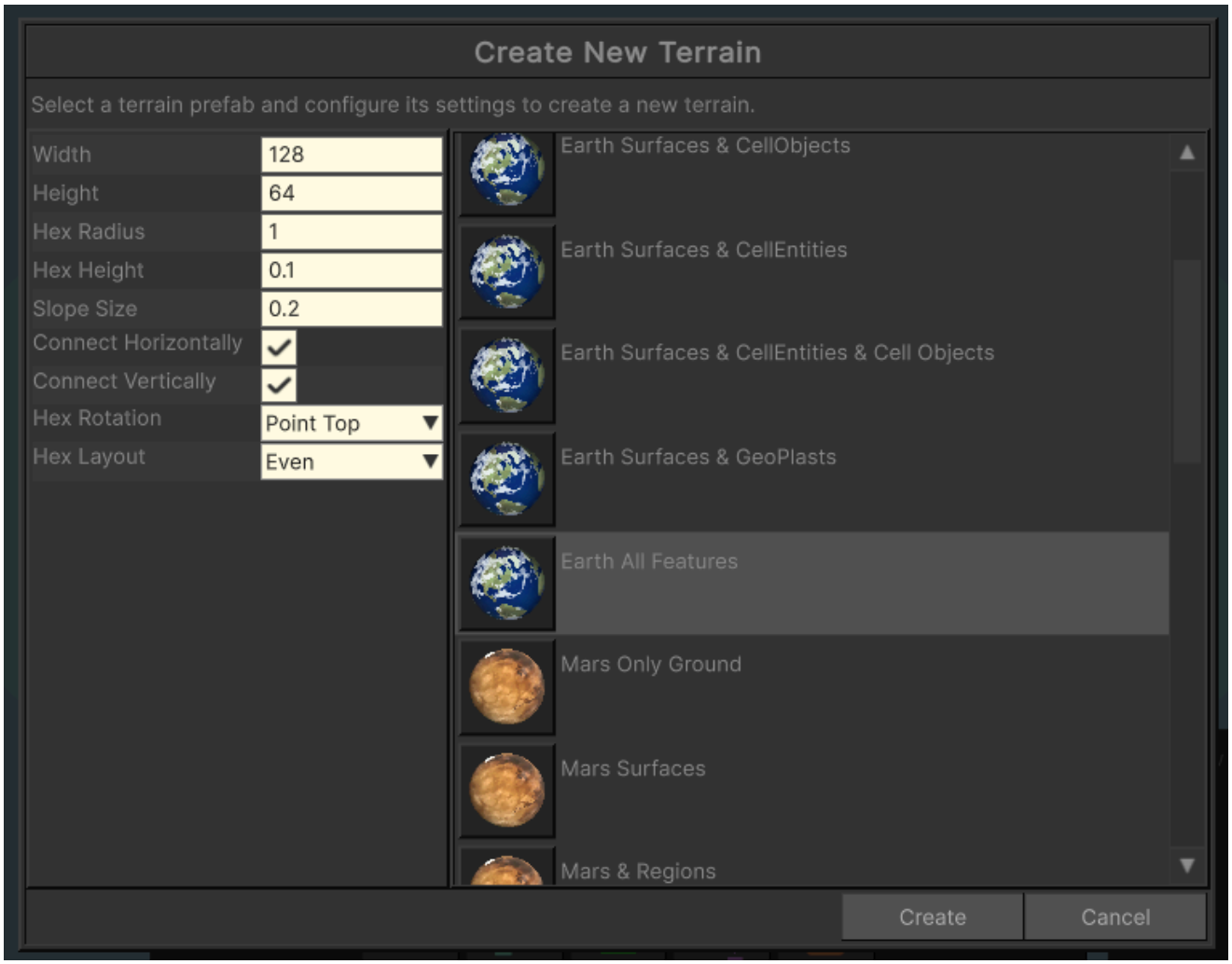
4. Open `Static Terrains` scene or `Dynamic Terrains` scene



5. Press "Play" button

6. To create another terrain, select "System" User Tool category, then "Create New"

7. In openned window select a terrain prefab and put the settings you want.



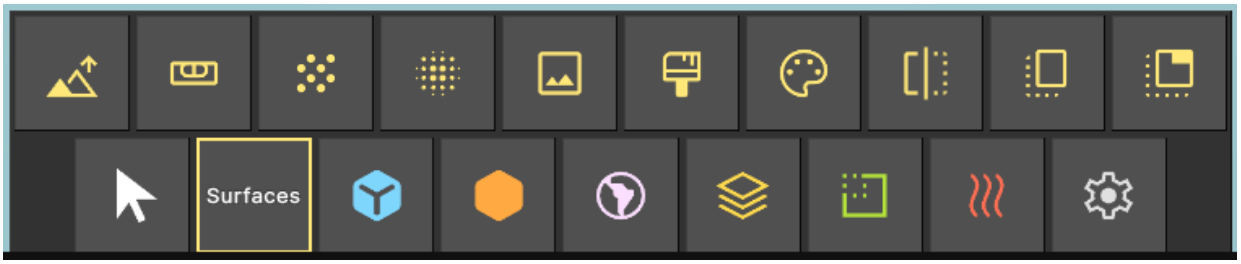
8. Press "Create" button

9. Enjoy!

In-game terrain editor

1. In-game editor is a State Machine called (see class `UserToolStateMachine`). State Machine has a set of User Tool States like Deform Terrain, Paint Biomes, Paint Cell Items, Paint Countries, etc. Every UserToolState communicates with a Terrain using a proxy implementation of the `IHexTerrainAPI` (see `HexTerrainAPI` class).
2. All User Tools are split between a set of categories:
    - `Surfaces Tools`

      (Deform, Level, Noise, Smooth, Apply Heightmap, Paint Biome, Auto-paint Biome, Stamp Height, Stamp Biome, Set Layer Visibility)

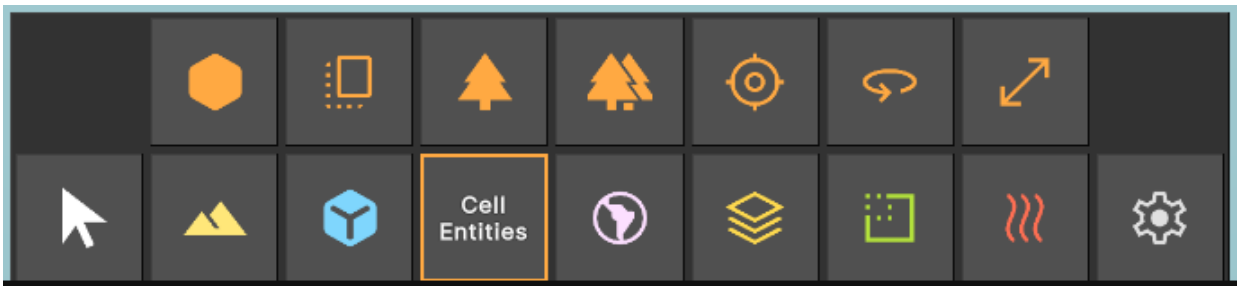
    - `Cell Objects`

      (Paint Cell Object, Stamp Cell Object, Cell Object Index, Cell Object State, Cell Object Position, Cell Object Rotation, Cell Object Scale)

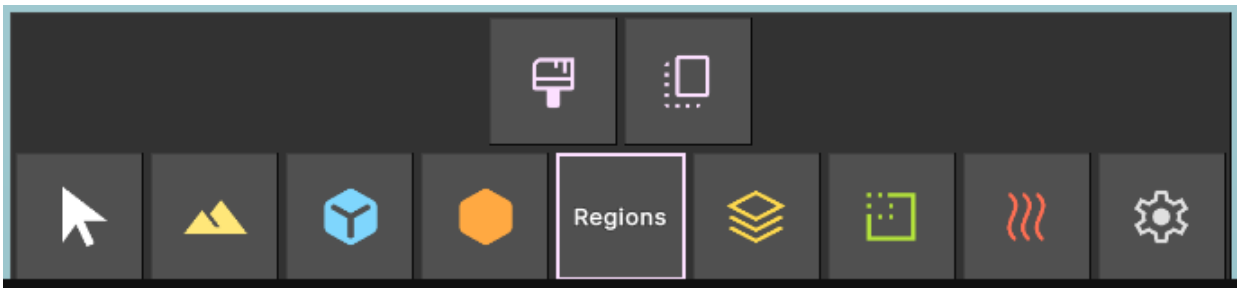
    - `Cell Entities`

      (Paint Cell Entity, Stamp Cell Entity, Cell Entity Index, Cell Entity State, Cell Entity Position, Cell Entity Rotation, Cell Entity Scale)

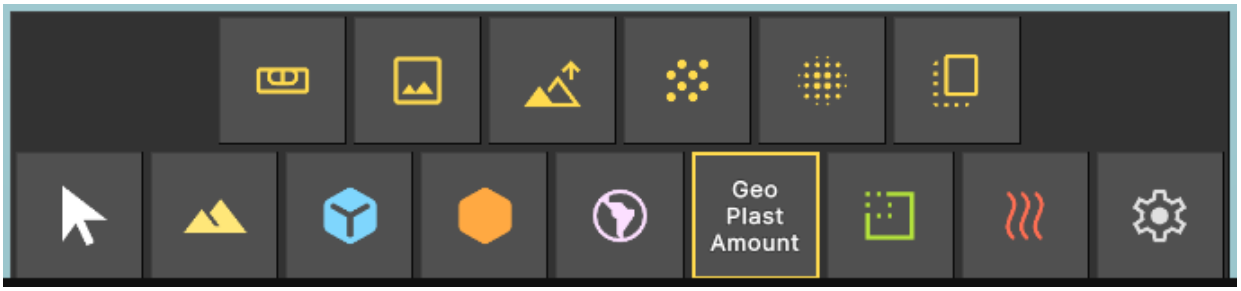
    - `Cell Regions`

      (Paint Regions, Stamp Regions)



- `GeoPlast Amount Tools`

  (Level, Apply Heightmap, Deform, Noise, Smooth, Stamp)

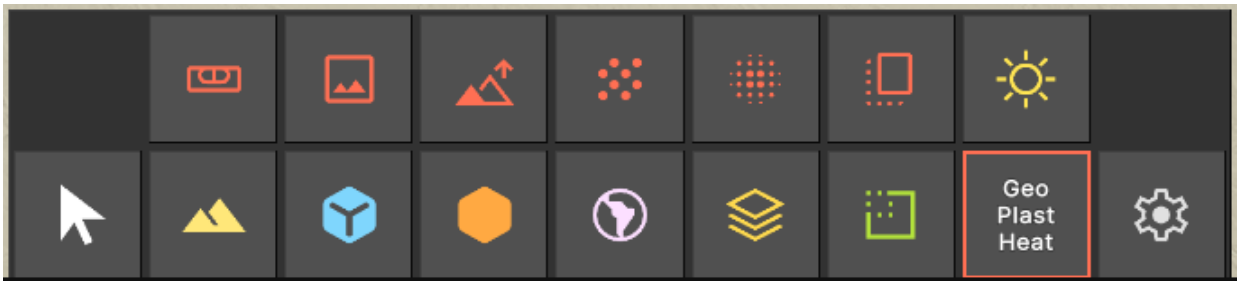


- `GeoPlast Density Tools`

  (Level, Apply Heightmap, Deform, Noise, Smooth, Stamp)

- `GeoPlast Heat Tools`

  (Level, Apply Heightmap, Deform, Noise, Smooth, Stamp, Sun)



- `System Tools`

  (Toggle Visibility, Resize Terrain, Save Terrain, Load Terrain, Create New)

View Modes

How to change the View Mode

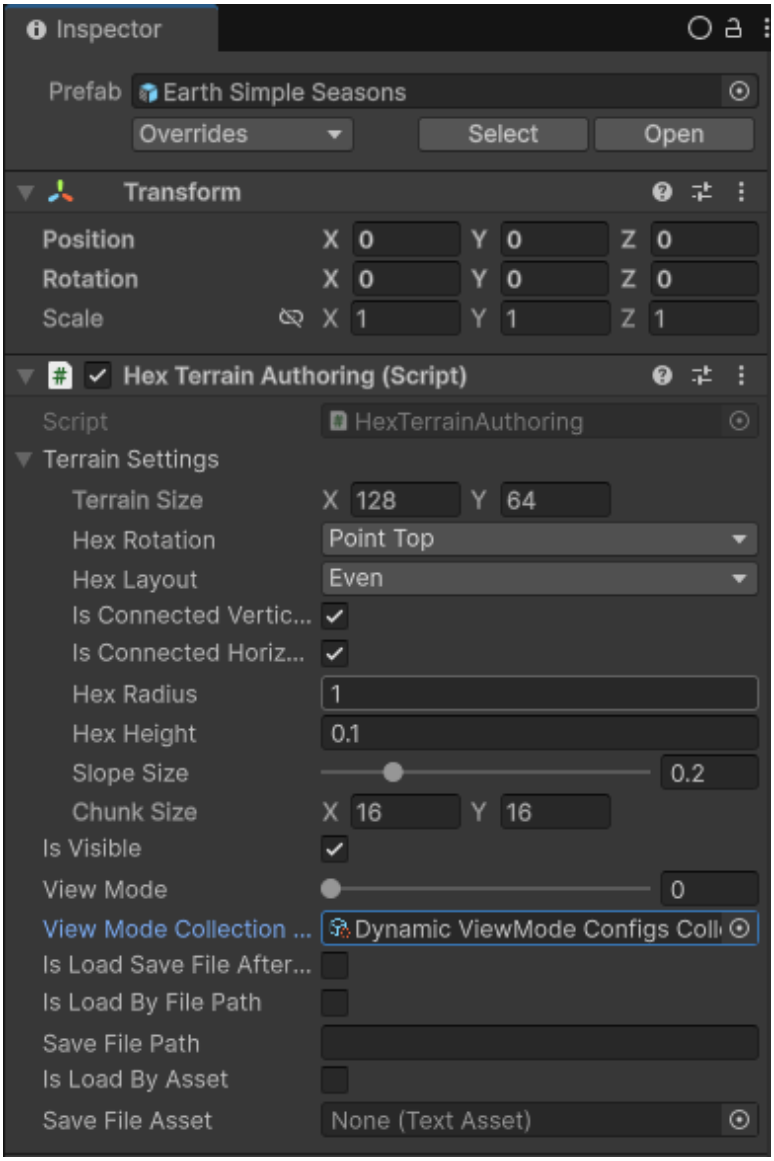
To change a view mode, click on "View Mode" dropdown and select an option you want.

Hex Terrain supports rendering in a different view modes. View mode is a number, the value of the `HexTerrainViewMode` component on a Terrain Entity.

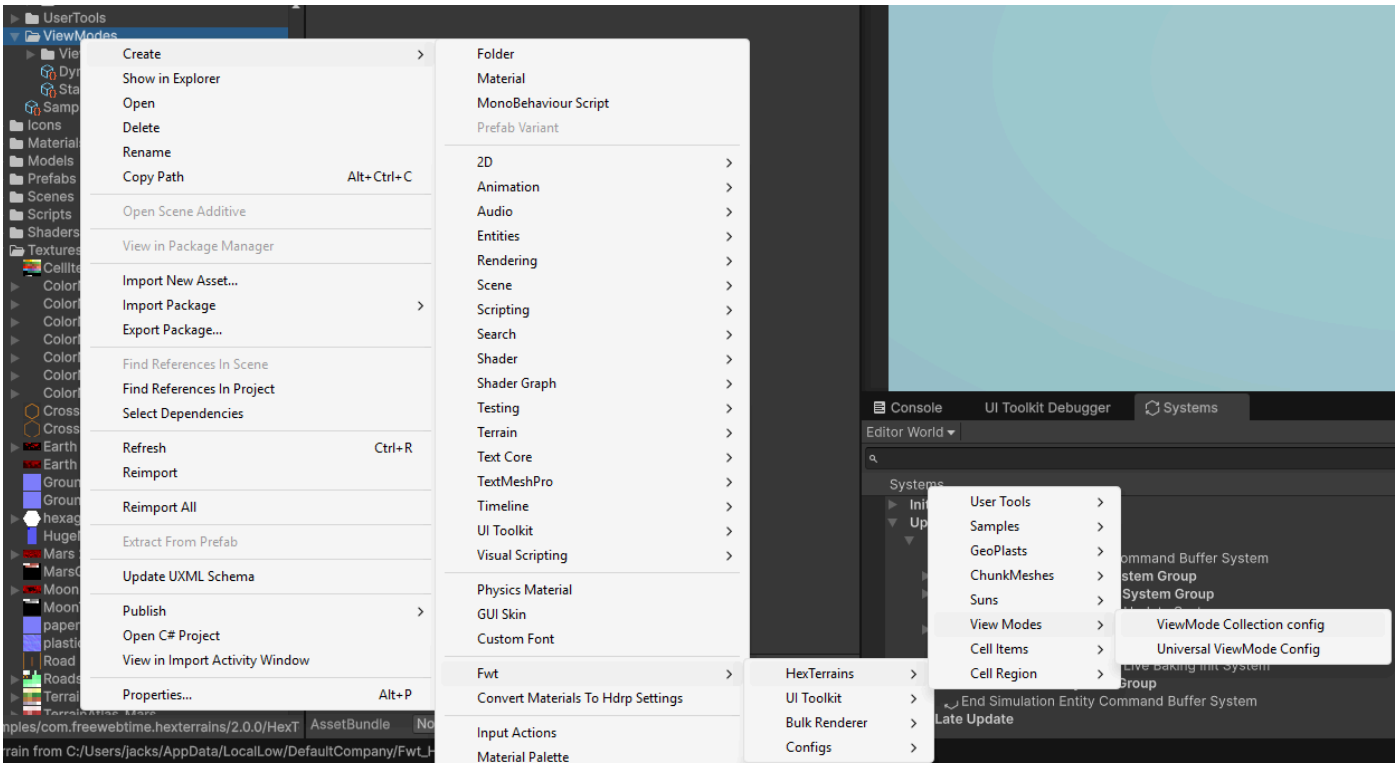
```
/// <summary>
/// Contains the current view mode of the terrain.
/// </summary>
public struct HexTerrainViewMode : IComponentData
{
    /// <summary>
    /// The current view mode of the terrain.
    /// </summary>
    public int Value;
}
```

Take a look at the Terrain Prefab. The Hex Terrain Authoring component has a `View Mode Collection Config` field.

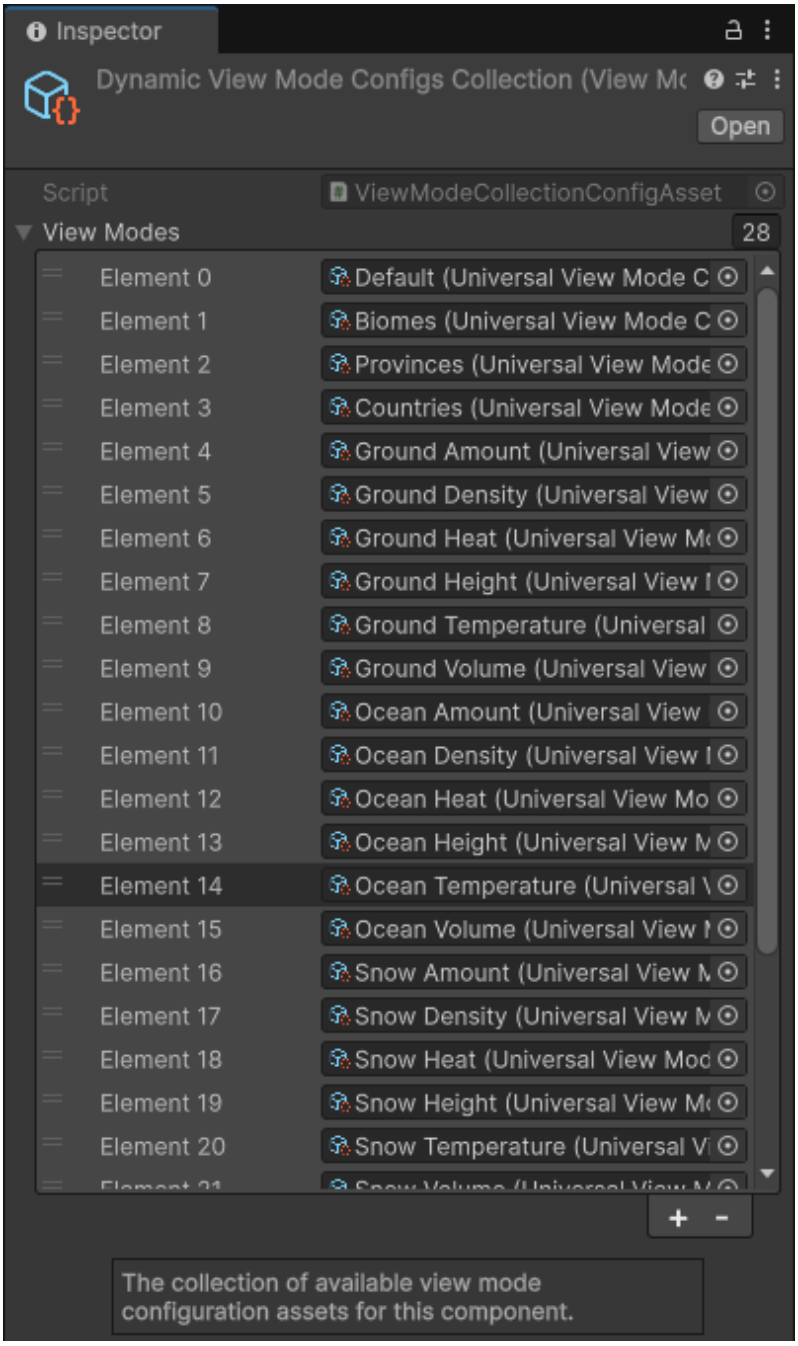
How to Setup View Modes



`View Mode Collection Config` is a scriptable object that can be created by selecting `Create/Fwt/HexTerrains/View Modes/ViewMode Collection config`



The asset holds a collection of View Mode configs. You may want to have different collections of View Modes for different terrains:



There are several built-in view modes already, but it's very easy to expand the list. Here are few examples:

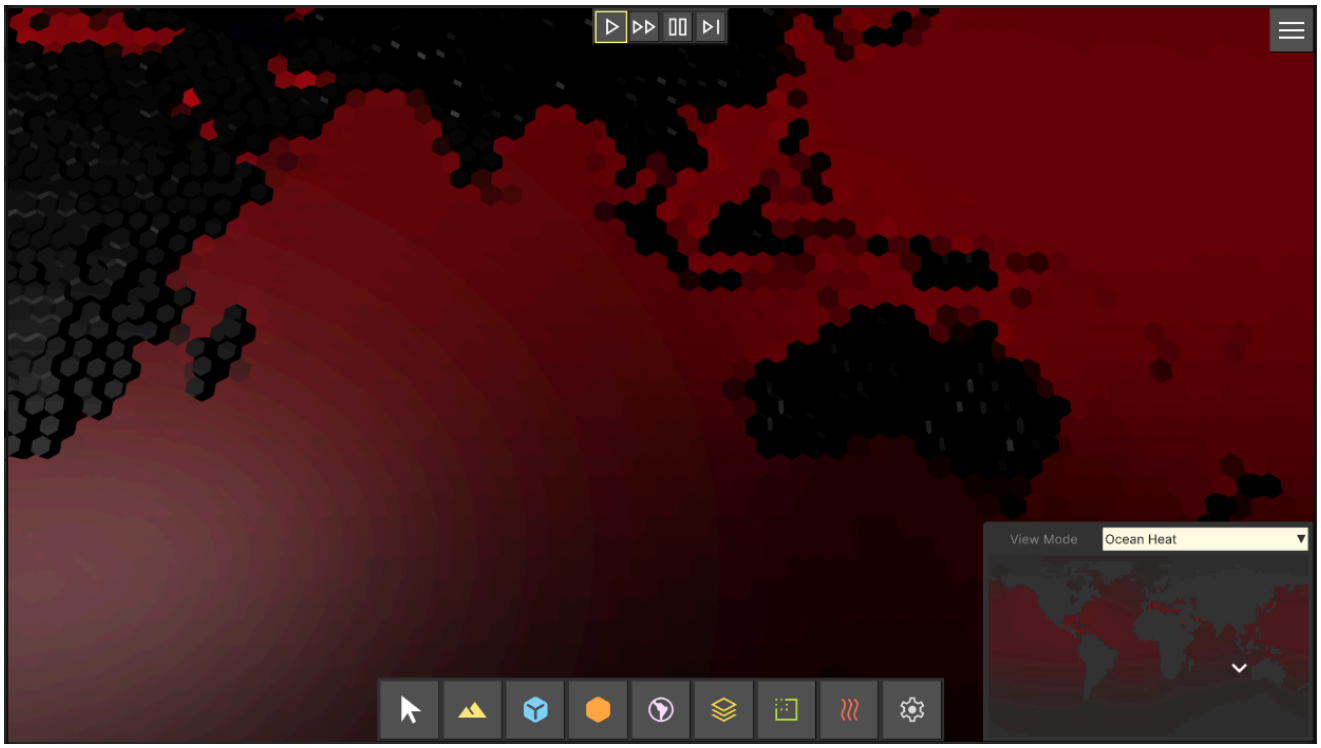
- Default View Mode (Regular terrain view)

- Biomes View Mode

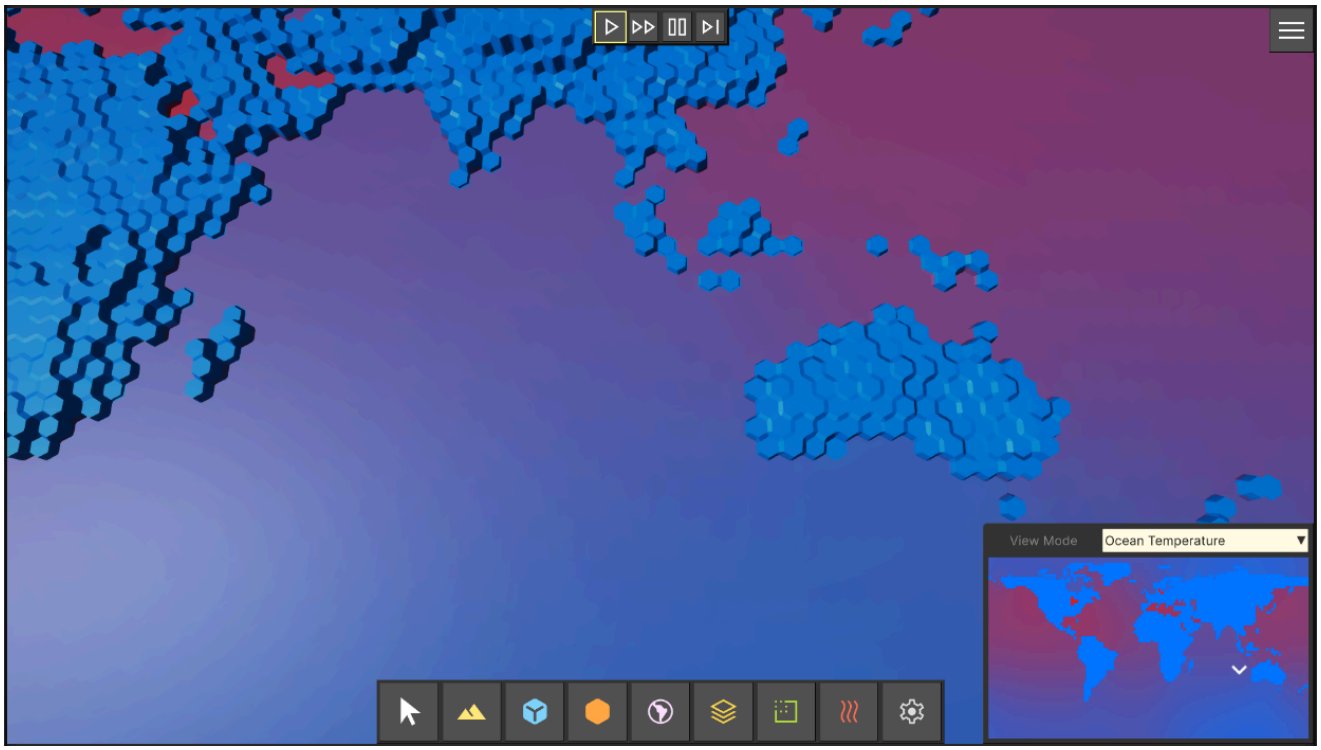
- Heighmap View Mode

- Provinces View Mode

- Ocean Heat View Mode

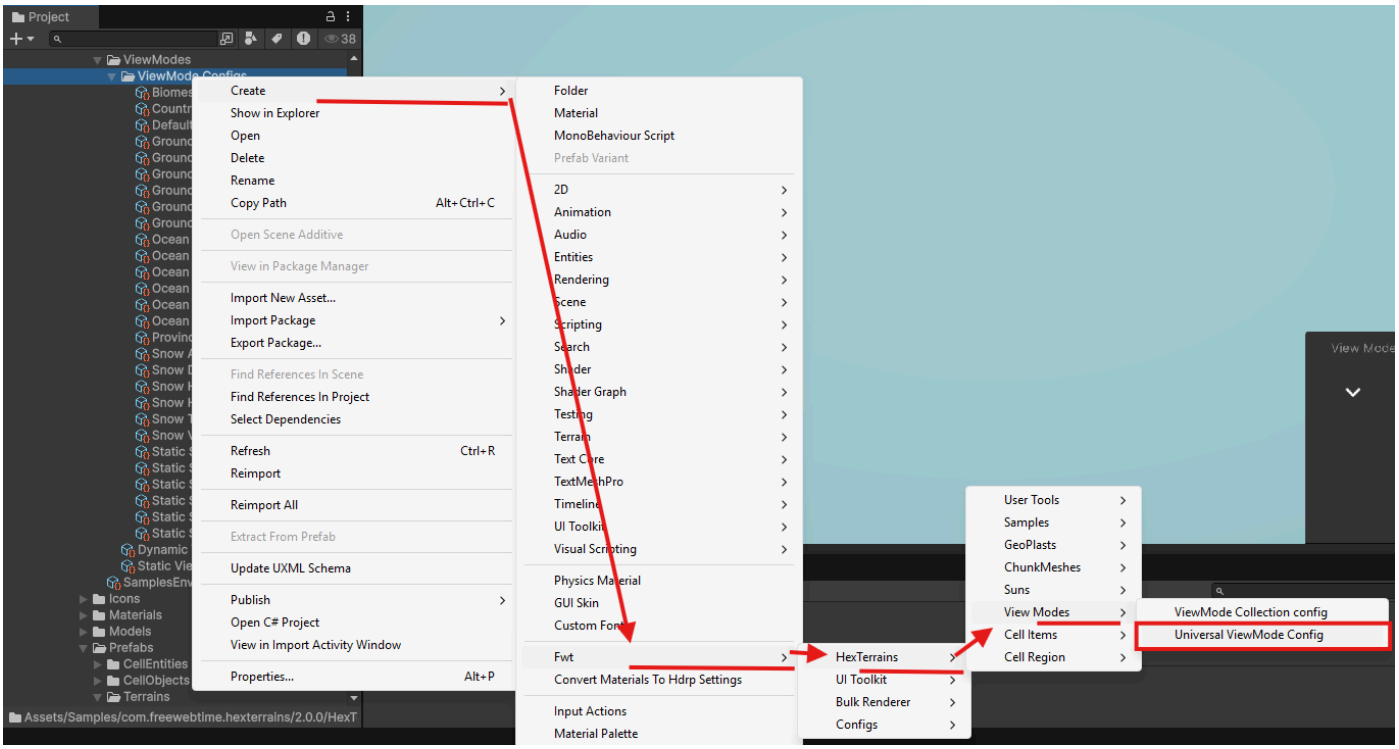


- Ocean Temperature View Mode

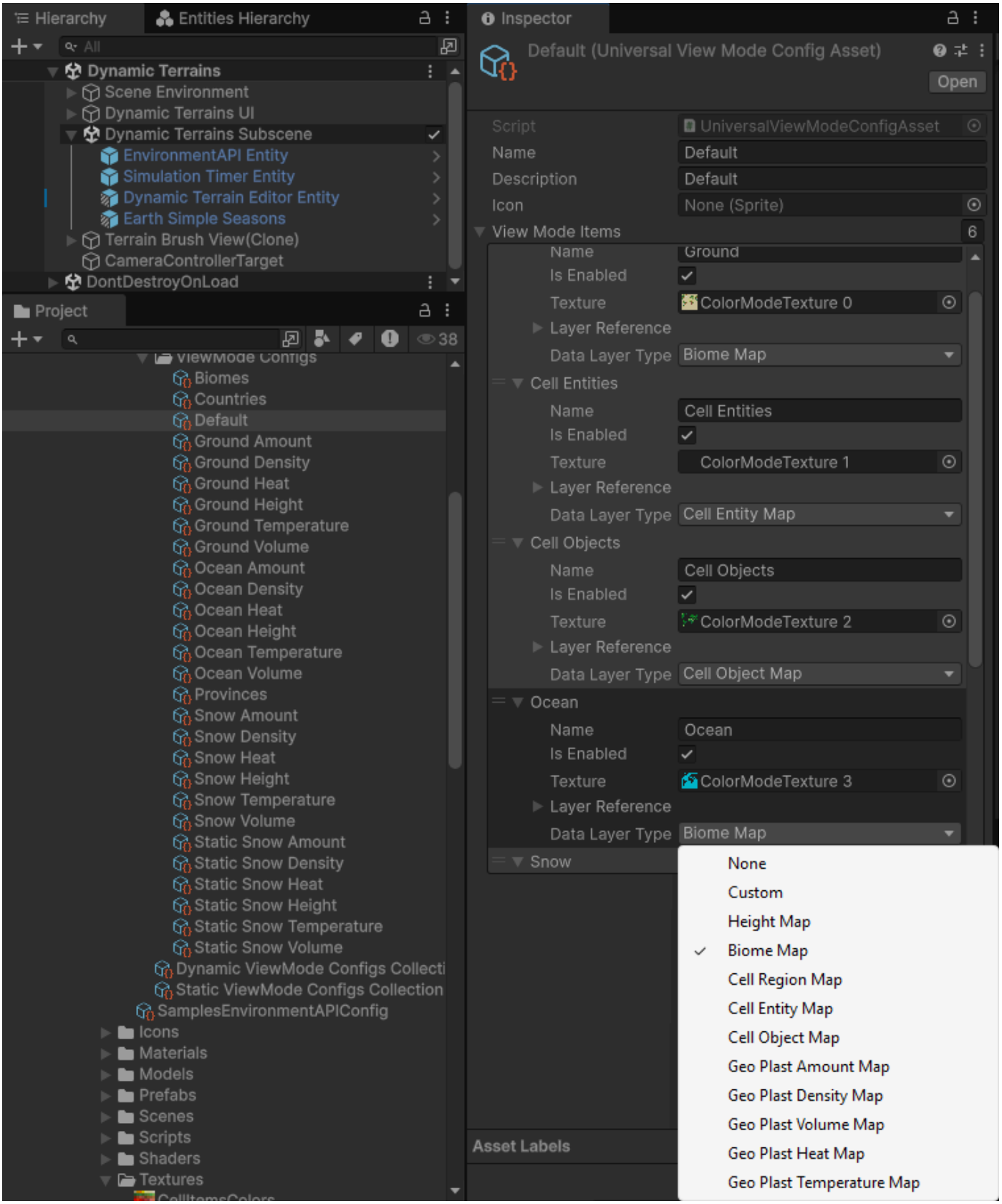


- And others (see "Assets/Samples/com.freewebtime.hexterrains/2.0.0/HexTerrains_Demo/Configs/ViewModes" folder)

To setup a ViewMode, create a new ViewMode asset scriptable object:



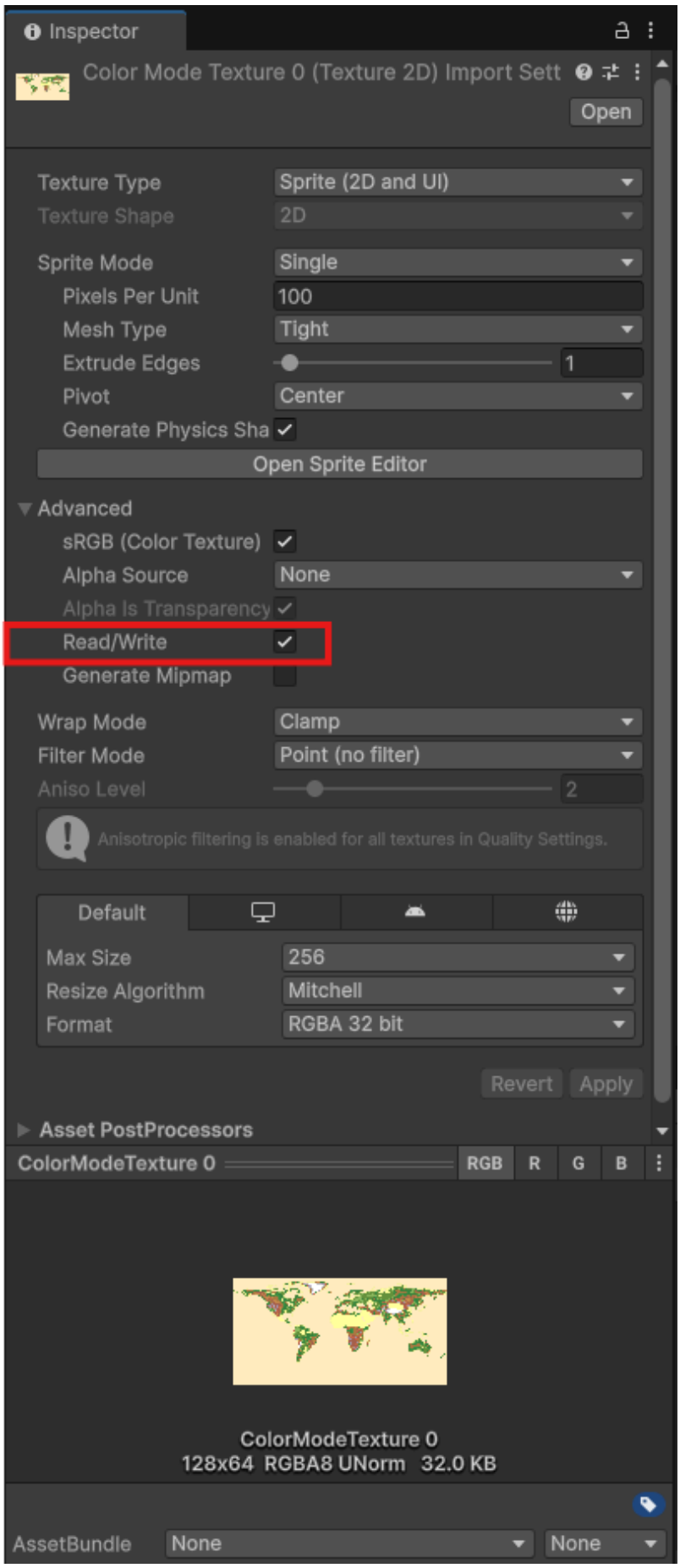
Configure the ViewMode asset:



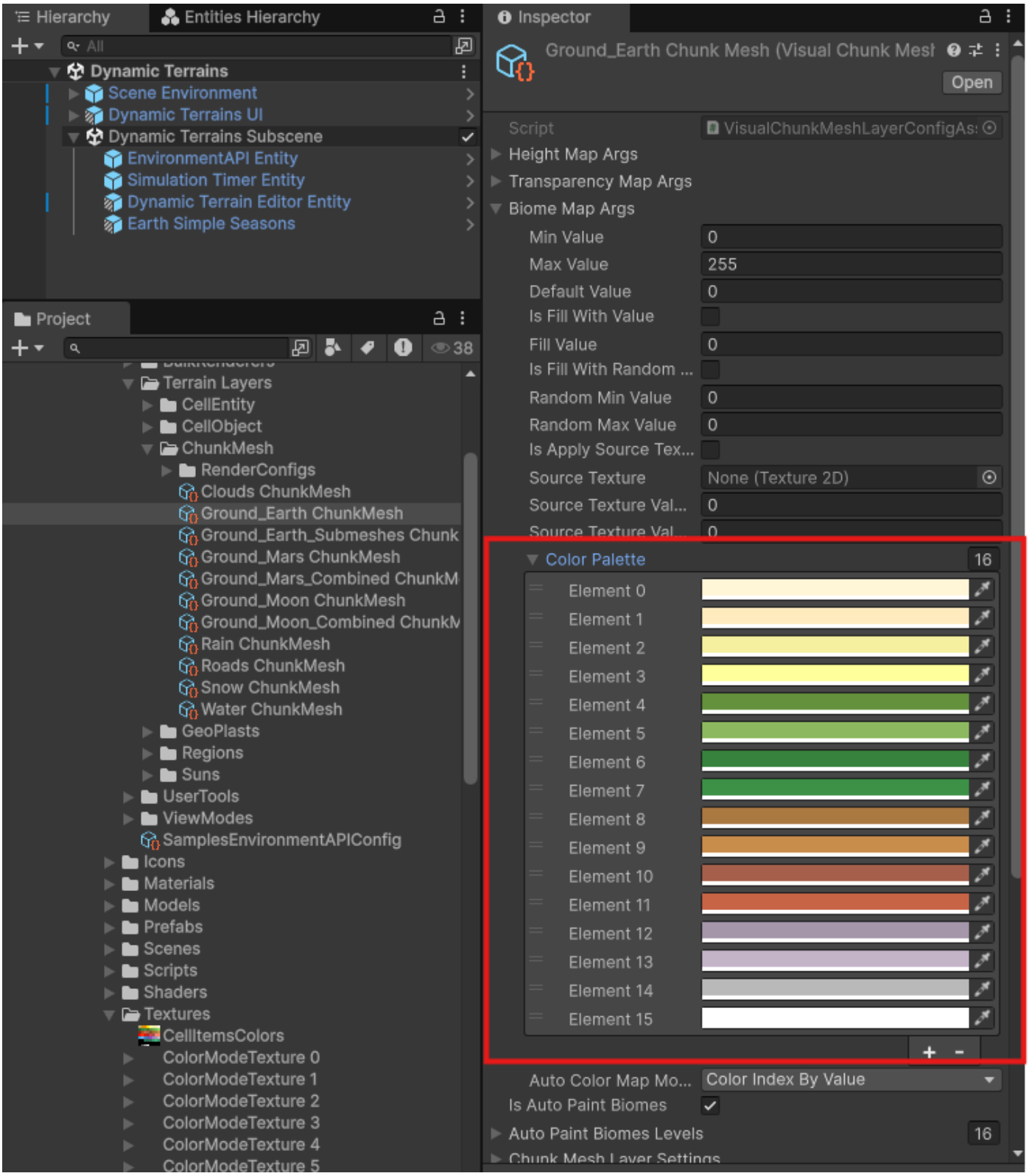
Add as much ColorMap layers as you need, and set which data should be displayed on each of them, and from which terrain layer to get the data.

For example, there is a "Default" View Mode that fills 6 textures with color data: Ground Biome, Cell Entities, Cell Objects, Ocean Biomes, Snow Biomes, Clouds Biomes. All 6 textures are filled with colors from respective data layers and then are sent to the Minimap UI where are displayed together one on top of another. See

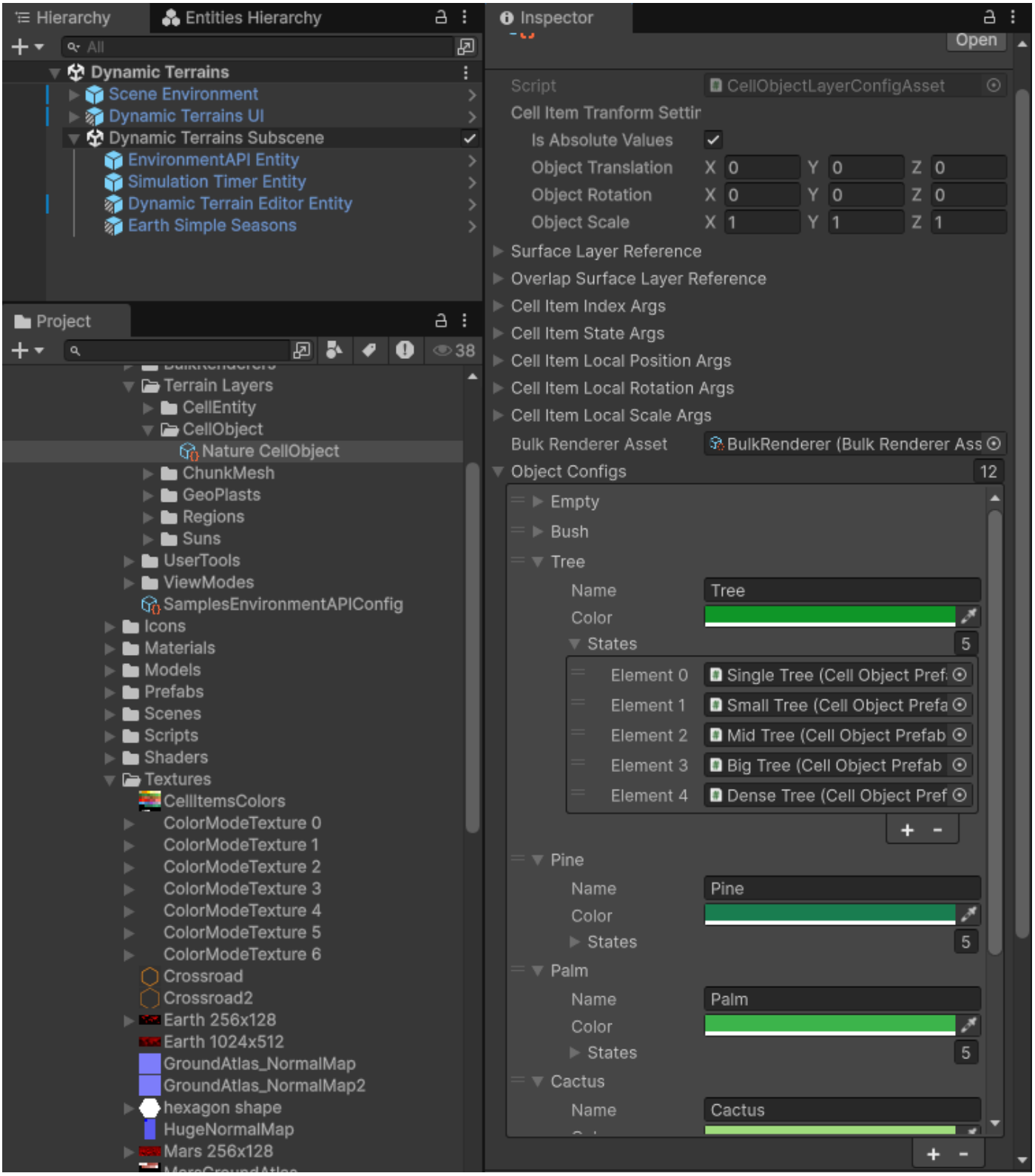
preconfigured ViewMode configs for more details. Note that textures that are filled with colors should have Read/Write option enabled in the texture import settings:



Most of data layers support a color map (they have a color map NativeArray and they fill a corresponding pixel with a color that respresents the current cell value). For example, the ChunkMesh has a BiomeMap data layer and each biome has it's own color:

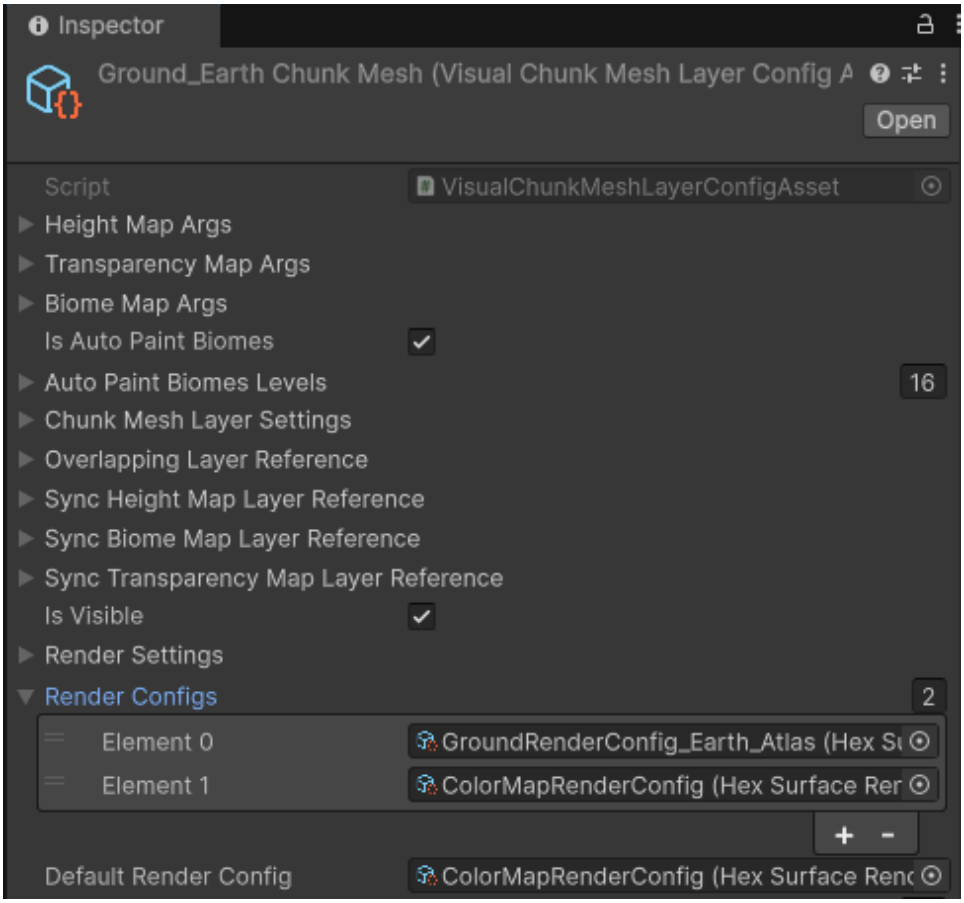


or, for example, the Color setup for each Cell Object:

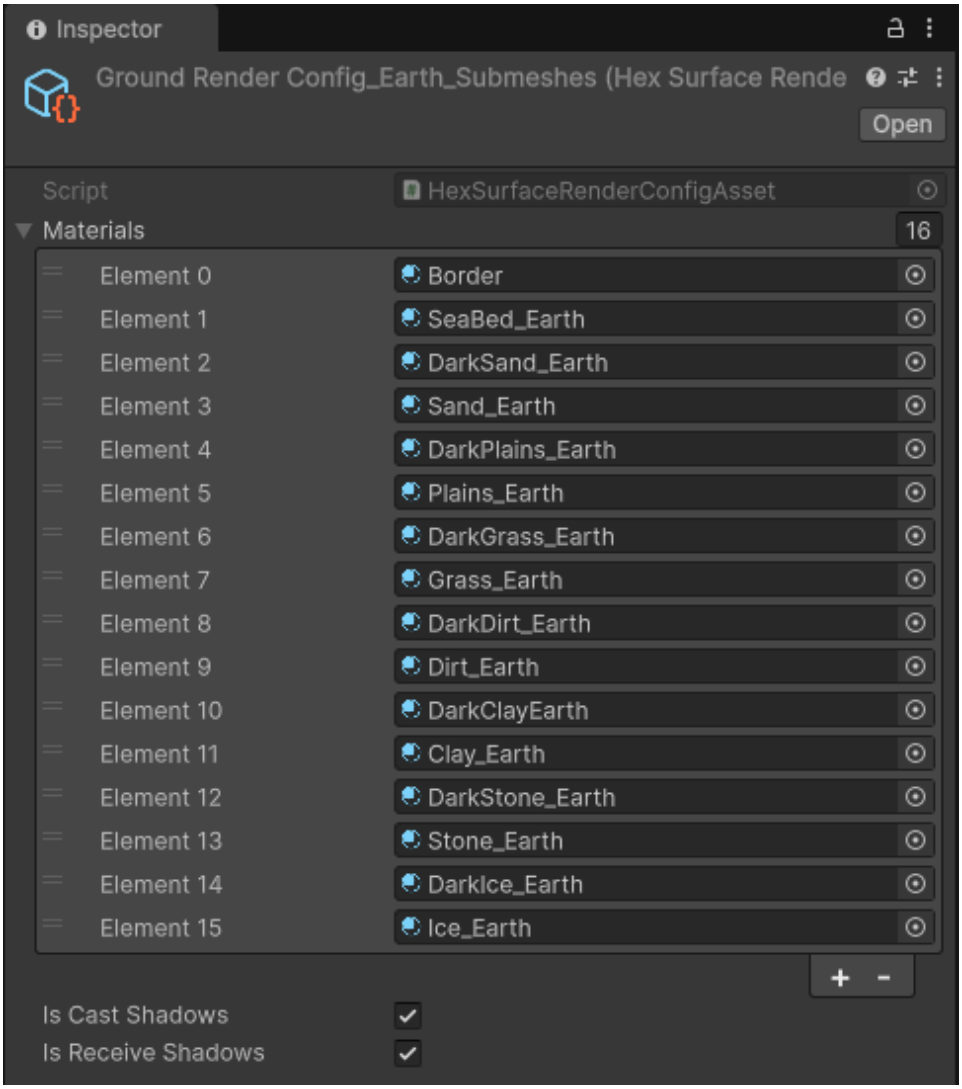


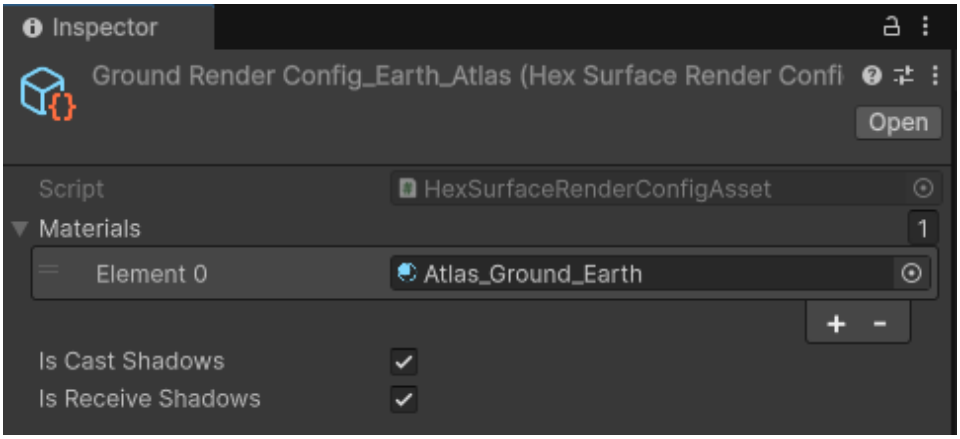
Terrain materials by View Mode

Take a look at configuration of one of your ChunkMesh Layer configs. For example, the Earth Ground Chunk Mesh Layer (`Assets/freewebtime/Samples/HexTerrains_Demo/Configs/Terrain Layers/ChunkMesh/Ground_Earth ChunkMesh`). You can see a `Render Configs` list and a `Default Render Config` . When ViewMode is 0, the ChunkMesh layer is rendered with `RenderConfigs[0]` , when ViewMode is 1, it uses `RenderConfigs[1]` , and so on. If there is no Render Config at index of a View Mode, the layer is rendered with `Default Render Config` . This way you can setup different looks in different View Modes.

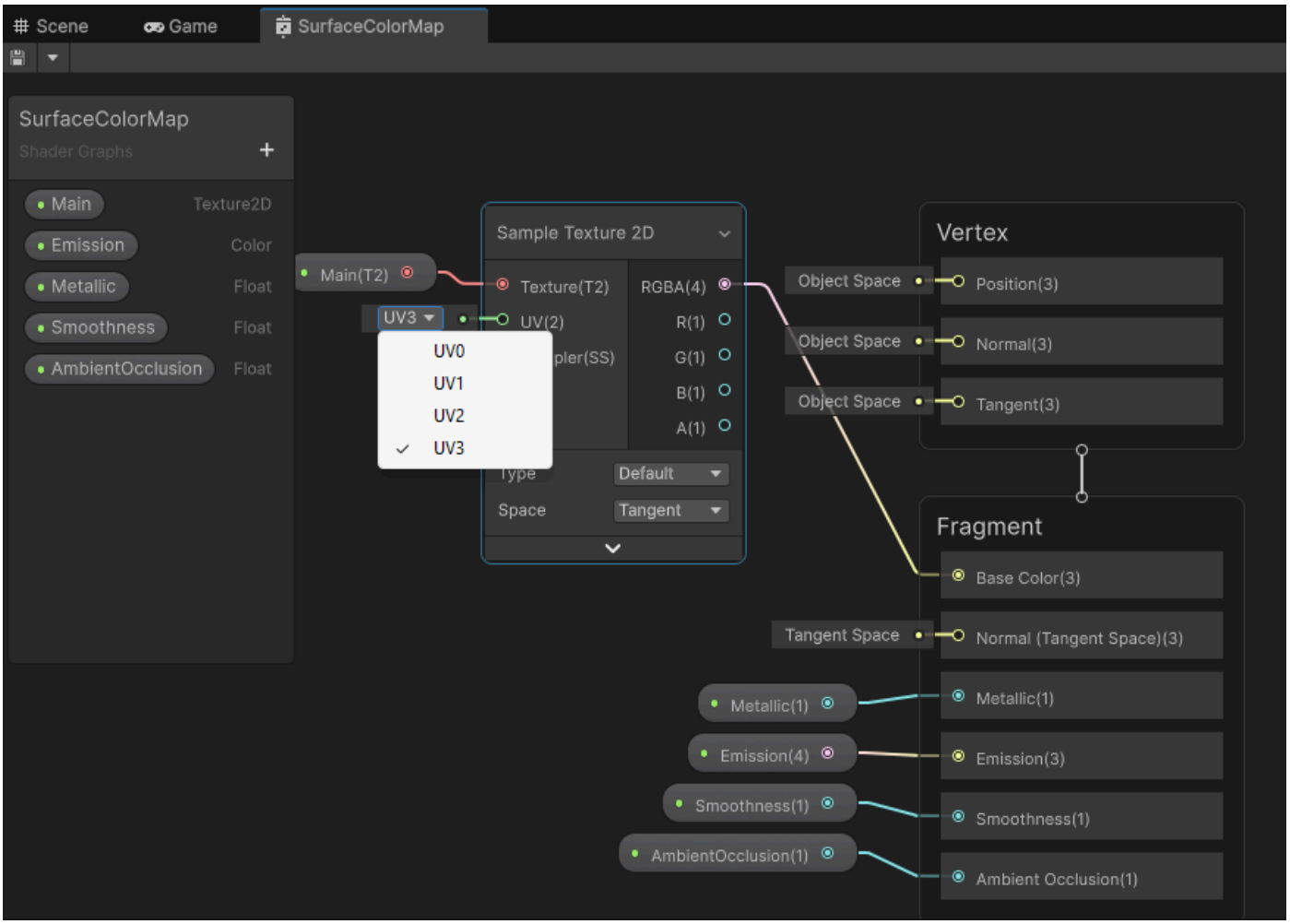


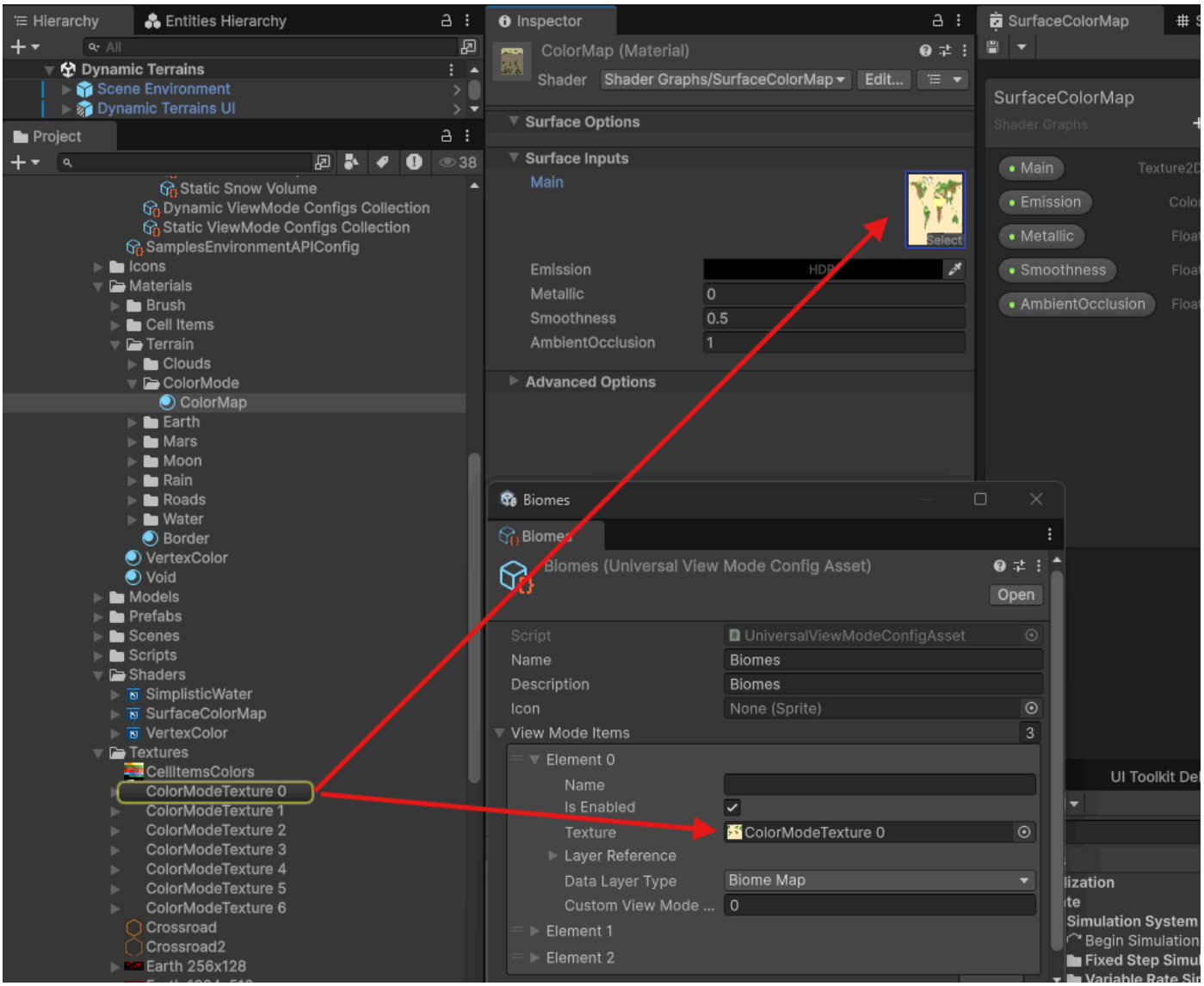
Render Config is a collection of Materials to be used for a generated Terrain Chunk Mesh (one material per submesh):





If you want to render a color map data on a terrain directly (the colors from Minimap on a terrain surface), in a terrain material for a specific ViewMode use the same texture that is used on Minimap UI, and make a material that uses UV3 (fourth UV set):

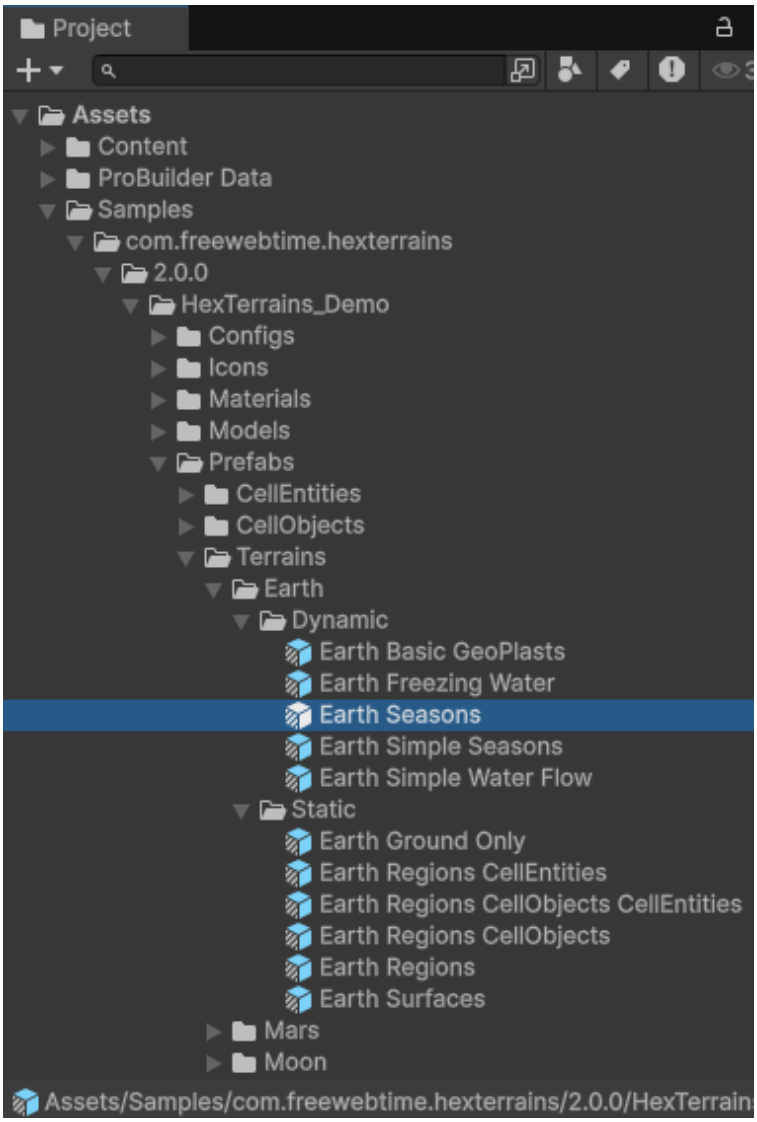

Scene
Game
SurfaceColorMap
SurfaceColorMap
Shader Graphs
Main
Texture2D
Emission
Color
Metallic
Float
Smoothness
Float
AmbientOcclusion
Float
Main(T2)
UV3
UV0
UV1
UV2
UV3
Sample Texture 2D
Texture(T2)
UV(2)
RGBA(4)
R(1)
G(1)
B(1)
A(1)
Type
Default
Space
Tangent
Vertex
Object Space
Position(3)
Object Space
Normal(3)
Object Space
Tangent(3)
Fragment
Base Color(3)
Tangent Space
Normal (Tangent Space)(3)
Metallic(1)
Metallic(1)
Emission(4)
Emission(3)
Smoothness(1)
Smoothness(1)
AmbientOcclusion(1)
Ambient Occlusion(1)



Create terrain on scene load

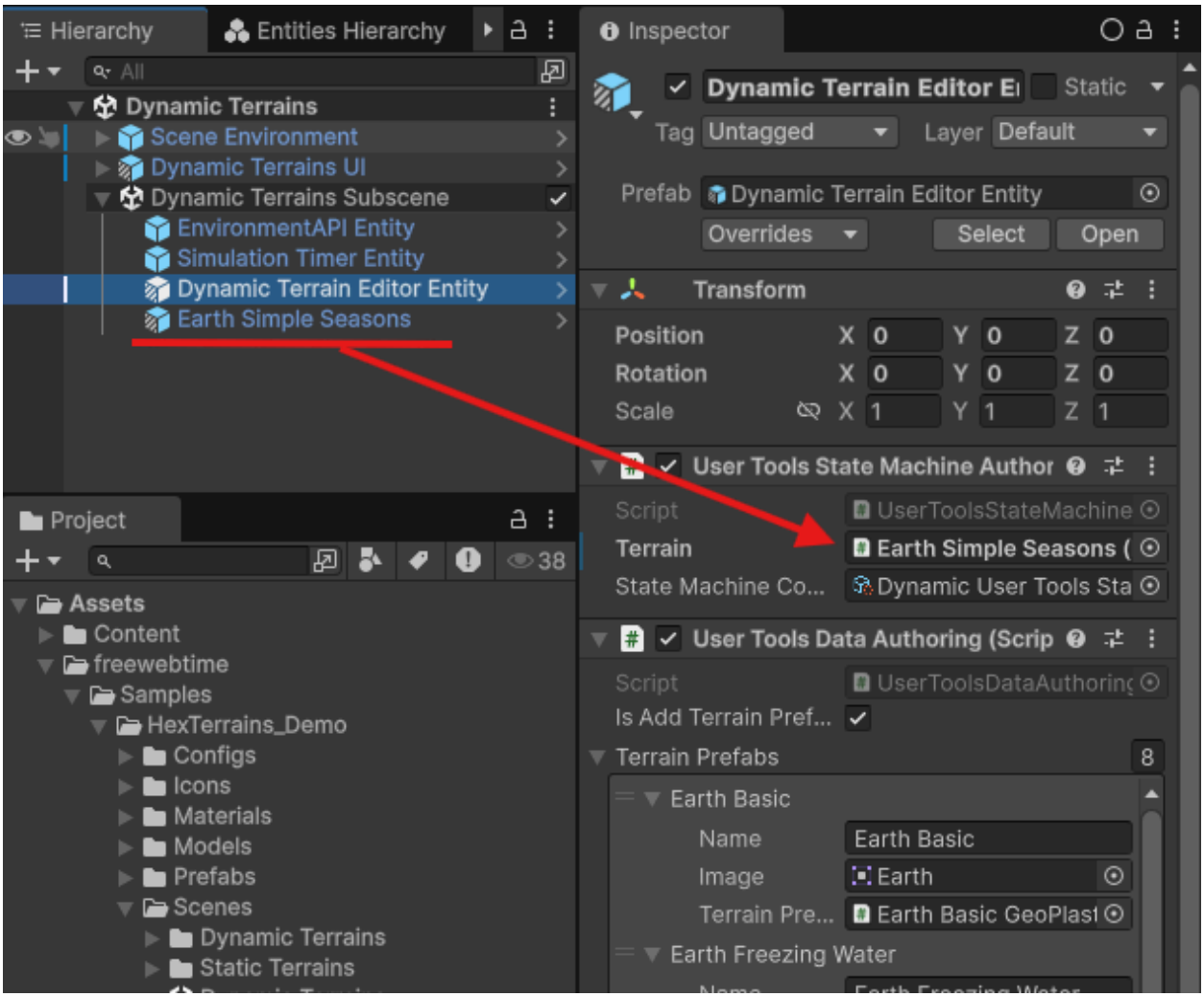
If you want to have a terrain loaded at scene load, just put on the subscene a terrain prefab. If you have a `HexTerrainEditor` prefab on the scene, set a reference to the terrain prefab in the `User Tools State Machine Authoring` script in the field `Terrain`.

You can find Terrain prefabs in the Samples folder by path "HexTerrains_Demo/Prefabs/Prefabs"



Put a prefab you want on the subscene. Now the terrain will be baked and appear on a scene in Play mode.

If you have a Terrain Editor on the subscene, set it a reference to the terrain authoring game object.



Note: if you don't set a reference to the terrain on the HexTerrainEditor, you will not be able to edit a terrain using the User Tools Panel at the bottom center of the screen.

Note: HexTerrainEditor prefab is optional. It is needed for having the User Tools Panel to be created. For your gameplay it might not be needed, so feel free to get rid of it.

How to interact with a Terrain

1. To see how it's done, take a look at interface `IHexTerrainAPI` and it's example implementation `HexTerrainAPI` class:

```
/// <summary>
/// Provides a comprehensive interface for interacting with hexagonal terrain systems, enabl
ing access to terrain
/// entities, layers, settings, and related UI and environment functionality.
/// </summary>
/// <remarks>The IHexTerrainAPI interface exposes methods and properties for managing and qu
erying various
/// aspects of a hex terrain, including cell data, terrain layers, brush controls, visibilit
y, and persistence. It
/// supports advanced operations such as layer manipulation, raycasting, prefab configuratio
n, and UI screen
/// management. Implementations should ensure thread safety and proper initialization of ter
rain entities and layers
/// before use. This interface is intended for use in applications that require dynamic and
granular control over
/// hex terrain features, such as editors or simulation tools.</remarks>
public interface IHexTerrainAPI
{
    ///...

    /// <summary>
    /// Returns a terrain layer of type <typeparamref name="TTerrainLayer"/> from the terrai
n layer group <typeparamref name="TTerrainLayerGroup"/> from the Terrain Entity
    /// </summary>
    /// <typeparam name="TTerrainLayer">Type of the Terrain layer to get from a layer group
</typeparam>
    /// <typeparam name="TTerrainLayerGroup">Type of the Terrain Layer Group to find on the
entity</typeparam>
    /// <param name="layerReference">reference data to needed layer</param>
    /// <returns>terrain layer if found and null otherwise</returns>
    TTerrainLayer GetTerrainLayer<TTerrainLayerGroup, TTerrainLayer>(HexTerrainLayerReferenc
e layerReference)
        where TTerrainLayer : HexTerrainLayer
        where TTerrainLayerGroup : HexTerrainLayerGroup
    {
        var layerGroup = GetTerrainLayer<TTerrainLayerGroup>();
        if (layerGroup != null)
        {
            return layerGroup.GetLayer<TTerrainLayer>(layerReference);
        }

        return null;
    }


    /// <summary>Returns the HeightMap value at the specified cell index from the specified
ChunkMeshLayer.
    /// </summary>
    /// <typeparam name="TLayerGroup">type of HexTerrainLayerGroup entity component to get t
he ChunkMeshLayer from</typeparam>
    /// <typeparam name="TLayer">type of ChunkMeshLayer from the HexTerrainLayerGroup to get
a cell value from</typeparam>
```

```
    /// <param name="cellIndex">cell index to get a value from</param>
    /// <param name="layerReference">reference to the layer to get a cell value from</param>
    /// <returns>value if found or null if not found</returns>
    float? GetCellHeight<TLayerGroup, TLayer>(int cellIndex, HexTerrainLayerReference layerR
eference)
        where TLayerGroup : HexTerrainLayerGroup, IComponentData
        where TLayer : ChunkMeshLayer;


    /// <summary>
    /// Sets the HeightMap value at the specified cell index at the specified ChunkMeshLaye
r.
    /// </summary>
    /// <typeparam name="TLayerGroup">type of HexTerrainLayerGroup entity component to get t
he ChunkMeshLayer from</typeparam>
    /// <typeparam name="TLayer">type of ChunkMeshLayer from the HexTerrainLayerGroup to set
a cell value at</typeparam>
    /// <param name="cellIndex">cell index to set a value at</param>
    /// <param name="layerReference">reference to the layer to set a cell value at</param>
    /// <param name="value">value to set</param>
    /// <returns>true if value has been set, otherwise - false</returns>
    bool SetCellHeight<TLayerGroup, TLayer>(int cellIndex, HexTerrainLayerReference layerRef
erence, float value)
        where TLayerGroup : HexTerrainLayerGroup, IComponentData
        where TLayer : ChunkMeshLayer;


    /// ...
}
```

In general all the interactions are done by accessing components on the Terrain Entity:

```
/// <summary>
/// returns a data layer from the TerrainEntity <see cref="TerrainEntity"/>
/// </summary>
/// <typeparam name="TDataLayer"></typeparam>
/// <returns></returns>
public virtual TDataLayer GetTerrainLayer<TDataLayer>() where TDataLayer : class
{
    if (!EntityManager.Exists(TerrainEntity) || !EntityManager.TryGetComponentObject<TDataLa
yer>(TerrainEntity, out var dataLayer))
    {
        return default;
    }
    return dataLayer;
}

/// <summary>Returns the HeightMap value at the specified cell index from the specified Chun
kMeshLayer.
/// </summary>
/// <typeparam name="TLayerGroup">type of HexTerrainLayerGroup entity component to get the C
hunkMeshLayer from</typeparam>
/// <typeparam name="TLayer">type of ChunkMeshLayer from the HexTerrainLayerGroup to get a c
ell value from</typeparam>
/// <param name="cellIndex">cell index to get a value from</param>
/// <param name="layerReference">reference to the layer to get a cell value from</param>
/// <returns>value if found or null if not found</returns>
public virtual float? GetCellHeight<TLayerGroup, TLayer>(int cellIndex, HexTerrainLayerRefer
ence layerReference)
    where TLayerGroup : HexTerrainLayerGroup, IComponentData
    where TLayer : ChunkMeshLayer
{
    var layersList = GetTerrainLayer<TLayerGroup>();
    if (layersList == null)
    {
        return default;
    }

    var terrainLayer = layersList.GetLayer<TLayer>(layerReference);
    if (terrainLayer == null)
    {
        return default;
    }

    var dataLayer = terrainLayer.HeightMap;

    if (!dataLayer.TryGetData(cellIndex, out var value))
    {
        return default;
    }

    return value;
}
```

```
/// <summary>
/// Sets the HeightMap value at the specified cell index at the specified ChunkMeshLayer.
/// </summary>
/// <typeparam name="TLayerGroup">type of HexTerrainLayerGroup entity component to get the C
hunkMeshLayer from</typeparam>
/// <typeparam name="TLayer">type of ChunkMeshLayer from the HexTerrainLayerGroup to set a c
ell value at</typeparam>
/// <param name="cellIndex">cell index to set a value at</param>
/// <param name="layerReference">reference to the layer to set a cell value at</param>
/// <param name="value">value to set</param>
/// <returns>true if value has been set, otherwise - false</returns>
public virtual bool SetCellHeight<TLayerGroup, TLayer>(int cellIndex, HexTerrainLayerReferen
ce layerReference, float value)
    where TLayerGroup : HexTerrainLayerGroup, IComponentData
    where TLayer : ChunkMeshLayer
{
    var layersList = GetTerrainLayer<TLayerGroup>();
    if (layersList == null)
    {
        return false;
    }
    var terrainLayer = layersList.GetLayer<TLayer>(layerReference);
    if (terrainLayer == null)
    {
        return false;
    }

    var dataLayer = terrainLayer.HeightMap;
    if (dataLayer == null)
    {
        return false;
    }

    return dataLayer.TrySetData(cellIndex, value);
}
```

HexTerrains Framework – Core Architecture Concept

Overview

At the heart of the HexTerrains Framework is a **single Terrain Entity**.
This entity does not directly store terrain logic or visuals. Instead, it acts as a **container and coordinator** for structured terrain layers.

The terrain is built using a three-level hierarchy:

```
Terrain Entity
└── Terrain Layer Groups
    └── Terrain Layers
```

This layered approach keeps the system modular, scalable, and easy to extend.

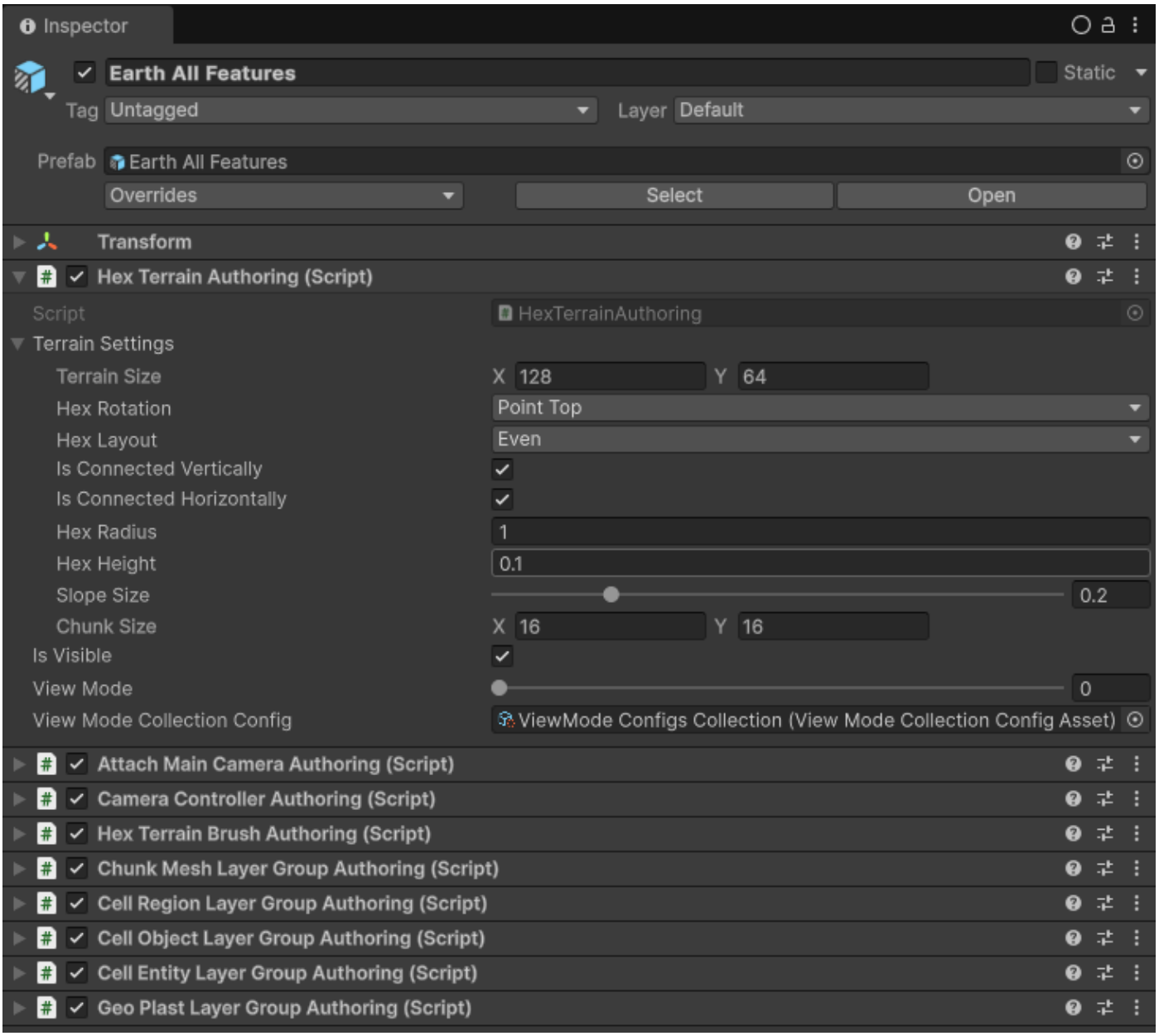
---

1. Terrain Entity

The **Terrain Entity** is a single ECS entity that represents the entire hex terrain.

It:

- Defines terrain dimensions (width, height, chunk size)
- Holds global settings
- Owns all terrain layer groups
- Acts as the root of all terrain-related systems



2. Terrain Layer Groups

A **Terrain Layer Group** is a logical category of terrain functionality.

Each group contains one or more related terrain layers.

Examples of possible layer groups:

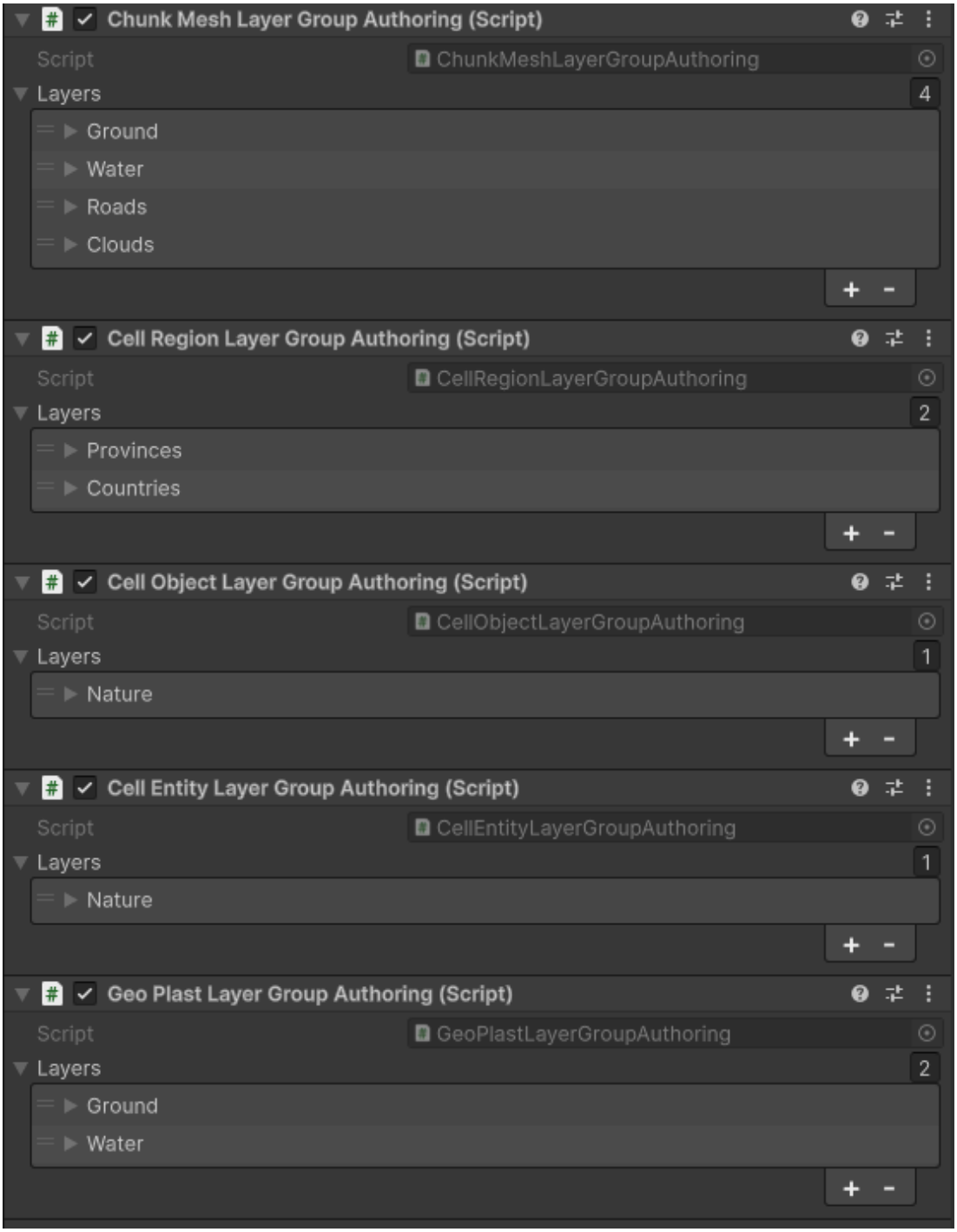
- ChunkMesh Layer Group (procedural generated mesh per chunk)
- CellObject Layer Group (static visual item per cell, instanced objects like trees)
- CellEntity Layer Group (ECS Entity per cell)
- CellRegion Layer Group (Region index per cell, like Country or Province)
- GeoPlast Layer Group (volumetric simulation like ground or water)

A layer group:

- Organizes related systems
- Controls update order within the group

- Manages shared behavior between its layers

Think of a Layer Group as a **department** inside the terrain.



---

3. Terrain Layers

A **Terrain Layer** is where actual terrain logic and data live.

Each layer represents a specific aspect of the terrain.

Examples:

- Ground surface layer
- Water surface layer

- Roads layer
- Nature CellObject layer
- Countries layer
- Water GeoPlast layer (water flow simulation)

A Terrain Layer typically contains:

- One or more Data Layers (per-cell data storage)
- Chunk-based dirty tracking
- Job scheduling logic
- Rendering or simulation logic

Layers are independent from each other but operate within their group's context.

> Think of a Terrain Layer as a **specialized system** responsible for one aspect of the terrain. For example, `ChunkMeshLayer` has a HeightMap (float value per cell), a BiomeMap (int value per cell), and a Transparency Map (bool value per cell). It generates a mesh per chunk and renders them on screen, so you can have a terrain surface. It is possible to have any amounts of layers inside the same `TerrainLayerGroup`, so you can have one layer for Ground, one layer for Water, one layer for Roads, one for Clouds, and so on.

Why This Structure?

This hierarchy provides:

- Clear separation of responsibilities
- High modularity
- Easy extensibility
- Controlled update order
- Safe data management
- Scalable simulation and rendering

Instead of building one monolithic terrain system, the framework builds many focused layers that work together under one Terrain Entity.

Mental Model Summary

- **Terrain Entity** → The world container
- **Layer Groups** → Categories of functionality
- **Terrain Layers** → Concrete implementations (rendering, objects, simulation)
- **Data Layers (inside layers)** → Actual per-cell data storage

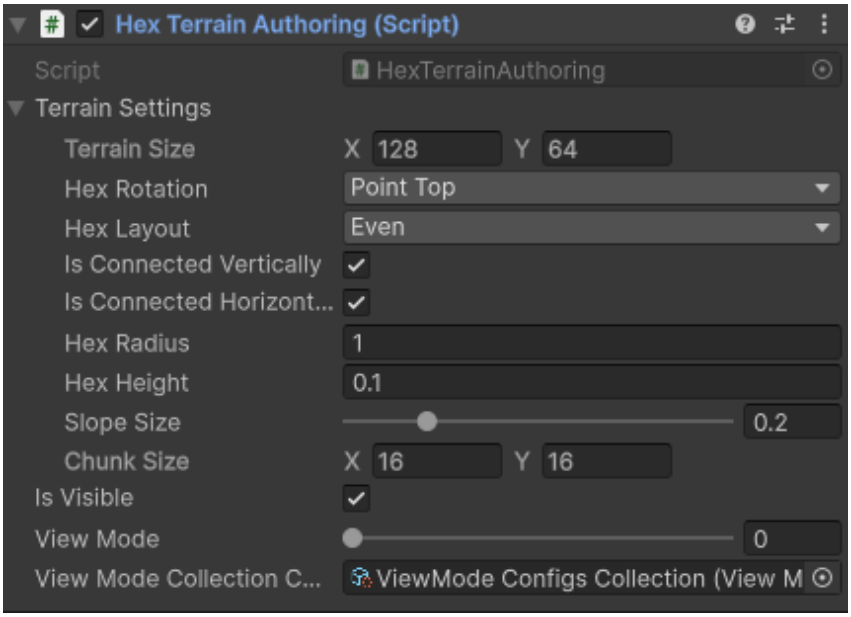
This structure allows the terrain to grow in complexity without becoming unmanageable.

How to create a Terrain Entity

Baking

Place a TerrainEntity authoring prefab game object on a subscene. Add all needed components to it:

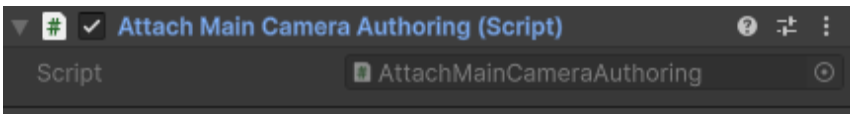
`HexTerrain`



This component contains terrain settings such as `TerrainSize` in cells (x, y), `HexRotation` (Point Top/Flat Top), `HexLayout` (Even/Odd), `HexRadius` (the lenght of the hexagon side), `HexHeight` (1 unit of height on HeightMap translates to this height of cell in meters, think of it as a vertical terrain scale), `SlopeSize` (share of the hex radius dedicated to slope between two adjacent cells), `IsConnectedVertically` and `IsConnectedHorizontally` flags (allow to connect opposite sides of the terrain, to create an illusion of a round planet like in Civilization), and `ChunkSize` (width and height of one terrain chunk in cells. 16x16 is an optimal value, as the Unity engine has a hard limit for the amount of vertices per mesh, and the ChunkMeshLayer generates one mesh per chunk. So depending on the geometry of your chunks the amount of vertexes per cell can easilly make your mesh big enough to go over that limit).

`AttachMainCamera`

> Optional component



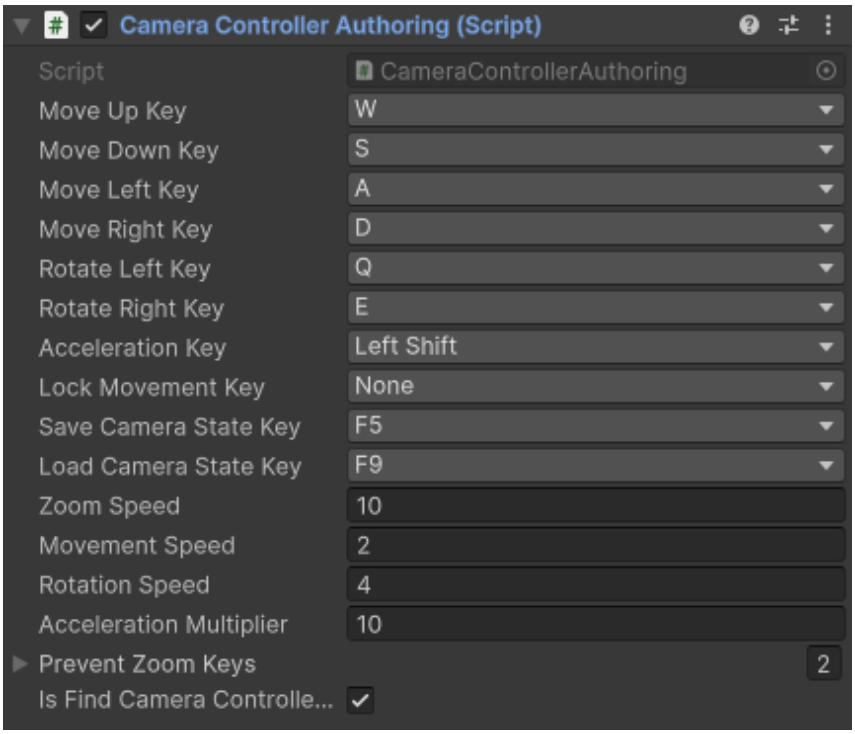
Attaches a Camera.main to the Terrain Entity on Initialization phase in runtime. In order to calculate visible chunks, the `ChunksGridLayer` needs a Camera instance attached to the Terrain Entity (to test chunk visibility inside the Camera Frustum).

If your logic requires different camera to be attached or no camera at all, don't add this component. If you need another camera to be attached, create your own component for that and check `AttachMainCameraToTerrainSystem` to learn how the camera is attached to the Terrain Entity:

```
[UpdateInGroup(typeof(CreateHexTerrainsGroup))]
public partial class AttachMainCameraToTerrainSystem : SystemBase
{
    private EntityQuery _terrainsQuery;

    protected override void OnCreate()
    {
        base.OnCreate();

        _terrainsQuery = GetEntityQuery(
            ComponentType.ReadOnly<AttachMainCameraRequest>(),
            ComponentType.Exclude<Camera>()
        );

        RequireForUpdate(_terrainsQuery);
    }

    protected override void OnUpdate()
    {
        using var ecb = new EntityCommandBuffer(Unity.Collections.Allocator.Temp);
        var camera = Camera.main;

        ecb.AddComponentObject(_terrainsQuery, camera);
        ecb.RemoveComponent<AttachMainCameraRequest>(_terrainsQuery, EntityQueryCaptureMode.
AtPlayback);

        ecb.Playback(EntityManager);
    }
}
```

CameraController

> Optional component

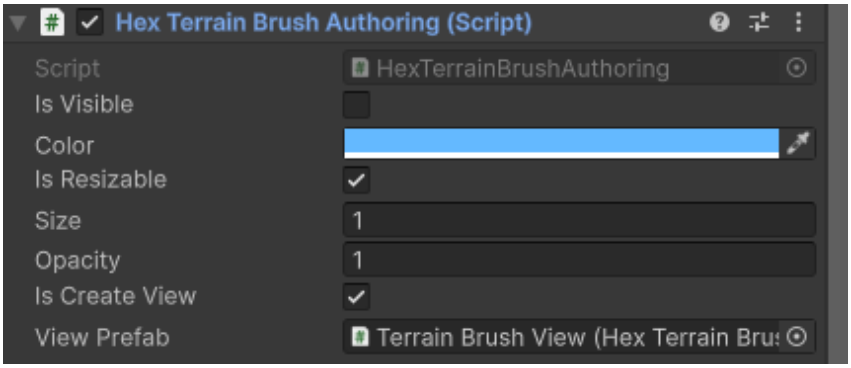


This component adds the default Camera Controller to the terrain that allows to control a position of the Camera attached to the Terrain Entity.

If you have your own implementation of the Camera Controller, or you simply don't need the default camera controller, don't add this component.

HexTerrainBrush

> Optional component

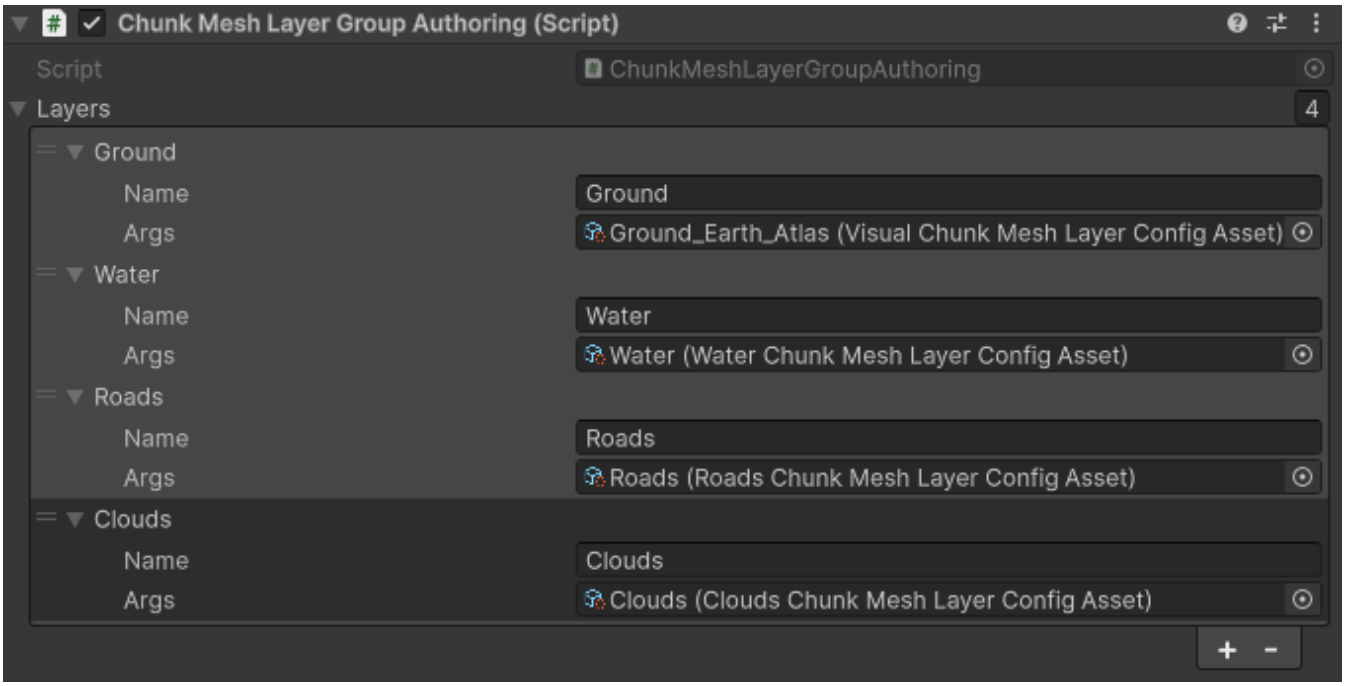


This component adds a 'Brush' that highlights the cells under cursor. Default implementation uses the `DecalProjector` to project a highlight on the terrain (see `View Prefab` on authoring component)

If you don't need a brush on the terrain, or you have your own implementation, don't add this component

ChunkMeshLayerGroup

> Optional component



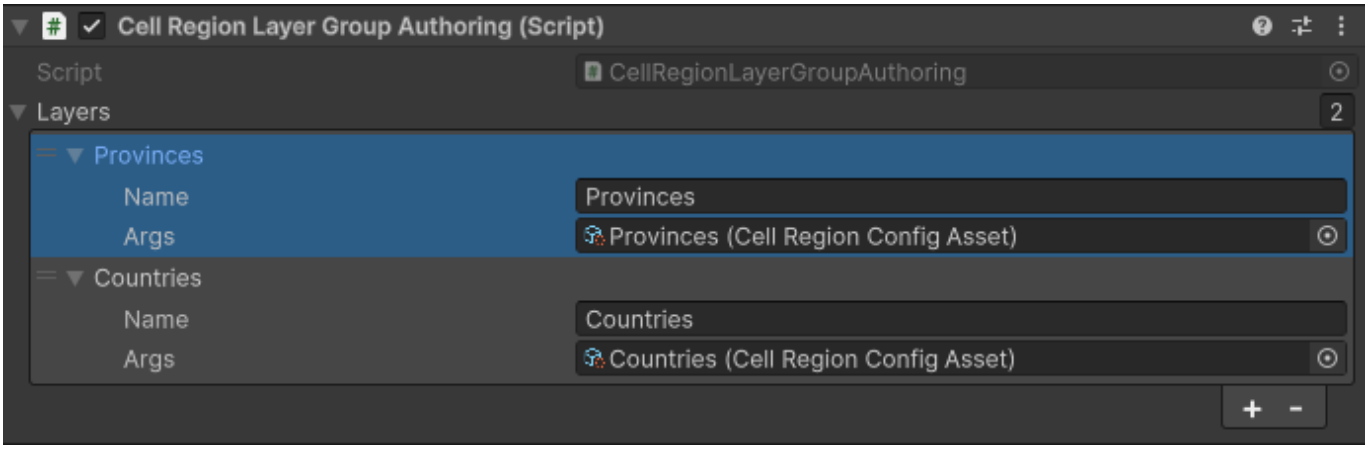
This component adds a `ChunkMeshLayerGroup` component to the TerrainEntity. This LayerGroup holds a list of `ChunkMeshLayer` layers such as TerrainSurface, Clouds, Roads, etc.

Each ChunkMeshLayer has a `HeightMap` (float per cell), `BiomeMap` (int per cell), and `TransparencyMap` (bool per cell). It also has a `CellMetricsMap` that holds precalculated data for each cell such as local and global center position, cell height, chunk index, chunk position, etc. This data is used by mesh generation as well as by other layers such as `CellObjectLayer` or `CellEntityLayer` to cell metrics dependent calculation. So `CellMetrics` are generated once and then used in different places.

This component is optional, but some other layers depend on it, such as CellObjectLayer and CellEntityLayer - for calculating the transforms for their items.

`CellRegionLayerGroup`

> Optional component



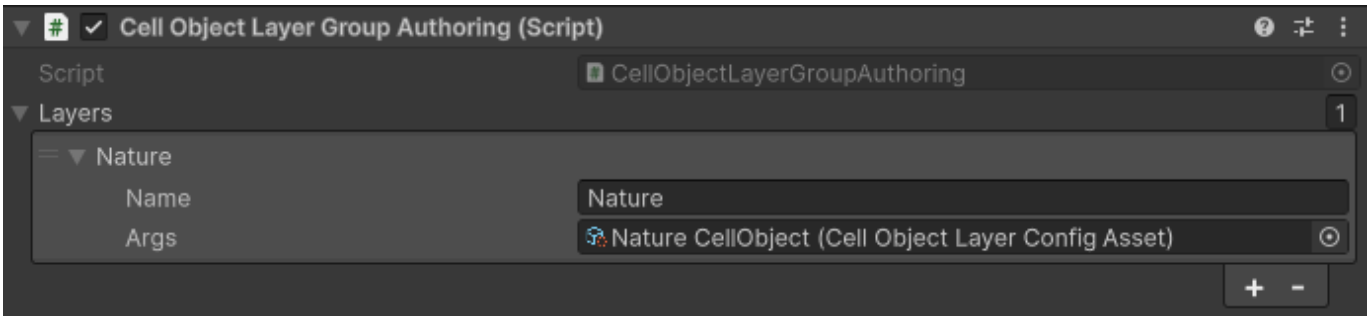
This component adds a `CellRegionLayerGroup` component to the Terrain Entity. This LayerGroup holds a list of `CellRegionLayer` layers such as Provinces, Countries, etc. `CellRegionLayer` has a `CellRegionMap` (int per cell), `RegionCells` (`NativeParallelMultiHashMap<int, uint>` where key is a region index and values is a collection of

cell indexes of the cells that are belong to that region), `RegionSizes` (uint per cell where index is a region index and value is the amount of cells of that region). `RegionCells` and `RegionSizes` are recalculated when `CellRegionMap` values change. `CellRegionMap` is a data layer with embedded color map, so whenever the data is changed, the colors in that color map are updated. That colors can be copied into the `Texture2D` instance to be displayed on UI or in terrain material to color the terrain cells into the respective colors (for example, to paint each cell into the color of the country that cell belongs to).

This component is totally optional and in default implementation no other layers depend on it.

`CellObjectLayerGroup`

> Optional component



This component adds a `CellObjectLayerGroup` component to the Terrain Entity. This LayerGroup holds a list of `CellObjectLayer` layers such as Trees, Buildings, etc.

`CellObjectLayer` has `ItemIndexMap` (int per cell), `ItemStateMap` (int per cell), `ItemPositionMap` (float3 per cell), `ItemRotationMap` (quaternion per cell), `ItemScaleMap` (float3 per cell), and `ItemTransformMap` (Matrix4x4 per cell). When `ItemPositionMap` or `ItemRotationMap` or `ItemScaleMap` data is changed, the `ItemTransformMap` is recalculated, so it always contains the transform matrix for each cell for the Cell Items to render at.

This layer holds a palette for Cell Items. Each CellItem index and CellItemState correspond to an item in the palette. So if in a ItemIndexMap[cell] is 1 and ItemStateMap[cell] is 3, on this cell will be rendered an item from palette for index 1 and state 3. Rendering is done using `Graphics.DrawMeshInstanced` .

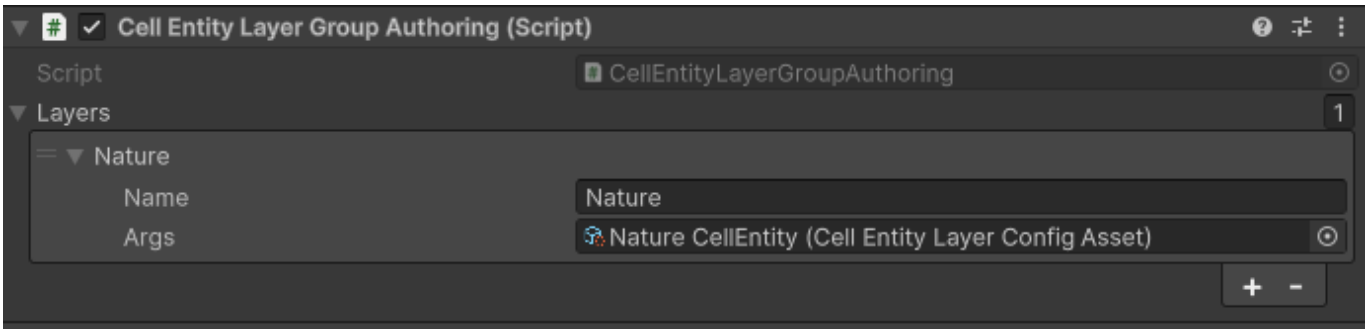
This layer supports LODs (see the CellObject prefab).

> This layer exists to solve a problem with inability to have millions of Entities in Unity. If you have a huge terrain with millions of cells, and you spawn an Entity in each cell, the Unity may crash. To solve this problem a `CellObject` mecanic was created. It can hold as much items as you need as long as you have memory for them. Cell Objects are more lightweight that Entities and run faster, but provide only static object(s) per cell without a hierarchy, no animation, etc. Cell Objects can co-exist with Cell Entity on the same terrain without any problems.

This layer is optional. In default implementation no other layers depend on it.

CellEntityLayerGroup

> Optional component



This component adds a `CellEntityLayerGroup` component to the Terrain Entity. This LayerGroup holds a list of `CellEntityLayer` layers such as Trees, Buildings, etc.

`CellEntityLayer` is very similar to the `CellObjectLayer`, but instead of rendering cell items using `Graphics.DrawMeshInstanced`, this layer spawns Entity per cell from palette according to the `ItemIndexMap` value and `ItemStateMap` value.

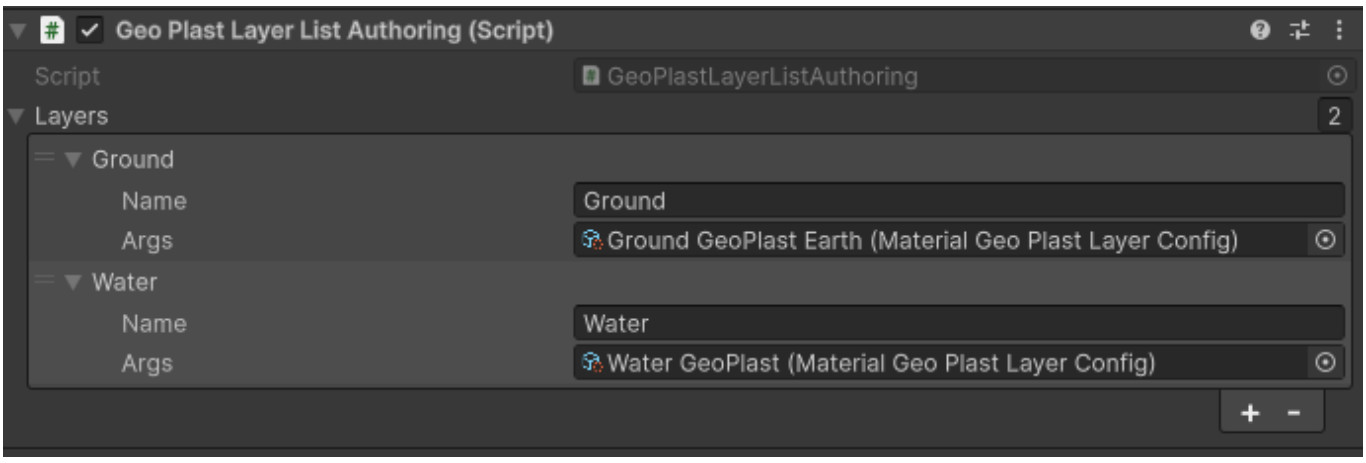
This layer holds a palette for Cell Items. Each CellItemIndex and CellItemState correspond to an Entity prefab in the palette. So if in a ItemIndexMap[cell] is 1 and ItemStateMap[cell] is 3, on this cell will be placed an Entity from palette for index 1 and state 3.

> Note that Unity may crash if you create millions of Entities, so if you have a huge terrain and you create an entity for each cell, keep this restriction in mind. The `CellObjectLayer` exists to fix this problem, as it can hold as many items as you want as long as you have memory for them.

This layer is optional. In default implementation no other layers depend on it.

GeoPlastLayerGroup

> Optional component



This component adds a `GeoPlastLayerGroup` to the Terrain Entity. This LayerGroup holds a list of `GeoPlastLayer` layers such as Ground, Water, Air, etc.

`GeoPlastLayer` has `AmountMap` (float per cell), `VolumeMap` (float per cell), `DensityMap` (float per cell), `BedrockHeightMap` (float per cell), `CeilingHeightMap` (float per cell). When `AmountMap` values or `DensityMap` values are changed, the `VolumeMap` is recalculated. Volume is calculated by multiplying the plast amount with it's density in the same cell. Whenever the `BedrockHeightMap` is changed or `VolumeMap` is changed, the `CeilingHeightMap` is recalculated. Ceiling height is Bedrock height + Volume in each cell.

It is possible to set a bedrock reference to another `GeoPlastLayer`. In this case the Bedrock height of the layer is equals to the Ceiling height of the bedrock layer. This allows to place a layer above layer. So, for instance, if you have a Water GeoPlast layer and it's bedrock is a Ground GeoPlast layer, the water is always above the ground, and when the ground height is changed, the water automatically adjusts.

If a `GeoPlastLayer` is a `DynamicGeoPlastLayer`, it has an ability to flow. Flow is an exchange of volume between adjacent cells in attempt to equalize Ceiling height. This naturally creates a realistic flow of the GeoPlast volume, so, for example, water flows downhill, filling the valleys on the ground.

> There is also an ability to set a vertical flow targets (up and down) to exchange volume between GeoPlast layers. For instance, to simulate evaporation from water to clouds or to simulate condensation from clouds back to water. Default implementation of the DynamicGeoPlastLayer does not calculate a vertical flow, but there is a MaterialGeoPlastLayer that has an implementation of the vertical flow depending on the Temperature (so you can have a freezing/evaporating water, melting snow, etc).

Runtime

Initialization process

Because the Terrain Layers are managed IComponentData components, it is not possible to bake them directly on Entity. So instead baking adds the CreateLayerRequest components.

For example, the ChunkMeshLayerGroupBaker adds a `DynamicBuffer<CreateChunkMeshLayerRequest>`:

```
public struct CreateChunkMeshLayerRequest : IBufferElementData, ITerrainLayerFactory
{
    public FixedString128Bytes Name;
    public UnityObjectRef<ChunkMeshLayerConfigAsset> Args;

    public HexTerrainLayer CreateTerrainLayer()
    {
        var config = Args.Value;
        if (config == null)
        {
            return default;
        }

        return config.CreateTerrainLayer();
    }
}
```

The `ChunkMeshLayerConfigAsset` here is a `ScriptableObject` that implements an `IChunkMeshLayerConfig` interface. This way the parameters of the `ChunkMeshLayer` is set on that `ScriptableObject` asset, and it is passed to the runtime world thorugh baking the buffer of requests.

When the `CreateChunkMeshLayerGroupSystem` sees the entity with this DynamicBuffer of requests, it creates a `ChunkMeshLayerGroup` component, puts it on Terrain Entity and then creates and initializes as much ChunkMeshLayers as much requests are in this DynamicBuffer. After initialization the requests DynamicBuffer is removed from Terrain Entity.

The same approach is used for all the Terrain Layers.

> Note that you don't have to use a `ScriptableObject` to initialize a TerrainLayer, you can use any class or struct you want as long as it implements the required interface. In case of `ChunkMeshLayer` it has to implement the `IChunkMeshLayerConfig`, in case of `CellEntityLayer` it must be `ICellEntityLayerConfig`, and so on.

Creating a Terrain Entity in runtime

If you don't bake your Terrain Entity from a scene, but want to create a Terrain in runtime, you have two options:

1. Instantiate a baked entity prefab with Create Layers Requests on it.
2. Instantiate an empty entity and manually add all the components to it. You can add CreateLayerRequests as baker does and then wait for one frame so the initialization systems create and initialize layers. Or alternatively you can create the TerrainLayerGroup components in your code, create and insert all the layers inside them, and add the LayerGroups to your entity.

See the code of Bakers of the authoring components to learn which components you have to add for the initialization process (Create Terrain Layer Requests processing):

Non-optional components:

```
public class HexTerrainBaker : Baker<HexTerrainAuthoring>
{
    public override void Bake(HexTerrainAuthoring authoring)
    {
        var terrainEntity = GetEntity(authoring, TransformUsageFlags.Dynamic);
        var terrainSettings = authoring.TerrainSettings;

        // terrain
        AddComponent(terrainEntity, new HexTerrain
        {
            Settings = terrainSettings
        });

        AddComponent(terrainEntity, new HexTerrainRaycastData());

        AddComponent(terrainEntity, new HexTerrainViewMode
        {
            Value = authoring.ViewMode
        });

        AddComponent(terrainEntity, new HexTerrainVisibility
        {
            Value = authoring.IsVisible
        });
        if (authoring.ViewModeCollectionConfig != null)
        {
            AddComponent(terrainEntity, new ViewModeCollectionConfigComponent
            {
                ConfigAsset = authoring.ViewModeCollectionConfig
            });
        }

        AddComponent(terrainEntity, new CreateChunksGridLayerRequest());

        AddComponent(terrainEntity, new NotInitializedTerrain
        {
            IsSetViewMode = true,
            ViewMode = authoring.ViewMode
        });
    }
}
```

Optional components.

Camera controller:

```
public class CameraControllerBaker : Baker<CameraControllerAuthoring>
{
    public override void Bake(CameraControllerAuthoring authoring)
    {
        var entity = GetEntity(TransformUsageFlags.Dynamic);
        AddComponent(entity, new CreateCameraControllerRequest
        {
            MoveUpKey = authoring.MoveUpKey,
            MoveDownKey = authoring.MoveDownKey,
            MoveLeftKey = authoring.MoveLeftKey,
            MoveRightKey = authoring.MoveRightKey,
            AccelerationKey = authoring.AccelerationKey,
            RotateLeftKey = authoring.RotateLeftKey,
            RotateRightKey = authoring.RotateRightKey,
            LockMovementKey = authoring.LockMovementKey,
            SaveCameraStateKey = authoring.SaveCameraStateKey,
            LoadCameraStateKey = authoring.LoadCameraStateKey,

            ZoomSpeed = authoring.ZoomSpeed,
            MovementSpeed = authoring.MovementSpeed,
            RotationSpeed = authoring.RotationSpeed,
            AccelerationMultiplier = authoring.AccelerationMultiplier,

            IsFindCameraControllerComponentOnScene = authoring.IsFindCameraControllerCompone
ntOnScene
        });

        var ignoreInputBuf = AddBuffer<IgnoreCameraZoomInputKeys>(entity);
        if (authoring.PreventZoomKeys != null)
        {
            for (var kIndex = 0; kIndex < authoring.PreventZoomKeys.Count; kIndex++)
            {
                ignoreInputBuf.Add(new IgnoreCameraZoomInputKeys
                {
                    Value = authoring.PreventZoomKeys[kIndex]
                });
            }
        }
    }
}
```

Attach main camera request:

```
public class AttachMainCameraBaker : Baker<AttachMainCameraAuthoring>
{
    public override void Bake(AttachMainCameraAuthoring authoring)
    {
        var entity = GetEntity(TransformUsageFlags.None);
        AddComponent(entity, new AttachMainCameraRequest());
    }
}
```

HexTerrainBrush:

```
public class HexTerrainBrushBaker : Baker<HexTerrainBrushAuthoring>
{
    public override void Bake(HexTerrainBrushAuthoring authoring)
    {
        var entity = GetEntity(TransformUsageFlags.Dynamic);

        AddComponent(entity, new HexTerrainBrush
        {
            Color = authoring.Color,
            IsResizable = authoring.IsResizable,
            IsVisible = authoring.IsVisible,
            Size = authoring.Size,
            Opacity = authoring.Opacity
        });

        if (authoring.IsCreateView && authoring.ViewPrefab != null)
        {
            var request = new CreateHexTerrainBrushViewRequest
            {
                Prefab = new UnityObjectRef<GameObject>
                {
                    Value = authoring.ViewPrefab.gameObject
                }
            };
            AddComponent(entity, request);
        }
    }
}
```

ChunkMeshLayerGroup:

```
public class ChunkMeshLayerGroupBaker : Baker<ChunkMeshLayerGroupAuthoring>
{
    public override void Bake(ChunkMeshLayerGroupAuthoring authoring)
    {
        var terrainEntity = GetEntity(TransformUsageFlags.None);

        var createRequests = AddBuffer<CreateChunkMeshLayerRequest>(terrainEntity);
        if (authoring.Layers != null)
        {
            for (var lIndex = 0; lIndex < authoring.Layers.Count; lIndex++)
            {
                var layerAuthoring = authoring.Layers[lIndex];
                if (layerAuthoring.Args == null)
                {
                    Debug.LogError($"Layer {layerAuthoring.Name} has no Args defined.");
                    continue;
                }

                var request = new CreateChunkMeshLayerRequest
                {
                    Name = layerAuthoring.Name,
                    Args = layerAuthoring.Args
                };
                createRequests.Add(request);
            }
        }
    }
}
```

CellRegionLayerGroup:

```
public class CellRegionLayerGroupBaker : Baker<CellRegionLayerGroupAuthoring>
{
    public override void Bake(CellRegionLayerGroupAuthoring authoring)
    {
        var terrainEntity = GetEntity(TransformUsageFlags.None);

        var createRequests = AddBuffer<CreateCellRegionLayerGroupRequest>(terrainEntity);
        if (authoring.Layers != null)
        {
            for (var lIndex = 0; lIndex < authoring.Layers.Count; lIndex++)
            {
                var layerAuthoring = authoring.Layers[lIndex];
                if (layerAuthoring.Args == null)
                {
                    Debug.LogError($"Layer {layerAuthoring.Name} has no Args defined.");
                    continue;
                }

                var request = new CreateCellRegionLayerGroupRequest
                {
                    Name = layerAuthoring.Name,
                    Args = layerAuthoring.Args
                };
                createRequests.Add(request);
            }
        }
    }
}
```

CellObjectLayerGroup:

```
public class CellObjectLayerGroupBaker : Baker<CellObjectLayerGroupAuthoring>
{
    public override void Bake(CellObjectLayerGroupAuthoring authoring)
    {
        var terrainEntity = GetEntity(TransformUsageFlags.None);

        var createRequests = AddBuffer<CreateCellObjectsLayerRequest>(terrainEntity);
        if (authoring.Layers != null)
        {
            for (var lIndex = 0; lIndex < authoring.Layers.Count; lIndex++)
            {
                var layerAuthoring = authoring.Layers[lIndex];
                if (layerAuthoring.Args == null)
                {
                    Debug.LogError($"Layer {layerAuthoring.Name} has no Args defined.");
                    continue;
                }

                var request = new CreateCellObjectsLayerRequest
                {
                    Name = layerAuthoring.Name,
                    Args = layerAuthoring.Args
                };
                createRequests.Add(request);
            }
        }
    }
}
```

CellEntityLayerGroup:

```
public class CellEntitiesLayersBaker : Baker<CellEntityLayerGroupAuthoring>
{
    public override void Bake(CellEntityLayerGroupAuthoring authoring)
    {
        var terrainEntity = GetEntity(TransformUsageFlags.Dynamic);
        var requestBuff = AddBuffer<CreateCellEntityLayerRequest>(terrainEntity);
        var cellEntityPrefabsBuff = AddBuffer<CellEntityConfig>(terrainEntity);

        if (authoring.Layers != null)
        {
            for (var lIndex = 0; lIndex < authoring.Layers.Count; lIndex++)
            {
                var layerAuthoring = authoring.Layers[lIndex];
                var createLayerArgs = layerAuthoring.Args;
                if (createLayerArgs != null)
                {
                    BakeCellEntities(createLayerArgs, lIndex, cellEntityPrefabsBuff);
                }

                var createLayerRequest = new CreateCellEntityLayerRequest
                {
                    Name = layerAuthoring.Name,
                    Args = new UnityObjectRef<CellEntityLayerConfigAsset>
                    {
                        Value = createLayerArgs
                    }
                };
                requestBuff.Add(createLayerRequest);
            }
        }
    }

    public virtual void BakeCellEntities(CellEntityLayerConfigAsset createArgs, int layerInd
ex, DynamicBuffer<CellEntityConfig> cellEntityConfigs)
    {
        var configsRangeStart = cellEntityConfigs.Length;
        var configsRangeSize = 0;

        for (int entityConfigIndex = 0; entityConfigIndex < createArgs.CellEntities.Count; e
ntityConfigIndex++)
        {
            var itemConfigAuthoring = createArgs.CellEntities[entityConfigIndex];
            var entityConfig = new CellEntityConfig
            {
                Color = itemConfigAuthoring.Color,
                Prefabs = new FixedList16<Entity>(),
                StatePerPrefab = new FixedList16<byte>()
            };

            for (var pIndex = 0; pIndex < itemConfigAuthoring.PrefabConfigs.Count; pIndex++)
            {
                var prefabConfigAuthoring = itemConfigAuthoring.PrefabConfigs[pIndex];
```

```
                RegisterPrefabForBaking(prefabConfigAuthoring.Prefab);
                var itemPrefab = GetEntity(prefabConfigAuthoring.Prefab, TransformUsageFlag
s.Dynamic);
                entityConfig.Prefabs.Add(itemPrefab);
                entityConfig.StatePerPrefab.Add((byte)prefabConfigAuthoring.State);
            }

            cellEntityConfigs.Add(entityConfig);

            configsRangeSize++;
        }

        createArgs.EntityConfigsRangeStart = configsRangeStart;
        createArgs.EntityConfigsRangeSize = configsRangeSize;

#if UNITY_EDITOR
        UnityEditor.EditorUtility.SetDirty(createArgs);
#endif
    }
}
```

GeoPlastLayerGroup:

```
public class GeoPlastLayersGroupBaker : Baker<GeoPlastLayerGroupAuthoring>
{
    public override void Bake(GeoPlastLayerGroupAuthoring authoring)
    {
        var terrainEntity = GetEntity(TransformUsageFlags.None);

        var createRequests = AddBuffer<CreateGeoPlastLayerRequest>(terrainEntity);
        if (authoring.Layers != null)
        {
            for (var lIndex = 0; lIndex < authoring.Layers.Count; lIndex++)
            {
                var layerAuthoring = authoring.Layers[lIndex];
                if (layerAuthoring.Args == null)
                {
                    Debug.LogError($"Layer {layerAuthoring.Name} has no Args defined.");
                    continue;
                }

                var request = new CreateGeoPlastLayerRequest
                {
                    Name = layerAuthoring.Name,
                    Args = layerAuthoring.Args
                };
                createRequests.Add(request);
            }
        }
    }
}
```

Interacting With a Terrain Entity

This page explains the main runtime interaction model in HexTerrains:

- how terrain state is stored on a **terrain entity**
- what `IHexTerrainAPI / HexTerrainAPI` are for
- what `IEnvironmentAPI / EnvironmentAPI` are for
- how `EnvironmentAPIComponent` is created (startup wiring)
- how to plug in a custom `IEnvironmentAPI` using `CreateEnvironmentAPIRequest` + `EnvironmentAPIConfigAssetBase`
- practical interaction examples (reading/writing terrain data)

---

1) The core idea: "Terrain = Entity + Components"

HexTerrains uses Unity Entities (ECS) as the storage backend.

A **terrain** is represented by an `Entity` (referred to as `terrainEntity`) that contains:

- `HexTerrain` (`IComponentData`) — holds `HexTerrainSettings` (size, chunk size, layout/rotation, etc.)
- multiple **managed components** (component objects) that implement various terrain layer/data concepts, commonly derived from `HexTerrainLayer` and/or `HexTerrainLayerGroup`

Most "terrain layers" are **managed component objects** attached to the same `terrainEntity`, e.g. a `ChunkMeshLayerGroup` that contains multiple `ChunkMeshLayer` instances.

Why APIs exist

Direct `EntityManager` access is not always convenient (or desirable) across tools/UI code. HexTerrains provides two API layers:

- `IEnvironmentAPI / EnvironmentAPI` — environment-wide operations (UI integration, save/load, create/destroy terrains, cursor raycast info, brush settings, etc.)
- `IHexTerrainAPI / HexTerrainAPI` — terrain-focused, convenience accessors/mutators (cell height/biome/regions/objects/entities/etc.), typically used by user tools

---

2) `IHexTerrainAPI` (terrain-focused API)

`IHexTerrainAPI` is a "terrain-scoped" façade around a specific `terrainEntity`.

Responsibilities

`IHexTerrainAPI` exposes:

- references to other subsystems:
  - `IUISystemAPI UISystemAPI { get; }`
  - `IEnvironmentAPI EnvironmentAPI { get; }`
- access to the current terrain:

    - `Entity GetTerrainEntity()`
    - `HexTerrainSettings? GetTerrainSettings()`
- managed layer access:
    - `TTerrainLayer GetTerrainLayer<TTerrainLayer>() where TTerrainLayer : class`
    - `bool IsLayerSupported<TDataLayer>()`
    - helper to fetch a layer from a group:
        - `GetTerrainLayer<TTerrainLayerGroup, TTerrainLayer>(HexTerrainLayerReference layerReference)`

The “LayerGroup + LayerReference” pattern

Many APIs use:

- a *group component* on the entity (ex: `ChunkMeshLayerGroup : HexTerrainLayerGroup<ChunkMeshLayer>, IComponentData`)
- a `HexTerrainLayerReference` to select a nested layer by:
    - index (`ByLayerIndex(...)`)
    - name (`ByLayerName(...)`)
    - index + name (`ByLayerIndexAndName(...)`)

This lets you address “the 2nd surface layer” or “the layer named Water” without hard-coding component types everywhere.

Typical operations

`IHexTerrainAPI` includes a large set of convenience methods grouped by feature, for example:

- **Cell height**: `GetCellHeight(...)`, `SetCellHeight(...)`
- **Biome**: `GetCellBiome(...)`, `SetCellBiome(...)`
- **Regions / objects / entities / geoplasts**: similar “get/set cell value” patterns
- **Visibility** of visual mesh layers: `GetChunkMeshLayerVisibility(...)`, `ShowChunkMeshLayer(...)`, `HideChunkMeshLayer(...)`

---

3) `HexTerrainAPI` (default `IHexTerrainAPI` implementation)

`HexTerrainAPI` is the provided baseline implementation.

What it caches

`HexTerrainAPI` stores:

- `Entity TerrainEntity` — the terrain it currently targets
- `EntityManager EntityManager` — used to query and mutate ECS storage
- `Entity UserToolEntity` — typically the entity that drives a tool/brush
- `IUISystemAPI UISystemAPI`
- `IEnvironmentAPI EnvironmentAPI`

It is designed so that if `TerrainEntity` changes, **all API calls automatically operate on the new entity** (the API queries the `EntityManager` each call).

Failure behavior (important)

Most methods are defensive:

- If the entity doesn't exist, or a layer isn't present, calls generally return `null` / `default` / `false` rather than throwing.
- Layer access uses `EntityManager.TryGetComponentObject<T>(...)` for managed components.

Example: how `HexTerrainAPI` reads/writes cell height

Internally `HexTerrainAPI` does this pattern:

1. Get the layer group component (managed or data component depending on the group type)
2. Use `HexTerrainLayerReference` to locate the nested layer
3. Use the layer's data layer (`HeightMap` , `BiomeMap` , etc.) and call `TryGetData / TrySetData`

This keeps tool code independent from the underlying data storage layout.

---

4) `IEnvironmentAPI` (environment-wide API)

`IEnvironmentAPI` is the "outer" API used to operate on **any** terrain entity, and also to integrate UI and environment-level assets.

Key properties

- `Entity EnvironmentAPIEntity { get; set; }`
  The entity that hosts the environment API component.
- `EntityManager EntityManager { get; set; }`
- `IUISystemAPI UISystemAPI { get; set; }`

Terrain-facing methods (selected highlights)

Terrain basics

- `HexTerrainSettings? GetTerrainSettings(Entity terrainEntity)`
- `TDataLayer GetTerrainLayer<TDataLayer>(Entity terrainEntity)`
- `bool IsTerrainLayerSupported<TDataLayer>(Entity terrainEntity)`

Visibility and view mode

- `bool? GetIsTerrainVisible(Entity terrainEntity)`
- `bool SetIsTerrainVisible(Entity terrainEntity, bool isVisible)`
- `int GetViewMode(Entity terrainEntity)`
- `bool SetViewMode(Entity terrainEntity, int viewMode)`

Persistence

- `bool SaveTerrain(Entity terrainEntity, string filePath)`

- `bool LoadTerrain(Entity terrainEntity, string filePath)`

Resizing and lifecycle

- `bool ResizeTerrain(Entity terrainEntity, int2 terrainSize)`
- `bool DestroyTerrain(Entity terrainEntity)`

Cursor / raycast

- `HexTerrainRaycastData? GetRaycastData(Entity terrainEntity)`
- `int? GetCellIndexUnderCursor(Entity terrainEntity)`
- `int2? GetCellCoordUnderCursor(Entity terrainEntity)`

Prefab configs and palettes

- `IList<IHexTerrainPrefabConfig> GetHexTerrainPrefabConfigsList()`
- `IList<Texture2D> GetHeightmapPalette()`

UI

- methods to create/get/destroy UI screens via `IUISystemAPI` (minimap, user tools screen, etc.)

---

5) `EnvironmentAPI` (default `IEnvironmentAPI` implementation)

`EnvironmentAPI` implements the interface using `EntityManager` reads/writes.

Notable behavior:

- `SaveTerrain(...)` / `LoadTerrain(...)` delegate to `HexTerrainSerializer`.
- `ResizeTerrain(...)` updates `HexTerrain.Settings.TerrainSize`, then iterates managed components on the terrain entity and calls `Resize(...)` for those that are `HexTerrainLayer`.
  This is the standard "resize propagates to managed layers" workflow.
- `GetHexTerrainPrefabConfigsList()` and `GetHeightmapPalette()` cache results after first query.

---

6) `EnvironmentAPIComponent`: how the environment API is stored on an entity

`EnvironmentAPIComponent` is a managed ECS component (`IComponentData`) that holds an `IEnvironmentAPI` instance:

- `public IEnvironmentAPI Value;`

It also implements `IDisposable` and will dispose `Value` if it implements `IDisposable`.

Marker component: `EnvironmentAPIEntity`

Alongside `EnvironmentAPIComponent`, the framework also adds a marker `EnvironmentAPIEntity : IComponentData` so systems can locate "the environment API entity" as a singleton.

---

7) How `EnvironmentAPIComponent` is created (startup flow)

Creation is request-driven via `CreateEnvironmentAPIRequest` and processed by `CreateEnvironmentAPISystem`.

7.1 The request component

`CreateEnvironmentAPIRequest : IComponentData` contains:

- `UnityObjectRef<EnvironmentAPIConfigAssetBase> ConfigAsset;`

7.2 The system that fulfills the request

`CreateEnvironmentAPISystem` (runs in `InitializationSystemGroup`, before `HexTerrainsInitGroup`) does:

1. Finds entities with `CreateEnvironmentAPIRequest` and *without* `EnvironmentAPIComponent`
2. Reads `request.ConfigAsset.Value`
3. Calls `apiConfig.CreateAPI(EntityManager, entity)`
4. Adds:
    - `EnvironmentAPIComponent { Value = environmentAPI }`
    - `EnvironmentAPIEntity` marker
5. Removes `CreateEnvironmentAPIRequest` so it only runs once per entity

Implication: to customize environment behavior, you typically customize the *config asset* used by the request.

---

8) Providing a custom `IEnvironmentAPI`

You customize environment API creation via a ScriptableObject derived from `EnvironmentAPIConfigAssetBase`.

8.1 What `EnvironmentAPIConfigAssetBase` does by default

- `CreateAPI(...)` creates `new EnvironmentAPI()` and calls:
    - `Init(entityManager, GetUISystemAPI(entityManager), apiEntity)`
- `GetUISystemAPI(entityManager)` is abstract, so every project must decide how to provide the UI API instance.

8.2 Customization options

Option A: Override only `GetUISystemAPI(...)`

Use this when you want the default `EnvironmentAPI` behavior, but must provide another UI API implementation.

Option B: Override `CreateAPI(...)`

Use this when you want a different environment API implementation:

- subclass `EnvironmentAPI` (recommended when you just extend behavior), or
- implement `IEnvironmentAPI` from scratch

8.3 Example config asset overriding `CreateAPI(...)`

Below is a minimal pattern you can adapt (the UI acquisition is project-specific):

```
// In your project (example)
using Fwt.Core.UI.Screens;
using Fwt.HexTerrains;
using Fwt.HexTerrains.Data;
using Unity.Entities;
using UnityEngine;

[CreateAssetMenu(menuName = "HexTerrains/Environment API Config (Custom)")]
public sealed class CustomEnvironmentAPIConfigAsset : EnvironmentAPIConfigAssetBase
{
    public override IEnvironmentAPI CreateAPI(EntityManager entityManager, Entity apiEntity)
    {
        var api = new CustomEnvironmentAPI();
        api.Init(entityManager, GetUISystemAPI(entityManager), apiEntity);
        return api;
    }

    public override IUISystemAPI GetUISystemAPI(EntityManager entityManager)
    {
        // Resolve your UI system API here (project-specific).
        // Typically: query a singleton component, a world-managed service, etc.
        return default;
    }

    private sealed class CustomEnvironmentAPI : EnvironmentAPI
    {
        // Override / extend behavior as needed:
        // - SaveTerrain / LoadTerrain (custom callbacks)
        // - CreateNewTerrain pipeline
        // - custom UI screen management
    }
}
```

8.4 Wiring it up at runtime

To activate your config, create an entity (or choose an existing “environment/bootstrap” entity) and add:

- `CreateEnvironmentAPIRequest { ConfigAsset = <your asset ref> }`

`CreateEnvironmentAPISystem` will attach the resulting API to that entity as `EnvironmentAPIComponent`.

---

9) Practical interaction examples (using `HexTerrainAPI`)

Example 1: Read/Write a cell height

Conceptually (pseudo-flow):

1. Determine a target cell index (from cursor raycast, selection, etc.)
2. Use `HexTerrainAPI.SetCellHeight(...)` to write into a specific `ChunkMeshLayer`
3. Downstream systems react because the underlying data layer is marked dirty

```
// Example usage pattern (pseudo-code):
var cellIndex = environmentApi.GetCellIndexUnderCursor(terrainEntity);
if (cellIndex.HasValue)
{
    // Choose the target mesh layer inside a layer group by name/index:
    var layerRef = HexTerrainLayerReference.ByLayerName("Surface");
    hexTerrainApi.SetCellHeight<MyChunkMeshLayerGroup, ChunkMeshLayer>(
        cellIndex.Value,
        layerRef,
        value: 5.0f
    );
}
```

Example 2: Toggle visibility of a visual mesh layer

```
var layerRef = HexTerrainLayerReference.ByLayerName("Water");
var isVisible = hexTerrainApi.GetChunkMeshLayerVisibility<MyChunkMeshLayerGroup, VisualChunk
MeshLayer>(layerRef);
if (isVisible.HasValue)
{
    hexTerrainApi.SetChunkMeshVisibility<MyChunkMeshLayerGroup, VisualChunkMeshLayer>(layerR
ef, !isVisible.Value);
}
```

Example 3: Read terrain settings safely

```
var settings = hexTerrainApi.GetTerrainSettings();
if (settings.HasValue)
{
    // settings.Value.TerrainSize, settings.Value.ChunkSize, etc.
}
```

10) When to use `IEnvironmentAPI` vs `IHexTerrainAPI`

Use `IEnvironmentAPI` when:

- you have a `terrainEntity` and need general operations (save/load, resize, raycast-under-cursor, view mode, UI screens)

Use `IHexTerrainAPI` when:

- you are writing tool/brush-style code and want compact helpers for "set cell value in layer X"
- you want logic that is naturally scoped to "the current terrain"

In many tool flows:

- `IEnvironmentAPI` determines *what cell is targeted* (raycast/cursor)
- `IHexTerrainAPI` mutates *the data inside a specific terrain layer*

🧩 Chunks Grid System

The Hex Terrains Framework divides the terrain into **chunks** to enable efficient rendering, visibility culling, and simulation scaling. The **Chunks Grid System** manages the lifecycle, visibility state, and dynamic spatial organization of these chunks.

Overview

Each terrain is split into a grid of chunks. This system ensures:

- Only **visible chunks** are rendered or in some cases even updated.
- Chunks can dynamically adjust their **transform offsets** for seamless terrain tiling.
- Simulation and rendering can be constrained to **active areas** based on camera visibility.

Core Concepts

`ChunksGridLayer`

A component that stores all spatial data related to chunk organization and visibility. It holds:

- Chunk settings (dimensions, count, etc.)
- Visibility flags per chunk (`ChunkVisibility`)
- Chunk metrics (`ChunkMetrics`)
- Dirty flags for dependency management and selective updating

Added on Terrain Entity by `CreateChunksGridLayerSystem`

`ChunkBounds`

Each chunk tracks 9 bounds:

- `Bounds_Center` — actual location
- `Bounds_{Direction}` — adjacent bounds used for edge wrapping

These enable neighborhood queries and visual interpolation across chunk edges.

`ChunkVisibility`

Stores per-chunk visibility information:

- `IsVisible` — true if inside camera frustum
- `VisualPosition` — where the chunk is currently rendered (may differ from actual world position if terrain wraps)
- `ChunkOffset` — logical offset used for virtual terrain scrolling
- `Transform` — final transform matrix applied to the chunk renderer

Lifecycle Systems

✅ CreateChunksGridLayerSystem

- Group: `CreateHexTerrainsGroup`
- Automatically adds `ChunksGridLayer` component to any entity with `CreateChunksGridLayerRequest`
- Initializes chunk data using terrain settings

```
chunksGridData.Init(request.Settings);
```

📐 CalcChunkVisibilitySystem

- Group: `HexTerrainsUpdateGroup`
- Recalculates visible chunks each frame based on camera frustum
- Stores new visibility states in `ChunkVisibility` per chunk
- Uses a `CalcHexChunkVisibilityJob` to run visibility culling in parallel

```
var job = new CalcHexChunkVisibilityJob {
    // Assign metrics, frustum, visibility maps...
};
job.Schedule(terrainSettings.ChunksCount, 16, combinedDeps);
```

This system is the core of **terrain culling and selective rendering**.

🧹 CleanupChunksGridLayerSystem

- Group: `HexTerrainsCleanupGroup`
- Calls `CleanupAsync()` on `ChunksGridLayer` when terrain is visible
- Happens at the beginning of the frame before other systems. Cleans all the dirty flags collected during last frame

Authoring Support

The `CreateChunksGridLayerRequest` is added on baked Terrain Entity by `HexTerrainBaker` during baking the `HexTerrainAuthoring`.

Execution Order Summary

System	Group	Purpose
`CreateChunksGridLayerSystem`	`CreateHexTerrainsGroup`	Initializes chunks on terrain load
`CalcChunkVisibilitySystem`	`HexTerrainsUpdateGroup`	Recomputes which chunks are visible
`CleanupChunksGridLayerSystem`	`HexTerrainsCleanupGroup`	Cleans up chunk data when no longer needed

System Execution Pipeline

The simulation logic of the Hex Terrains Framework is structured into well-defined **execution phases**, organized through a hierarchy of `ComponentSystemGroup` s. This design ensures that all systems run in a deterministic and performant order, respecting job dependencies and data lifecycle phases such as cleanup, initialization, simulation, rendering.

Each major phase of the pipeline is represented by a dedicated system group and often accompanied by a corresponding **barrier system** (`EntityCommandBufferSystem`) to queue structural changes safely at the end of the phase.

Overview of Execution Order

The system groups are executed in the following sequence throughout each frame:

1. **Initialization Phase** (`InitializationSystemGroup`)
    - `HexTerrainsCleanupGroup`
    - `HexTerrainsInitGroup`

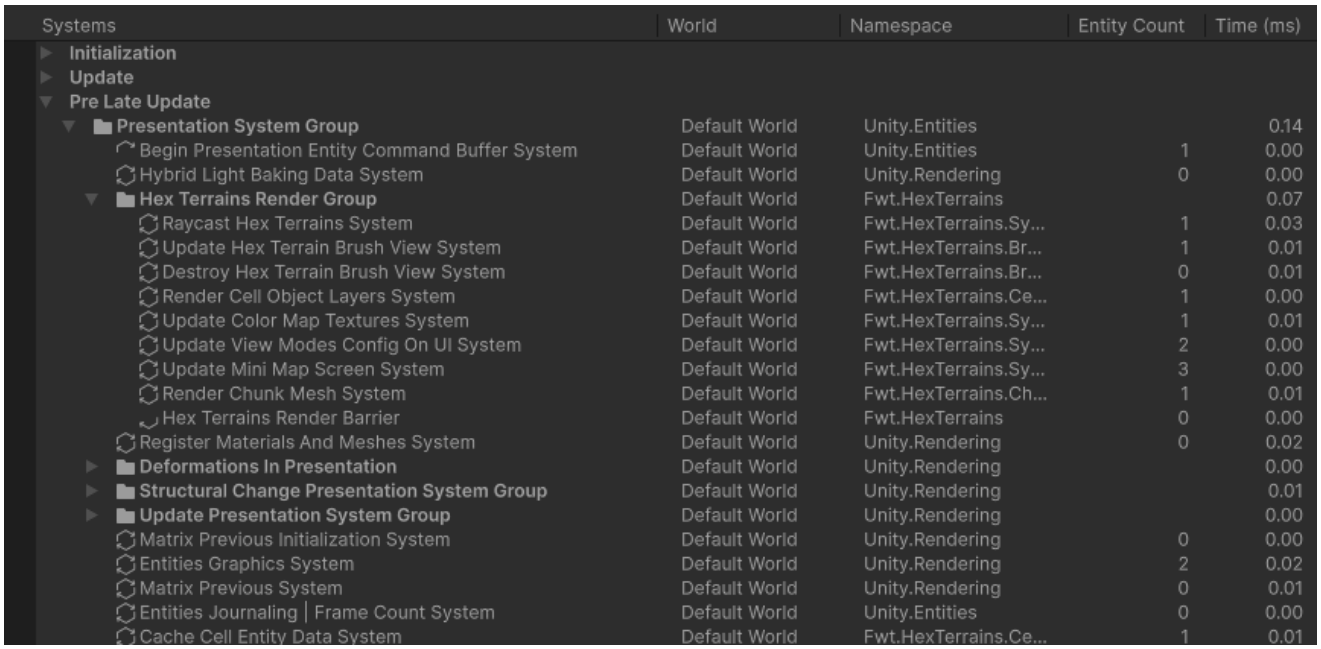
2. **Simulation Phase** (`SimulationSystemGroup`)
    - `HexTerrainsSimulationGroup`
    - `HexTerrainsUpdateGroup`
    - `HexTerrainsStartPreRenderGroup`

Systems	World	Namespace	Entity Count	Time (ms)
Initialization				
Update				
Simulation System Group	Default World	Unity.Entities		0.14
Begin Simulation Entity Command Buffer System	Default World	Unity.Entities	1	0.00
Fixed Step Simulation System Group	Default World	Unity.Entities		0.00
Variable Rate Simulation System Group	Default World	Unity.Entities		0.00
Companion Game Object Update System	Default World	Unity.Entities	0	0.00
Companion Game Object Live Baking Init System	Default World	Unity.Entities	0	-
Hex Terrains Simulation Group	Default World	Fwt.HexTerrains		0.01
Update Hex Terrain Simulation System	Default World	Fwt.HexTerrains.Si...	1	0.00
Simulate Sun Terrain Layer System	Default World	Fwt.HexTerrains.Su...	2	0.00
Simulate Geo Plast Layer System	Default World	Fwt.HexTerrains.Ge...	2	0.00
Update Geo Plast Layers System	Default World	Fwt.HexTerrains.Ge...	1	0.00
Render Geo Plast Layers System	Default World	Fwt.HexTerrains.Ge...	1	0.00
Hex Terrains Simulatios Barrier	Default World	Fwt.HexTerrains	0	0.00
Hex Terrains Update Group	Default World	Fwt.HexTerrains		0.04
Update Camera Controller System	Default World	Fwt.HexTerrains.C...	2	0.01
Calc Chunk Visibility System	Default World	Fwt.HexTerrains.Sy...	1	0.00
Calc Regions Sizes System	Default World	Fwt.HexTerrains.Re...	1	0.00
Calc Regions Cells System	Default World	Fwt.HexTerrains.Re...	1	0.00
Calc Chunk Mesh Cell Metrics System	Default World	Fwt.HexTerrains.Ch...	1	0.00
Calc Dirty Chunk Mesh Sources System	Default World	Fwt.HexTerrains.Ch...	1	0.00
Calc Cell Object Layers System	Default World	Fwt.HexTerrains.Ce...	1	0.00
Calc Cell Entity Entities System	Default World	Fwt.HexTerrains.Ce...	1	0.00
Update Cell Entities System	Default World	Fwt.HexTerrains.Ce...	1	0.01
Hex Terrains Update Barrier	Default World	Fwt.HexTerrains	0	0.00
Hex Terrains Start Pre Render Group	Default World	Fwt.HexTerrains		0.06
Playback Update Cell Entity Command Buffers System	Default World	Fwt.HexTerrains.Ce...	1	0.01
Update Cell Entity Layer Entities State System	Default World	Fwt.HexTerrains.Ce...	1	0.00
Calc Chunk Mesh Sources System	Default World	Fwt.HexTerrains.Ch...	1	0.03
Calc Geo Plast Color Map System	Default World	Fwt.HexTerrains.Ge...	1	0.01
Calc Cell Entity Color Map System	Default World	Fwt.HexTerrains.Ce...	1	0.00
Calc Regions Color Map System	Default World	Fwt.HexTerrains.Re...	1	0.00
Calc Chunk Mesh Color Map System	Default World	Fwt.HexTerrains.Ch...	1	0.00
Calc Cell Objects Color Map System	Default World	Fwt.HexTerrains.Ce...	1	0.00
Hex Terrains Start Pre Render Barrier	Default World	Fwt.HexTerrains	0	0.00
Transform System Group	Default World	Unity.Transforms		0.01
Companion Game Object Update Transform System	Default World	Unity.Entities	0	0.00
Late Simulation System Group	Default World	Unity.Entities		0.01
End Simulation Entity Command Buffer System	Default World	Unity.Entities	1	0.00
Pre Late Update				

3. **Late Simulation Phase** (`LateSimulationSystemGroup`)
    - `HexTerrainsFinishPreRenderGroup`

Systems	World	Namespace	Entity Count	Time (ms)
Initialization				
Update				
Simulation System Group	Default World	Unity.Entities		0.21
Begin Simulation Entity Command Buffer System	Default World	Unity.Entities	1	0.00
Fixed Step Simulation System Group	Default World	Unity.Entities		0.00
Variable Rate Simulation System Group	Default World	Unity.Entities		0.00
Companion Game Object Update System	Default World	Unity.Entities	0	0.00
Companion Game Object Live Baking Init System	Default World	Unity.Entities	0	-
Hex Terrains Simulation Group	Default World	Fwt.HexTerrains		0.02
Hex Terrains Update Group	Default World	Fwt.HexTerrains		0.06
Hex Terrains Start Pre Render Group	Default World	Fwt.HexTerrains		0.09
Transform System Group	Default World	Unity.Transforms		0.01
Companion Game Object Update Transform System	Default World	Unity.Entities	0	0.00
Late Simulation System Group	Default World	Unity.Entities		0.02
Hex Terrains Finish Pre Render Group	Default World	Fwt.HexTerrains		0.01
Create Chunk Meshes System	Default World	Fwt.HexTerrains.Ch...	1	0.01
Hex Terrains Finish Pre Render Barrier	Default World	Fwt.HexTerrains	0	0.00
Update Cell Entity Visibility System	Default World	Fwt.HexTerrains.Ce...	0	0.00
End Simulation Entity Command Buffer System	Default World	Unity.Entities	1	0.00
Pre Late Update				

4. **Presentation Phase** (`PresentationSystemGroup`)
    - `HexTerrainsRenderGroup`



Systems	World	Namespace	Entity Count	Time (ms)
Initialization				
Update				
Pre Late Update				
Presentation System Group	Default World	Unity.Entities		0.14
Begin Presentation Entity Command Buffer System	Default World	Unity.Entities	1	0.00
Hybrid Light Baking Data System	Default World	Unity.Rendering	0	0.00
Hex Terrains Render Group	Default World	Fwt.HexTerrains		0.07
Raycast Hex Terrains System	Default World	Fwt.HexTerrains.Sy...	1	0.03
Update Hex Terrain Brush View System	Default World	Fwt.HexTerrains.Br...	1	0.01
Destroy Hex Terrain Brush View System	Default World	Fwt.HexTerrains.Br...	0	0.01
Render Cell Object Layers System	Default World	Fwt.HexTerrains.Ce...	1	0.00
Update Color Map Textures System	Default World	Fwt.HexTerrains.Sy...	1	0.01
Update View Modes Config On UI System	Default World	Fwt.HexTerrains.Sy...	2	0.00
Update Mini Map Screen System	Default World	Fwt.HexTerrains.Sy...	3	0.00
Render Chunk Mesh System	Default World	Fwt.HexTerrains.Ch...	1	0.01
Hex Terrains Render Barrier	Default World	Fwt.HexTerrains	0	0.00
Register Materials And Meshes System	Default World	Unity.Rendering	0	0.02
Deformations In Presentation	Default World	Unity.Rendering		0.00
Structural Change Presentation System Group	Default World	Unity.Rendering		0.01
Update Presentation System Group	Default World	Unity.Rendering		0.00
Matrix Previous Initialization System	Default World	Unity.Rendering	0	0.00
Entities Graphics System	Default World	Unity.Rendering	2	0.02
Matrix Previous System	Default World	Unity.Rendering	0	0.01
Entities Journaling | Frame Count System	Default World	Unity.Entities	0	0.00
Cache Cell Entity Data System	Default World	Fwt.HexTerrains.Ce...	1	0.01

Each group is scheduled inside a broader Unity ECS phase (`InitializationSystemGroup` , `SimulationSystemGroup` , `LateSimulationSystemGroup` , `PresentationSystemGroup`) to ensure correct integration into Unity's default update loop.

---

1. `HexTerrainsCleanupGroup`

- **Parent:** `InitializationSystemGroup`
- **Order:** Executes **first** in the frame (`OrderFirst = true`)
- **Purpose:** Resets flags, clears buffers, and prepares terrain data for the new simulation frame.
- **Typical Systems:** Reset dirty flags, mark layers for recomputation.

---

3. `HexTerrainsInitGroup`

- **Parent:** `InitializationSystemGroup`
- **Order:** Runs **after** `HexTerrainsSimulationGroup`
- **Purpose:** Handles the creation of new terrain entities, initialization of ECS components, and data allocation.

---

2. `HexTerrainsSimulationGroup`

- **Parent:** `SimulationSystemGroup`
- **Order:** Runs **before** `HexTerrainsUpdateGroup`
- **Purpose:** Handles the simulation processes like water flow, snow melting, etc.

---

3. `HexTerrainsUpdateGroup`

- **Parent:** `SimulationSystemGroup`
- **Order:** Runs **after** `HexTerrainsSimulationGroup`

- **Purpose:** Every frame update logic runs here. Processes input, accumulated changes to terrain state. Here happens the reaction on data change of the terrain. This includes terrain analysis, updates to data layers, generation of mesh data, etc.

🚦 4. `HexTerrainsStartPreRenderGroup`

- **Parent:** `SimulationSystemGroup`
- **Order:** Executes **after** `HexTerrainsUpdateGroup`
- **Purpose:** Runs all the logic that happens after the input is processed. Such as ColorMap colors, CellEntityLayer entities, etc.

✍️ 5. `HexTerrainsFinishPreRenderGroup`

- **Parent:** `LateSimulationSystemGroup`
- **Purpose:** Final cleanup and processing just before rendering.
- **Typical Use:** Create meshes, combine buffers, finalize render settings, compress or normalize data.

🎨 6. `HexTerrainsRenderGroup`

- **Parent:** `PresentationSystemGroup`
- **Purpose:** Renders the terrain using data prepared in previous steps. Filling ColorMap textures with calculated colors.
- **Use Case:** Rendering systems using `Graphics.DrawMeshInstanced`, material property block application, GPU data upload.

🛡️ Barrier Systems

Each phase has a corresponding **EntityCommandBufferSystem** that ensures deferred structural changes do not break parallel simulation:

Barrier System	Associated Group
`HexTerrainsCleanupBarrier`	`HexTerrainsCleanupGroup`
`HexTerrainsInitBarrier`	`HexTerrainsInitGroup`
(others exist for each group)	*(other hex terrain groups)*

📊 Summary Diagram (Text)

```
[InitializationSystemGroup]
 ├── HexTerrainsCleanupGroup (first)
 │     └── HexTerrainsCleanupBarrier
 └── HexTerrainsInitGroup (after cleanup)
       └── HexTerrainsInitBarrier

[SimulationSystemGroup]
 ├── HexTerrainsSimulationGroup
 │     └── HexTerrainsSimulationBarrier
 ├── HexTerrainsUpdateGroup
 │     └── HexTerrainsUpdateBarrier
 └── HexTerrainsStartPreRenderGroup
       └── HexTerrainsStartPreRenderBarrier

[LateSimulationSystemGroup]
 └── HexTerrainsFinishPreRenderGroup
 │     └── HexTerrainsFinishPreRenderBarrier

[PresentationSystemGroup]
 └── HexTerrainsRenderGroup
       └── HexTerrainsRenderBarrier
```

Camera Controller System

The Camera Controller provides an ECS-driven camera system designed to smoothly navigate procedural hex terrains. It handles input processing, boundary wrapping (for infinite or looped terrains), camera state saving/loading, and minimap integration.

Overview

The system operates using Unity's Entity Component System (ECS) and relies on an authoring component to configure input keys and speeds. Behind the scenes, the framework handles the creation, update, and cleanup of the physical camera target object.

Features

- **Configurable Input:** Customizable keys for movement, rotation, zoom, acceleration, and movement locking.
- **UI Detection:** Safely ignores input when the mouse is hovering over UI elements using the Unity Event System.
- **Boundary Wrapping:** Automatically wraps camera position around the terrain bounds if the terrain is configured to be connected horizontally or vertically.
- **State Management:** Quick save (F5) and load (F9) of camera position, rotation, and zoom using `PlayerPrefs`.
- **Minimap Integration:** Automatically synchronizes the normalized camera position and rotation mapping with the `IMinimapScreen` interface.

Setup and Authoring

To use the camera controller, attach the `CameraControllerAuthoring` script to a GameObject in your subscene. The Baker will automatically convert this into the required ECS setup (`CreateCameraControllerRequest` and `IgnoreCameraZoomInputKeys` buffer).

Authoring Properties

Input Keys

Property	Default Key	Description
MoveUpKey	`W`	Moves the camera target forward along the Y axis (2D plane).
MoveDownKey	`S`	Moves the camera target backward.
MoveLeftKey	`A`	Moves the camera target left.
MoveRightKey	`D`	Moves the camera target right.
RotateLeftKey	`Q`	Rotates the camera counter-clockwise.

Property	Default Key	Description
RotateRightKey	`E`	Rotates the camera clockwise.
AccelerationKey	`LeftShift`	Multiplies movement speeds by the acceleration modifier.
LockMovementKey	`LeftControl`	Locks the current movement velocity and auto-moves the camera continuously.
SaveCameraStateKey	`F5`	Saves the current camera rotation, zoom, and position to `PlayerPrefs`.
LoadCameraStateKey	`F9`	Loads the saved camera state from `PlayerPrefs`.

Speeds and Modifiers

Property	Description
ZoomSpeed	Scroll wheel zoom sensitivity multiplier.
MovementSpeed	Base panning movement speed.
RotationSpeed	Base rotation speed when using `Q` / `E`.
AccelerationMultiplier	Speed multiplier applied when holding the `AccelerationKey`.
PreventZoomKeys	List of `KeyCode`s that, when held, ignore mouse scroll zoom (Defaults: `LeftAlt`, `LeftControl`).
IsFindCameraControllerComponentOnScene	If true, automatically tries to find a `CameraControllerComponent` in the active scene and link it to the target transform during initialization.

Architecture & Systems

The mechanic is divided into three primary ECS Systems:

1. `CreateCameraControllerSystem`

- Runs in the `HexTerrainsInitGroup`.
- Processes the `CreateCameraControllerRequest`.
- Generates a new `GameObject` named "CameraControllerTarget" to act as the scene anchor.
- Links the active `CameraControllerComponent` from the scene if requested via `IsFindCameraControllerComponentOnScene`.
- Assigns the `CameraControllerTarget` component to track all target attributes and references.

2. `UpdateCameraControllerSystem`

- Runs in the `HexTerrainsUpdateGroup`.
- Only updates if the terrain is visible (`HexTerrainVisibility`).

- Computes framerate-independent deltas for continuous movement, rotation, and zooming based on the configured inputs.
- Handles the locking behavior logic to continuously orbit or pan without further user input.
- Computes out-of-bounds positioning. Using `HexTerrainSettings.IsConnectedHorizontally` and `IsConnectedVertically`, it seamlessly wraps the X/Y positions on cyclic environments or clamps them to boundaries for fixed terrains.
- Synchronizes with UI minimaps via the `UpdateMinimapScreen` lifecycle method.

3. `DestroyCameraControllerSystem` (Cleanup)

- Runs in the `HexTerrainsCleanupGroup`.
- Responds to the absence of a `HexTerrain` component on entities that still possess a `CameraControllerTarget`.
- Cleans up and destroys the instantiated `CameraControllerTarget` GameObject to prevent memory and hierarchy leaks.

Terrain Layers Overview

This document explains the HexTerrains **Terrain Layer** architecture: what a layer is, how layers are initialized and updated, how layers are organized into `HexTerrainLayerGroup`s, which layers are provided by the framework, and how to implement custom layers.

> Terminology note: in this framework a "layer" is a **runtime subsystem + data container**. It is *not* a Unity `LayerMask`.

Table of contents

---

1. What is a Terrain Layer?

A **Terrain Layer** is a modular unit of terrain functionality. Each layer:

- Owns a coherent set of terrain data (typically stored in native collections / data layers).
- Implements a predictable lifecycle (`Init` , per-frame work via jobs, `Cleanup` , `Dispose`).
- Can depend on other layers (via `HexTerrainLayerReference`).
- Can be grouped and managed alongside other layers using `HexTerrainLayerGroup` .

The goal is to keep the terrain system composable:

- Rendering is separated from simulation.
- Entities are separated from purely visual objects.
- Region logic is separated from geometry.
- Layers can be added/removed/replaced without rewriting the whole pipeline.

---

2. `HexTerrainLayer`: lifecycle and responsibilities

All terrain layers ultimately derive from:

- `HexTerrainLayer` (abstract base class)

Key members and concepts from `HexTerrainLayer`:

Core state

- `public string Name { get; set; }`
  Optional but strongly recommended. `HexTerrainLayerGroup` can index layers by name.
- `public HexTerrainSettings Settings;`
  The settings this layer was initialized with (terrain size, chunk counts, etc.).
- `public HexTerrainLayer ParentLayer { get; set; }`
  If the layer is inside a group, it typically points to the owning `HexTerrainLayerGroup`.

Lifecycle methods

- `Init(HexTerrainSettings settings)`
  Base method completes jobs, updates settings, and marks data dirty if settings changed.
- `Init<TInitArgs>(HexTerrainSettings settings, TInitArgs initArgs)`
  Used to inject configuration (usually ScriptableObject assets implementing config interfaces). Most concrete layers override this to read their config.
- `Resize(int2 terrainSize)`
  Convenience wrapper that updates `Settings.TerrainSize` then re-`Init`s.
- `CalculateColorMap(JobHandle dependency)`
  A common "hook" used by several layers to schedule color-map related jobs.
- `CompleteAllJobs()`
  Should complete any outstanding job handles used by the layer's data layers.
- `SetAllDirty(bool isDirty)` *(abstract)*
  Marks all internal data layers dirty/clean.
- `Cleanup()` and `CleanupAsync(JobHandle dependency)` *(abstract)*
  Used to reset dirty flags and/or schedule "start-of-frame" cleanup jobs.
- `Dispose()`
  Releases native allocations.

Cross-layer access helpers

`HexTerrainLayer` provides helper methods to fetch layers from the parent group:

- `GetTerrainLayerFromParent<TLayer>(int index)`
- `GetTerrainLayerFromParent<TLayer>(string name)`
- `GetTerrainLayerFromParent<TLayer>(HexTerrainLayerReference reference)`

These are useful when a layer needs to read/write other layers' data.

3. Data layers, jobs, and dirty flags

Most runtime terrain data in HexTerrains is stored using *data layer* abstractions (examples seen in the codebase):

- `CellValueDataLayer<T>`
- `ColorMapCellValueDataLayer_Int`, `ColorMapCellValueDataLayer_Float`, ...
- Chunk-scoped layers such as:
  - `ChunkMetricsDataLayer`
  - `ChunkVisibilityDataLayer`
  - `NativeParallelHashSetDataLayer<int>`
  - `NativeParallelMultiHashMapDataLayer<TKey, TValue>`
  - `NativeListDataLayer<T>`

These wrappers typically provide:

- Native storage (`NativeArray`, `NativeList`, hash sets/maps).
- Dirty tracking (global and often per-chunk).
- Job dependency tracking (read/write fences).

Typical job scheduling pattern

A common pattern across layers (e.g., `ChunksGridLayer`, `VisualGeoPlastLayer`, `SunTerrainLayer`) looks like:

1. **Early-out** if nothing is dirty / nothing changed.
2. **Prepare dependencies**:
   - `PrepareToRead()` for inputs
   - `PrepareToWrite()` for outputs
   - Combine with incoming `dependency`.
3. **Schedule jobs**.
4. **Register dependencies** back on data layers:
   - `AddReadDependency(handle) / AddWriteDependency(handle)`
5. **Mark dirty**:
   - `SetDirty(true)`
   - and often schedule "mark dirty chunks" jobs when using per-chunk dirty sets.

Example (conceptual):

```
var depsA = inputLayer.PrepareToRead();
var depsB = outputLayer.PrepareToWrite();
dependency = JobHandle.CombineDependencies(dependency, depsA, depsB);
dependency = job.Schedule(Settings.CellsCount, Settings.CellsPerChunk, dependency);
inputLayer.AddReadDependency(dependency);
outputLayer.AddWriteDependency(dependency);
outputLayer.SetDirty(true);
```

Dirty flags: why they matter

Dirty flags are used to avoid unnecessary work:

- If a data layer is not dirty, many layers simply return the incoming `JobHandle` unchanged.
- Some layers also track *dirty chunks* to minimize chunk mesh regeneration.

---

4. Referencing other layers: `HexTerrainLayerReference`

`HexTerrainLayerReference` is a serializable struct used to locate a layer within a `HexTerrainLayerGroup`.

It supports:

- Lookup by index (`IsSearchByIndex`, `LayerIndex`)
- Lookup by name (`IsSearchByName`, `LayerName`)
- Or both (index + name)

Helpers:

- `HexTerrainLayerReference.ByLayerIndex(int)`
- `HexTerrainLayerReference.ByLayerName(string)`
- `HexTerrainLayerReference.ByLayerIndexAndName(int, string)`
- `HexTerrainLayerReference.Default`

This enables clean cross-layer wiring from configuration assets (e.g., "this GeoPlast layer renders into surface layer X").

---

5. `HexTerrainLayerGroup`: organizing multiple layers

A `HexTerrainLayerGroup<TTerrainLayer>` is itself a `HexTerrainLayer`, but its role is to manage a list of child layers.

Key capabilities (from the framework API surface):

- Stores:
  - `List<TTerrainLayer> Layers`
  - `Dictionary<string, TTerrainLayer> LayersByName`
  - `Dictionary<string, int> LayerIndexByName`
  - `Dictionary<TTerrainLayer, int> LayerIndexByInstance`
- Provides many `GetLayer(...)` overloads:
  - by index
  - by name
  - by type
  - by `HexTerrainLayerReference`
- Provides group-wide lifecycle:
  - `Init(...)` initializes and/or recreates all child layers.
  - `CalculateColorMap(...)` can combine color map job handles from all layers.
  - `SetAllDirty(...)`, `Cleanup(...)`, `CleanupAsync(...)`, `Dispose()` forward to all children.

Specialized groups in the framework

You will commonly work with groups dedicated to specific layer types, e.g.:

- `ChunkMeshLayerGroup : HexTerrainLayerGroup<ChunkMeshLayer>`
- `CellEntityLayerGroup : HexTerrainLayerGroup<CellEntityLayer>`
- `CellObjectLayerGroup : HexTerrainLayerGroup<CellObjectLayer>`
- `CellRegionLayerGroup : HexTerrainLayerGroup<CellRegionLayer>`
- `GeoPlastLayerGroup : HexTerrainLayerGroup<GeoPlastLayer>`

These derived groups often add orchestration methods, e.g.:

- `ChunkMeshLayerGroup.CalcMeshSources(...)`, `CreateChunkMeshes()`, `Render(...)`
- `CellEntityLayerGroup.CalculateEntities(...)`, `UpdateEntities(...)`
- `GeoPlastLayerGroup.Simulate(...)`, `Render(...)`
- `CellObjectLayerGroup.Render(...)`

Parent-child relationship

When a layer is created by a group, the group should set:

- child `ParentLayer = this`
- child `Name` (commonly from creation request args)

This is what enables `GetTerrainLayerFromParent<T>` lookups from inside a child layer.

---

6. Initialization patterns

Most runtime layers are created and configured using ScriptableObject config assets.

The common pattern

1. **Create a config asset** in Unity:
   - For example `ChunkMeshLayerConfigAsset`, `CellEntityLayerConfigAsset`, `GeoPlastLayerConfigAsset`, etc.
2. **The config asset implements an interface** used by the layer's `Init<TInitArgs>`:
   - Example: `ChunkMeshLayerConfigAsset : IChunkMeshLayerConfig`
3. **The asset is used as an init arg** when building the terrain:
   - The group creates the layer via `CreateTerrainLayer()` (factory pattern)
   - The group calls `InitTerrainLayer(layer, settings, initArgs)`
   - The layer reads config and initializes internal data layers

What should happen in `Init(...)`

Concrete layers typically do two things:

- Ensure their internal data layers exist and are sized to match `settings`.
- Apply config parameters, references, and default values.

Example evidence from the codebase:

- `ChunksGridLayer.Init(settings)` allocates chunk visibility/metrics layers, resets viewport caches, and recalculates metrics when settings change.
- `SunTerrainLayer.Init<TInitArgs>` reads `ISunTerrainLayerConfig` and populates `SunSettings`, `TargetLayers`, and progress values.
- `VisualGeoPlastLayer.InitLayerData(args)` reads `IVisualGeoPlastLayerConfig` and stores `SurfaceLayerRef` + visual rendering settings.

---

7. The 6 main layer types (and how to use them)

This section describes the framework's six "pillar" layer types and the standard workflow for each.

> Some types include multiple derived/related layers in the codebase; this section focuses on the conceptual role and the visible API surface.

7.1 `ChunkMeshLayer`

Purpose: Surface-like cell data (height/biome/transparency) and cell metrics used for chunk mesh generation.

Common data owned by `ChunkMeshLayer`:

- `HeightMap : ColorMapCellValueDataLayer_Float`
- `BiomeMap : ColorMapCellValueDataLayer_Int`
- `TransparencyMap : CellValueDataLayer<bool>`
- `CellMetricsMap : CellMetricsDataLayer`
- `ChunkMeshLayerSettings : ChunkMeshLayerSettings`
- Sync references:
  - `SyncHeightMapLayerReference`
  - `SyncBiomeMapLayerReference`
  - `SyncTransparencyMapLayerReference`

Key operations (conceptually):

- **Sync** values from another layer (`SyncData`, `SyncHeightMap`, etc.)
- **Calculate** cell metrics (`CalcCellMetrics`)
- **CalculateColorMap** (often biome/height based)
- **Serialize/Deserialize** layer state for persistence

Rendering pipeline connection: `VisualChunkMeshLayer`

`VisualChunkMeshLayer` extends the chunk mesh concept into actual mesh generation and rendering. It typically uses:

- `ChunkMeshSources` (mesh source buffers per chunk)
- `ChunkMeshes` (Unity `Mesh` instances per chunk)
- Render configs (`HexSurfaceRenderConfigAsset`) per view mode
- A `RenderSettings` struct controlling meshing parameters

Key steps:

1. `CalcMeshSources(...)` schedules per-chunk mesh source jobs.
2. `CreateChunkMeshes()` uploads mesh data to Unity meshes.
3. `Render(...)` draws the meshes (respecting view mode and visibility).

Group orchestration: `ChunkMeshLayerGroup`

`ChunkMeshLayerGroup` adds methods that operate across many layers:

- `SyncData(dependency)`
- `CalcCellMetrics(dependency)`
- `CalcDirtyMeshSources(dependency)`
- `CalcMeshSources(chunksGridLayer, dependency)`
- `CreateChunkMeshes()`
- `Render(chunksGridLayer, viewMode, terrainTransform, camera)`

Related but important: `ChunksGridLayer`

Even though it is not one of the "six main types", `ChunksGridLayer` is foundational for rendering workflows:

- Tracks per-chunk metrics and visibility.
- Calculates visible chunk sets based on camera frustum.
- Provides `VisibleChunks` as a `NativeParallelHashSetDataLayer<int>`.

Most render systems use `ChunksGridLayer` to avoid generating or rendering offscreen chunks.

7.2 `CellEntityLayer`

Purpose: Spawn and update ECS entities associated with terrain cells.

Key data and members (as seen in `CellEntityLayer`):

- `CellEntityViews : CellEntityViewDataLayer`
- `NativeList<CellEntityConfig> CellEntityConfigs`
- `EntityManager`
- Optional `EntityCommandBuffer` for batch structural updates
- `CalculateEntities(dependency)` and `UpdateEntities(...)`

Configuration pattern:

- `CellEntityLayerConfigAsset` supplies:
  - authoring list (`CellEntities`)
  - baked range info (`EntityConfigsRangeStart`, `EntityConfigsRangeSize`)
  - plus common cell-item config inherited behavior (index/state/transform maps)

Group orchestration:

- `CellEntityLayerGroup` stores the combined baked configs and passes slices into each layer.
- It provides:
  - `CalculateEntities(dependency)` across all layers
  - `UpdateEntities(layerEntity, dependency)` across all layers

When to use:

- Use `CellEntityLayer` when cell content should be represented as ECS entities (AI, physics, gameplay systems).
- Prefer `CellObjectLayer` for large volumes of purely visual, instanced objects.

7.3 `CellObjectLayer`

Purpose: Render a large number of cell-attached visual objects efficiently using bulk instancing.

Key concepts in `CellObjectLayer`:

- Uses an `IBulkRenderer` implementation (from `Fwt.Core.BulkRendering`) for instanced rendering.
- Maintains LOD logic:
  - `LODSettings : NativeList<CellObjectLayerLODSettings>`
  - `AllCellObjectsLODSettings : NativeList<CellObjectLODSettings>`
  - `VisualsByLODLayer : List<BulkRenderItemsDataLayer>`
- Supports "states" per object and multiple submeshes per cell.
- Main runtime entry points:
  - `CalculateCellObjects(...)` / `CalculateCellObjectsLOD(...)`
  - `Render(camera, normalizedCameraZoom)` or `RenderLODs(...)`

Configuration and baking:

- `CellObjectLayerConfigAsset` (implements `ICellObjectLayerConfig`) contains:
  - object authoring list (`ObjectConfigs`)
  - LOD distances / baked LOD settings
  - baked configs, visuals, palette, state configs, etc.
- Baking is explicit via context menu commands:
  - `BakeCellObjectConfigs`
  - `BakeLODSettings`
  - `ClearBakedData`

When to use:

- Use for foliage, rocks, props, decorative clutter, or any high-count visuals.
- It is the "visual object" counterpart to `CellEntityLayer`.

7.4 `CellRegionLayer`

Purpose: Maintain and compute "regions" (clusters of cells) and provide region-based lookups.

Key data (from `CellRegionLayer`):

- `CellRegionMap : ColorMapCellValueDataLayer_Int`
  Per-cell region index.
- `RegionCells : NativeParallelMultiHashMapDataLayer<int, uint>`
  Maps region index → set of cell indices.
- `RegionSizes : NativeListDataLayer<uint>`
  Stores region sizes.

Core operations:

- `CalcRegionsCells(dependency)` and `CalcRegionsSizes(dependency)`

- `GetCellRegion(cellIndex)` and setters:
  - `SetCellRegion(...)`
  - `SetAllCellsRegion(...)`
- Serialization / deserialization support

Configuration:

- `CellRegionLayerConfigAsset` implements `ICellRegionConfig`
  - `InitValuesArgs` controls default values and color mapping (palette, auto-color mode)
  - Provides a context-menu helper: `AutoFillColorPalette()`

When to use:

- Political regions, biome regions, provinces, connected-component labeling, flood-fill results, pathfinding zones, etc.

7.5 `GeoPlastLayer`

Purpose: Simulate and/or store "material volumes" on the terrain (amount, density, volume), including derived heights like bedrock and ceiling.

`GeoPlastLayer` is the base type. It owns maps such as:

- `AmountMap : ColorMapCellValueDataLayer_Float`
- `DensityMap : ColorMapCellValueDataLayer_Float`
- `VolumeMap : ColorMapCellValueDataLayer_Float`
- `BedrockHeightMap : CellValueDataLayer<float>`
- `CeilingHeightMap : CellValueDataLayer<float>`

Core phases (exposed by `GeoPlastLayerGroup` and layers):

- `PrepareSimulation(...)`
- `CalculateSimulation(...)`
- `ApplySimulation(...)`
- Update / derived data phases:
  - `PreparePlastUpdate(...)`
  - `CalculateDensity(...)`
  - `CalculateVolume(...)`
  - `CalculateBedrock(...)`
  - `CalculateCeiling(...)`

Visual rendering: `VisualGeoPlastLayer`

`VisualGeoPlastLayer` extends `GeoPlastLayer` and renders plast values into a target `ChunkMeshLayer`:

- `SurfaceLayerRef : HexTerrainLayerReference`
- `VisualGeoPlastSettings`:
  - render mode: `Amount` or `Volume`
  - transparency behavior and thresholds

It schedules `RenderGeoPlastToHeightMapJob` that writes into:

- surface `HeightMap`

- surface `BiomeMap`
- surface `TransparencyMap`

This is the bridge between simulation data and the visible terrain surface.

Dynamic simulation: `DynamicGeoPlastLayer`

Adds flow maps and flow potentials:

- `VolumeFlowPotentialMap`, `VolumeFlowMap`
- Target references:
  - `UpFlowTargetLayerRef`
  - `DownFlowTargetLayerRef`
- Chunk-level incoming sets to optimize updates

It provides specialized flow computations (vertical and horizontal), and clear passes with `ProfilerMarker`s.

Material + thermal: `MaterialGeoPlastLayer`

Adds:

- `HeatMap`, `TemperatureMap`
- `HeatFlowPotentialMap`, `HeatFlowMap`
- Plus material settings controlling behavior

This is the primary layer type targeted by the Sun system (see `SunTerrainLayer.ApplySunHeat`).

7.6 `SunTerrainLayer` (Sun Layer)

Purpose: A global system layer that simulates sun position and applies heat/cooling to target GeoPlast layers.

Key state:

- `SunPosition : float2`
- `DayProgress`, `YearProgress`
- `SunSettings`
- `TargetLayers : List<SunTargetLayerSettings>`

Core behavior:

- `Simulate(GeoPlastLayerGroup geoPlastLayers, HexTerrainSimulationTimer timer, JobHandle dependency)`
  - checks tick settings
  - moves the sun (`MoveTheSun`)
  - applies sun heat to each configured target layer (`ApplySunHeat`)

`ApplySunHeat` targets `MaterialGeoPlastLayer` and writes into its `HeatMap` using `ApplySunHeatJob`.

Persistence:

- Implements `ISerializableTerrainLayer`:
  - `SerializeLayer(BinaryWriter writer)`
  - `DeserializeLayer(BinaryReader reader, HexTerrainSettings terrainSettings)`

When to use:

- Day/night cycles, seasonal cycles, climate-driven simulation, energy input for thermal flow, etc.

---

8. Creating your own Terrain Layer

A custom layer typically consists of:

1. A **config interface** (optional but recommended)
2. A **ScriptableObject config asset** that implements that interface and `CreateTerrainLayer()`
3. A **layer class** inheriting `HexTerrainLayer`
4. (Optional) A **group** inheriting `HexTerrainLayerGroup<T>` if you want multiple instances

8.1 Define a config interface

Keep it small: only what the layer needs at runtime.

```
using Fwt.HexTerrains.Data;

public interface IMyLayerConfig
{
        InitCellValueDataLayerArgs<float> ValuesArgs { get; set; }
        float Multiplier { get; set; }
}
```

8.2 Implement the layer

Key points:

- Allocate/resize native data in `Init(HexTerrainSettings)`.
- Read config in `Init<TInitArgs>`.
- Use `PrepareToRead/Write + AddRead/WriteDependency`.
- Respect dirty flags for performance.
- Release native memory in `Dispose()`.

```
using Fwt.HexTerrains.Data;
using Fwt.HexTerrains.DataLayers;
using Unity.Jobs;

public class MyLayer : HexTerrainLayer
{
        public float Multiplier { get; private set; } = 1f;
        public ColorMapCellValueDataLayer_Float Values;

    public override void Init(HexTerrainSettings settings)
    {
        Values ??= new ColorMapCellValueDataLayer_Float();
        Values.InitColoredDataLayer(new InitColorMapCellValueDataLayerArgs<float>
        {
            DefaultValue = 0f
        });

        base.Init(settings);
    }

    public override void Init<TInitArgs>(HexTerrainSettings settings, TInitArgs initArgs)
    {
        base.Init(settings, initArgs);

        if (initArgs is IMyLayerConfig config)
        {
            Multiplier = config.Multiplier;
            InitColoredDataLayer(Values, config.ValuesArgs);
        }
    }

    public JobHandle DoWork(JobHandle dependency)
    {
        if (Values == null || !Values.IsDirty)
        {
            return dependency;
        }

        var deps = Values.PrepareToWrite();
        dependency = JobHandle.CombineDependencies(dependency, deps);

        // Schedule jobs here...
        // dependency = job.Schedule(Settings.CellsCount, Settings.CellsPerChunk, dependenc
y);

        Values.AddWriteDependency(dependency);
        Values.SetDirty(true);
        return dependency;
    }

    public override void SetAllDirty(bool isDirty)
    {
```

```
            Values?.SetAllChunksDirty(isDirty);
        }

        public override void CompleteAllJobs()
        {
            base.CompleteAllJobs();
            Values?.CompleteAllJobs();
        }

        public override void Cleanup()
        {
            Values?.Cleanup();
        }

        public override JobHandle CleanupAsync(JobHandle dependency)
        {
            return Values?.CleanupAsync(dependency) ?? dependency;
        }

        public override void Dispose()
        {
            base.Dispose();
            Values?.Dispose();
        }
    }
```

8.3 Create the ScriptableObject config asset

```
using Fwt.HexTerrains.Data;
using UnityEngine;

[CreateAssetMenu(fileName = "MyLayerConfig", menuName = "Fwt/HexTerrains/Custom/My Layer Con
fig")]
public class MyLayerConfigAsset : HexTerrainLayerConfigAsset<MyLayer>, IMyLayerConfig
{
    [SerializeField]
    private InitColorMapCellValueDataLayerArgs<float> _valuesArgs;

    [SerializeField]
    private float _multiplier = 1f;

    public InitCellValueDataLayerArgs<float> ValuesArgs
    {
        get => _valuesArgs;
        set => _valuesArgs = (InitColorMapCellValueDataLayerArgs<float>)value;
    }

    public float Multiplier { get => _multiplier; set => _multiplier = value; }

    public override HexTerrainLayer CreateTerrainLayer()
    {
        return new MyLayer();
    }
}
```

> Implementation note: prefer matching the exact argument type you use (`InitCellValueDataLayerArgs<T>` vs `InitColorMapCellValueDataLayerArgs<T>`). The snippet above illustrates the idea; use the appropriate type for your layer's data.

8.4 (Optional) Add a group

If you want multiple instances (e.g., multiple `MyLayer` s), create:

- `MyLayerGroup : HexTerrainLayerGroup<MyLayer>`

Then add orchestration methods the way existing groups do (combine job handles, render loops, etc.).

---

9. Best practices & pitfalls

9.1 Always respect job dependencies

- Call `PrepareToRead() / PrepareToWrite()` before scheduling.

- Combine dependencies with the incoming `JobHandle`.
- Register the resulting handle back into each data layer.

This prevents race conditions and hard-to-debug memory safety issues.

9.2 Avoid unnecessary work

- Use `IsDirty` checks early.
- Prefer chunk-level dirty sets when generating meshes.

9.3 Complete jobs before resizing or disposing

- `HexTerrainLayer.Init` calls `CompleteAllJobs()` for safety.
- In your layer, override `CompleteAllJobs()` and complete your data layers too.

9.4 Keep UnityEngine API calls out of jobs

Jobs should operate on blittable/native data. Keep any Unity object manipulation (like creating `Mesh` instances) on the main thread, typically in methods such as `CreateChunkMeshes()`.

9.5 Use `HexTerrainLayerReference` for cross-layer wiring

Prefer references over hard-coded indices so configurations remain stable when layer order changes.

Summary

- `HexTerrainLayer` defines a strict lifecycle and safe job-based data access patterns.
- `HexTerrainLayerGroup` composes layers, provides lookup by name/index/reference, and orchestrates jobs across many layers.
- The framework's main pillar layers cover:
  - surface data & meshes (`ChunkMeshLayer / VisualChunkMeshLayer`)
  - ECS-backed content (`CellEntityLayer`)
  - instanced visuals (`CellObjectLayer`)
  - region clustering (`CellRegionLayer`)
  - geo/material simulation (`GeoPlastLayer` family)
  - global energy input (`SunTerrainLayer`)
- Custom layers follow the same pattern: config interface + config asset + layer class (+ optional group).

📦 Data Layers Overview

What is a Data Layer?

A **Data Layer** is a specialized container used to store, manage, and process data efficiently in Unity. Specifically designed to work seamlessly with Unity's Job System and Burst Compiler, Data Layers provide built-in dependency tracking, state monitoring (dirty flags), and memory management.

Data Layers excel in scenarios where massive amounts of data—such as terrain cell values, colors, or heights—need to be modified on the main thread and read by background worker jobs, or vice versa.

Core Capabilities:

1. **Dependency Tracking:** Built-in `JobHandle` management for safe, parallel reading and writing without race conditions.
2. **State & Versioning:** Tracks if the data is dirty (`IsDirty`) and maintains a `Version` number, making it easy to invalidate caches or trigger updates only when necessary.
3. **Memory Management:** Automatically handles disposal of unmanaged memory, native collections, and optionally, disposable data items contained within the layer.
4. **Chunking Support:** Capable of dividing localized grid data into "chunks" to track dirty states spatially (e.g., when a single terrain cell is modified, only its specific chunk is marked for regeneration).

How to Work with Data Layers

Data layers are designed to be accessed both immediately on the main thread and asynchronously via the Unity Job System.

Synchronous Access (Immediate)

If you need to read or write data immediately on the main thread, you must signal to the layer to complete any running jobs before proceeding.

- **`OpenToRead()`** : Completes all currently scheduled *write* jobs. Use this before reading data on the main thread.
- **`OpenToWrite()`** : Completes *all* currently scheduled (read and write) jobs. Use this before modifying data on the main thread.

```
myNativeDataLayer.OpenToWrite();
myNativeDataLayer.SetData(cellIndex, newValue);
myNativeDataLayer.CommitChanges(); // Marks the layer as dirty
```

Asynchronous Access (Unity Jobs)

When scheduling jobs that use a Data Layer's underlying structures, you request the necessary dependencies and update the layer once the job is scheduled.

- **`PrepareToRead()`** : Returns a `JobHandle` representing all current write dependencies. Your job runs after the writes finish.
- **`PrepareToWrite()`** : Returns a `JobHandle` representing all current read and write dependencies. Your job runs after everything else finishes.
- **`AddReadDependency(JobHandle)`** / **`AddWriteDependency(JobHandle)`** : Stores your newly scheduled job's handle in the layer so future operations wait for it.

```
// 1. Prepare to write
JobHandle deps = myNativeDataLayer.PrepareToWrite();

// 2. Schedule your job
var myJob = new ModifyDataJob
{
        Data = myNativeDataLayer.Data.AsArray()
};
JobHandle jobHandle = myJob.Schedule(myNativeDataLayer.Length, 64, deps);

// 3. Register the newly scheduled job
myNativeDataLayer.AddWriteDependency(jobHandle);
myNativeDataLayer.SetDirty(true);
```

Types of Data Layers

The framework provides a comprehensive hierarchy of generic Data Layers catering to various storage needs.

1. Basic & Native Collections

These standard models wrap common collection types with Data Layer tracking logic.

- **Managed Collections**: `ArrayDataLayer<T>` , `ListDataLayer<T>` , `DictionaryDataLayer<TKey, TItem>` , `HashSetDataLayer<T>` .
- **Native Collections**: Provide `Unity.Collections` variants safe for use inside the Burst Compiler (e.g., `NativeArrayDataLayer<T>` , `NativeListDataLayer<T>` , `NativeHashMapDataLayer<TKey, TItem>` , `NativeParallelHashSetDataLayer<T>`).

2. Chunked Data Layers

`ChunkedDataLayer` forms the backbone of spatial data tracking. It splits grid-based datasets (like terrain maps) into localized "chunks."

When you set a cell as dirty via `SetCellDirty(cellIndex)` , the layer flags the underlying chunk as dirty via the `ChunkDirtyGrid` and `DirtyChunks` arrays.

- **Edge Wrapping:** Chunk layers understand grid connections. If `IsConnectedHorizontally` or `IsConnectedVertically` are enabled, setting a cell on the grid seam automatically flags the wrapped adjacent chunk on the far side of the map as dirty.
- **Subclasses:** `ArrayChunkedDataLayer<T>`, `ListChunkedDataLayer<T>`, `NativeArrayChunkedDataLayer<T>`, `NativeListChunkedDataLayer<T>`.

3. Hex Terrain Specific Layers

These provide direct integration with `HexTerrainSettings` ensuring automatic size matching and terrain chunk topology rules.

- **`HexTerrainNativeListChunkedDataLayer<TItem>`**: Inherits the Native List chunking concepts but auto-configures sizes through a `HexTerrainSettings` profile.
- **`CellValueDataLayer<TCellValue>`**: Stores a numeric or structural value per discrete hexagon cell.
- **`CellColorsDataLayer`**: Explicitly stores a `Color32` value per cell. Allows bulk-applying its stored color data directly to a given `Texture2D` utilizing fast memory transfer operations (`ApplyColorsToTexture()`).
- **`ColorMapDataLayer`**: An extension of `CellColorsDataLayer` that also stores a `ColorPalette` (`NativeList<Color32>`). Ideal for coloring terrain surfaces based on specific index-to-color palettes.
- **`ColorMapCellValueDataLayer<TCellValue>`**: A synchronized pairing of a `CellValueDataLayer` and a `ColorMapDataLayer`. It stores the actual *value* (e.g., height, temperature, moisture) and maintains an explicit visual *Color Map* layer linked to it.
- **`AutoColorMapCellValueDataLayer<TCellValue>`**: Automates the pipeline of converting cell values into visual colors via Jobs. It supports different mapping modes via `AutoColorMapMode`:
    - **ColorIndexByValue**: The raw cell value is an exact index mapping to an entry in the Color Palette.
    - **GradientThroughPalette**: Normalized lerping between `MinValue` and `MaxValue` determines the position across the color palette array.
    - **Custom**: Override `CalculateColorMap_Custom()` to supply entirely bespoke translation Jobs. *(Implementations provided out-of-the-box include `ColorMapCellValueDataLayer_Int` and `ColorMapCellValueDataLayer_Float`.)*

Memory Management and Disposal

Data Layers implement `IDisposable`. Because they heavily utilize Unity Native Collections (`NativeList`, `NativeArray`, etc.), calling `Dispose()` on the Data Layer is required to prevent memory leaks.

By default:

1. `Dispose()` calls `CompleteAllJobs()` ensuring underlying data is no longer actively being accessed by worker threads.
2. Unmanaged collections are freed.
3. If `IsDisposableItems` is `true`, it iterates and safely disposes of the inner discrete element items before structural release.

Practical Example: Chunk Mesh Layer & Cell Metrics

To fully grasp how Data Layers interact in practice, let's look at `ChunkMeshLayer` and how it calculates `HexCellMetrics`.

A `ChunkMeshLayer` holds multiple Data Layers representing different terrain features:

- `HeightMap` (`ColorMapCellValueDataLayer_Float`)
- `BiomeMap` (`ColorMapCellValueDataLayer_Int`)
- `TransparencyMap` (`CellValueDataLayer<bool>`)
- `CellMetricsMap` (`CellMetricsDataLayer` - purely an output layer based on the above)

When terrain generation or modification occurs, we need to recalculate the physical cell properties (metrics) such as local/global coordinates, calculated height in units, and transformation matrices. Data Layers ensure this process is performant, asynchronous, and only processes what has actually changed.

The Calculation Pipeline

The `CalcCellMetrics(JobHandle dependency)` method in `ChunkMeshLayer` demonstrates the complete lifecycle of reading from multiple Data Layers and writing to another using the Unity Job System.

1. Checking Dirty States

First, the system prevents unnecessary work by checking the `IsDirty` flag of the source layers and the destination layer.

```
// If nothing has changed, simply return the input dependency
if (!HeightMap.IsDirty && !BiomeMap.IsDirty && !CellMetricsMap.IsDirty && !TransparencyMap.I
sDirty)
{
        return dependency;
}
```

2. Preparing Job Dependencies

Before reading or writing to Native Collections, the layer must wait for any previously scheduled jobs to finish. `PrepareToRead()` and `PrepareToWrite()` fetch these dependencies.

```
// The Metrics map will be modified, so we prepare to WRITE
var deps1 = CellMetricsMap.PrepareToWrite();

// Height, Biome, and Transparency maps will only be read, so we prepare to READ
var deps2 = HeightMap.PrepareToRead();
var deps3 = BiomeMap.PrepareToRead();
var deps4 = TransparencyMap.PrepareToRead();

// Combine all handles with the input dependency
dependency = JobHandle.CombineDependencies(dependency, deps1, deps2);
dependency = JobHandle.CombineDependencies(dependency, deps3, deps4);
```

3. Merging Dirty Grids (Chunking Optimization)

Since these are *Chunked* Data Layers, we don't want to recalculate metrics for the entire map—only for the chunks where height, biome, or transparency changed. `MergeChunkDirtyGrids` combines the dirty flags from the source layers into the destination's chunk grid.

```
dependency = CellMetricsMap.MergeChunkDirtyGrids(HeightMap, BiomeMap, TransparencyMap, depen
dency);

// CellMetricsMap's ChunkDirtyGrid now contains 'true' for any chunk
// that was marked dirty in HeightMap, BiomeMap, or TransparencyMap.
```

4. Scheduling the Burst Job

Next, the actual data is passed to a Burst-compiled `IJobParallelFor` struct (`CalcHexCellMetricsJob`). Notice the use of `.AsArray()` to extract the underlying Native Arrays cleanly.

```
var job = new CalcHexCellMetricsJob
{
        TerrainSettings = Settings,
        TerrainMetrics = TerrainMetrics,
        TransparencyMap = TransparencyMap.Data.AsArray(),
        HeightMap = HeightMap.Data.AsArray(),
        ColorMap = BiomeMap.Data.AsArray(),
        // We pass the merged dirty grid so the job knows which chunks to process
        ChunkDirtyGrid = CellMetricsMap.ChunkDirtyGrid.AsArray(),

        // Output arrays
        CellMetrics = CellMetricsMap.Data.AsArray(),
        CellPositions = CellMetricsMap.CellPositions.AsArray(),
};

dependency = job.Schedule(Settings.CellsCount, Settings.CellsPerChunk, dependency);
```

Inside the Job's `Execute(int index)` method, the code checks `ChunkDirtyGrid[chunkIndex]`. If the chunk isn't dirty, it immediately returns, saving massive amounts of CPU time.

5. Registering the Scheduled Job

Finally, the modified and read Data Layers need to be informed of this newly scheduled job so they can protect their memory in future operations. The destination layer is also marked as dirty, triggering its own downstream updates (like mesh generation).

```
// Set write dependency on the output layer
CellMetricsMap.SetWriteDependency(dependency);

// Add read dependencies on the source layers
HeightMap.AddReadDependency(dependency);
BiomeMap.AddReadDependency(dependency);
TransparencyMap.AddReadDependency(dependency);

// Mark the CellMetricsMap as dirty so downstream systems know it changed
CellMetricsMap.SetDirty(true);
return dependency;
```

Summary of the Workflow

By utilizing Data Layers, `ChunkMeshLayer` completely avoids blocking the main thread during heavy calculations. Data Layers safely bridge the gap between complex game logic state, dependency tracking, chunk-based culling, and highly optimized Unity Burst jobs.

Chunk Mesh Layers

This document explains the **Chunk Mesh** feature in HexTerrains: how `ChunkMeshLayer` stores per-cell surface data, how `VisualChunkMeshLayer` turns that data into chunk meshes, how `ChunkMeshLayerGroup` orchestrates multiple mesh layers, what specialized mesh layers exist (Surface/Water/Clouds/Roads/Rain), which **jobs** generate mesh geometry and biome color maps, and how to implement a new chunk mesh layer (using Clouds as the reference model).

---

1 What "Chunk Mesh" means in HexTerrains

HexTerrains renders terrain as a grid of **chunks**, where each chunk contains a fixed-size rectangle of hex cells. A **chunk mesh layer** is responsible for:

1. Holding per-cell surface data (height, biome, transparency)
2. Converting that data into per-cell **metrics** (`HexCellMetrics`)
3. Scheduling mesh generation jobs per **dirty + visible** chunk
4. Producing `Mesh` objects and rendering them with one or more materials

The architecture separates data from visuals:

- `ChunkMeshLayer` = **data & metrics** (height/biome/transparency/cell metrics + biome color map)
- `VisualChunkMeshLayer` = **mesh sources, meshes, render settings, render loop**
- Specialized layers (Clouds/Water/Roads/Rain) override only the **mesh generator job** while reusing the same pipeline.

---

2 Key classes and responsibilities

2.1 `ChunkMeshLayer` (data + metrics)

`ChunkMeshLayer` (`HexTerrainLayer` , `ISerializableTerrainLayer`) owns the fundamental per-cell maps:

- `HeightMap : ColorMapCellValueDataLayer_Float`
- `BiomeMap : ColorMapCellValueDataLayer_Int`
- `TransparencyMap : CellValueDataLayer<bool>`
- `CellMetricsMap : CellMetricsDataLayer`
- `TerrainMetrics : HexTerrainMetrics`

It also supports synchronization from another layer:

- `SyncHeightMapLayerReference`
- `SyncBiomeMapLayerReference`
- `SyncTransparencyMapLayerReference`

Auto-biome painting (Height → Biome)

When initialized with `IChunkMeshLayerConfig` and `IsAutoPaintBiomes = true` , `ChunkMeshLayer.Init<TInitArgs>` calls:

- `FillBiomeByHeight(biomeMap, heightMap, AutoPaintBiomesLevels)`
- which uses `GetAutoBiomeHeightLevel(height, levels)` (descending threshold match)

This means biome indices can be derived entirely from the height map without a custom biome generation job.

2.2 `VisualChunkMeshLayer` (mesh sources + meshes + rendering)

`VisualChunkMeshLayer : ChunkMeshLayer` adds everything required to **generate** and **render** meshes:

- `RenderSettings : ChunkMeshRenderSettings`
- `ChunkMeshSources : ChunkMeshSourcesDataLayer` (one `MeshSource` per chunk)
- `ChunkMeshes : ChunkMeshesDataLayer` (one Unity `Mesh` per chunk)
- `BiomeUvConfigs : NativeList<ChunkMeshBiomeUVConfig>` (atlas rects)
- `RenderConfigs : List<HexSurfaceRenderConfigAsset>` (per view mode)
- `DefaultRenderConfig : HexSurfaceRenderConfigAsset`
- `IsVisible`
- `OffsetTransform` (typically height offset via `RenderSettings.HeightOffset`)

`MeshSource` and why it exists

Mesh generation runs in jobs, so it cannot safely write Unity `Mesh` directly. Instead jobs write into a Burst-friendly container:

- `MeshSource` (from `com.freewebtime.core`) stores vertices, normals, colors, UVs (4 channels), indices (up to 16 submeshes), plus bounds.
- After the job finishes, the main thread converts `MeshSource` → Unity `Mesh`.

`ChunkMeshSourcesDataLayer` additionally tracks:

- `GeneratedMeshIndexes : NativeList<int>` (which chunks finished generation this frame)
- Dirty chunk flags (`DirtyChunks`, `ChunkDirtyGrid` inherited from chunked data layer base)

Two mesh upload modes

`VisualChunkMeshLayer` supports two ways to upload mesh data:

1. **Classic**: `MeshSource.FillMesh(mesh, ...)` on main thread
   (`CreateChunkMeshes_Classic`)
2. **MeshData path**: allocate `MeshDataArray`, fill it using `FillMeshDataJob`, then apply via:
   `Mesh.ApplyAndDisposeWritableMeshData(...)`
   (`CreateChunkMeshes_MeshData + CompleteFillingMeshes`)

Which path is used is controlled by:

- `IsGenerateMeshesWithMeshData`
- and the `ChunkMeshSources.MeshSourceMode` is set accordingly (`SeparatedData` vs `CombinedData`)

2.3 `ChunkMeshLayerGroup` (orchestration across multiple layers)

`ChunkMeshLayerGroup : HexTerrainLayerGroup<ChunkMeshLayer>, IComponentData` is a container for multiple chunk mesh layers and exposes a common pipeline:

- `SyncData(dependency)` → calls `layer.SyncData` for each
- `CalcCellMetrics(dependency)` → calls `layer.CalcCellMetrics` for each
- `CalcDirtyMeshSources(dependency)` → for each `VisualChunkMeshLayer`, maps dirty metrics → dirty chunk mesh sources
- `CalcMeshSources(chunksGridLayer, dependency)` → schedules per-chunk mesh generation jobs
- `CreateChunkMeshes()` → uploads meshes
- `Render(chunksGridLayer, viewMode, terrainTransform, camera)` → draws meshes

Layer creation is factory-driven via `CreateChunkMeshLayerRequest` and the config asset's `CreateTerrainLayer()`.

---

3 The Chunk Mesh update pipeline (end-to-end)

A typical frame/update pipeline is:

1. **Sync maps (optional)**
    - `ChunkMeshLayer.SyncData()` calls:
        - `SyncHeightMap()`
        - `SyncBiomeMap()`
        - `SyncTransparencyMap()`
    - Each uses `SyncDataLayersMapJob<T>` and merges dirty chunk grids.
2. **Recalculate `HexCellMetrics`**
    - `ChunkMeshLayer.CalcCellMetrics(dependency)` schedules `CalcHexCellMetricsJob`
    - Inputs:
        - `HeightMap`, `BiomeMap`, `TransparencyMap`
    - Output:
        - `CellMetricsMap + CellPositions`
    - Transparency can be forced by `ChunkMeshLayerSettings.UseHeightTresholdForTransparency + HeightTresholdForTransparency`.
3. **Map dirty metrics → dirty chunk mesh sources**
    - `VisualChunkMeshLayer.CalcDirtyMeshSources(dependency)` merges dirty grids from:
        - `CellMetricsMap` and `TransparencyMap`
    - Marks `ChunkMeshSources` dirty per chunk.
4. **Generate mesh sources for dirty + visible chunks**
    - `VisualChunkMeshLayer.CalcMeshSources(chunksGridLayer, isCalcAllMeshes, dependency)`:
        - iterates over `chunksGridLayer.VisibleChunks`
        - schedules `ScheduleGenerateMeshJob(...)` per chunk
        - tracks generated chunks in `ChunkMeshSources.GeneratedMeshIndexes`
5. **Upload meshes to Unity**
    - `VisualChunkMeshLayer.CreateChunkMeshes()`:
        - classic or MeshData path
        - clears `GeneratedMeshIndexes`
6. **Render**

- `VisualChunkMeshLayer.Render(...)` :
  - fetches `HexSurfaceRenderConfigAsset` by view mode
  - uses `Graphics.DrawMesh` per chunk and per material/submesh

> Visibility culling comes from `ChunksGridLayer` (not a mesh layer). It computes `ChunkVisibility` and `VisibleChunks` using the camera frustum.

---

4 Specialized `VisualChunkMeshLayer` implementations

All specialized layers share the same pipeline, but override **which job** generates mesh geometry.

4.1 Surface (default) — `VisualChunkMeshLayer` + `GenerateSurfaceChunkMeshJob`

If you use `VisualChunkMeshLayer` directly, mesh generation is:

- `VisualChunkMeshLayer.ScheduleGenerateMeshJob(...)` → schedules `GenerateSurfaceChunkMeshJob`

Characteristics:

- Skips `HexCellMetrics.IsTransparent` cells
- Generates:
  - a central “plato” (top surface)
  - “bridges” between cells
- Supports:
  - colored borders (`ChunkMeshRenderSettings.IsColoredBorders`)
  - slope threshold border painting (`ColoredSlopeTreshold`)
  - biome atlas mode (`IsUseBiomeAtlas`)

4.2 Water — `WaterChunkMeshLayer` + `GenerateWaterChunkMeshJob`

`WaterChunkMeshLayer : VisualChunkMeshLayer` schedules:

- `GenerateWaterChunkMeshJob`

Notable behavior differences in the job:

- Handles transparency differently (water surface can still produce bridge geometry depending on neighbor transparency/height logic)
- Still uses the same UV channel layout and border logic concepts

Config asset:

- `WaterChunkMeshLayerConfigAsset : VisualChunkMeshLayerConfigAsset`

4.3 Clouds — `CloudsChunkMeshLayer` + `GenerateCloudsChunkMeshJob`

`CloudsChunkMeshLayer : VisualChunkMeshLayer` schedules:

- `GenerateCloudsChunkMeshJob`

Adds `CloudsRenderSettings`:

- Thickness scaling (`CloudThickness`, `MaxCellHeight`)
- Optional "pillow" scaling threshold (`CloudScaleTreshold`)
- Optional bottom side rendering (`IsDrawBottomSide`)

Config asset:

- `CloudsChunkMeshLayerConfigAsset : VisualChunkMeshLayerConfigAsset, ICloudsChunkMeshLayerConfig`

4.4 Rain — `RainChunkMeshLayer + GenerateRainChunkMeshJob`

`RainChunkMeshLayer : VisualChunkMeshLayer` schedules:

- `GenerateRainChunkMeshJob`

Adds `RainRenderSettings`:

- Rain plane height (`RainHeight`)
- Thickness based on normalized cell height (`MaxCellHeight`)

Config asset:

- `RainChunkMeshLayerConfigAsset : VisualChunkMeshLayerConfigAsset, IRainChunkMeshLayerConfig`

4.5 Roads — `RoadsChunkMeshLayer` + `GenerateRoadsChunkMeshJob`

`RoadsChunkMeshLayer : VisualChunkMeshLayer` schedules:

- `GenerateRoadsChunkMeshJob`

Adds `HexRoadsRenderSettings`:

- Road width (`RoadThickness`)
- UV scaling for road tiling (`RoadsTileScalePlato`, `RoadsTileScaleBridge`)
- Optional height threshold to skip road connections across steep steps (`IsUseHeightTreshold`, `HeightTreshold`)

Important detail: the roads job uses multiple submeshes by design:

- submesh 0 = background / base
- submesh 1 = road geometry

Config asset:

- `RoadsChunkMeshLayerConfigAsset : VisualChunkMeshLayerConfigAsset, IRoadsChunkMeshLayerConfig`

---

5 Mesh generation jobs (what they do)

Each `Generate*ChunkMeshJob` follows the same high-level structure:

1. Validate that `MeshSource` is initialized.
2. `MeshSource.Clear()`
3. Loop over cells within `ChunkMetrics.ChunkSize_Cells`
4. Convert local chunk cell position → global `cellIndex`
5. Read `HexCellMetrics` and decide whether to generate geometry
6. Emit vertices/triangles into `MeshSource`
7. Compute UVs and assign triangles into submeshes

5.1 Common inputs

All mesh jobs are given (directly or indirectly):

- `HexTerrainSettings TerrainSettings`
- `HexTerrainMetrics TerrainMetrics`
- `ChunkMeshRenderSettings RenderSettings`
- `HexChunkMetrics ChunkMetrics` (the current chunk)
- `NativeList<HexCellMetrics> CellMetrics` (the whole terrain metrics array)
- `NativeList<ChunkMeshBiomeUVConfig> BiomeUVs`
- UV channel indices:
  - `UvChannelCell` (usually 0)
  - `UvChannelChunk` (usually 1)
  - `UvChannelTerrain` (usually 2)
  - `UvChannelCoord` (usually 3)

5.2 UV channel semantics (as implemented by the jobs)

Jobs compute multiple UV sets so materials can choose which mapping to use:

- **UV0 (“cell”)**: maps the hex into its biome atlas rect (or 0..1 if atlas disabled)
- **UV1 (“chunk”)**: maps the entire chunk area to 0..1
- **UV2 (“terrain”)**: maps the entire terrain area to 0..1
- **UV3 (“cellCoord”)**: encodes cell coordinate as UV, useful for data textures / lookups

5.3 Submesh strategy: atlas vs per-biome

In surface/water/clouds/rain jobs:

- If `RenderSettings.IsUseBiomeAtlas == true`:
  - geometry typically goes to **submesh 0** (single material, atlas-driven UVs)
- If `IsUseBiomeAtlas == false`:
  - submesh index is commonly `cellMetrics.Biome` (one submesh per biome/material slot)

In roads job:

- submesh allocation is specialized (base vs roads) regardless of biome.

5.4 Deep dive: how `GenerateCloudsChunkMeshJob` generates meshes (and what it teaches about all generators)

`GenerateCloudsChunkMeshJob` is one of the best references for understanding how mesh generation works in the framework because it combines:

- the standard "surface-like" hex meshing flow (`plato + bridges`)
- feature-specific geometry rules (cloud thickness + "pillow" scaling)
- optional double-sided geometry (top + bottom) using triangle winding

This section walks through the job step-by-step and points out what is shared with `GenerateSurfaceChunkMeshJob` , `GenerateWaterChunkMeshJob` , etc.

5.4.1 Input data: where `HexCellMetrics` comes from

Mesh jobs do **not** read `HeightMap / BiomeMap` directly. They read `HexCellMetrics` , which is produced earlier by `CalcHexCellMetricsJob` (via `ChunkMeshLayer.CalcCellMetrics(...)`).

`HexCellMetrics` contains (among other things):

- `CenterPoint_Local` (x/z in chunk-local space)
- `CellHeight_Points` and `CellHeight_Units`
- `Biome`
- `IsTransparent`
- `CellCoord_Global / ChunkCoord / ChunkIndex`

That's why all mesh jobs accept:

- `HexChunkMetrics ChunkMetrics` (chunk-local basis)
- `NativeList<HexCellMetrics> CellMetrics` (global per-cell array)

5.4.2 The outer loop: generate one chunk's mesh

All chunk mesh jobs start the same way:

- early out if `!MeshData.IsInitialized()`
- `MeshData.Clear()`
- iterate all cells within the chunk bounds
- compute global `cellIndex` from `ChunkMetrics.ChunkPosition_Cells` and `TerrainMetrics.TerrainSize_Cells.x`

For clouds specifically, the job skips cells when:

- `cellMetrics.IsTransparent` **or**
- `cellMetrics.CellHeight_Points <= 0f`

That second condition is cloud-specific: clouds are "born" only above a baseline height.

5.4.3 Per-cell meshing: `DrawCell`

`DrawCell` is the standard "one cell → some triangles" entry point.

For clouds it renders:

- Top side:

    - `DrawCellPlato(cell, isInverted: false)`
    - `DrawCellBridges(cell, isInverted: false)`
- Optional bottom side (if `CloudsRenderSettings.IsDrawBottomSide`):
    - `DrawCellPlato(cell, isInverted: true)`
    - `DrawCellBridges(cell, isInverted: true)`

This single pattern explains two important conventions used across the framework:

1. **Cell is the unit of generation** (each job "stamps" geometry per cell).
2. **Connectedness is handled during bridge/corner generation**, not by a global triangulation step.

5.4.4 Cloud thickness: how Y is chosen

Unlike the surface job (which uses `cellMetrics.CellHeight_Units` for Y), clouds use a computed thickness:

- `CalculateCloudThickness(cellMetrics, isInverted)`

Implementation idea:

- Start with `CloudsRenderSettings.CloudThickness`
- Scale by normalized height:
    - `clamp(cellMetrics.CellHeight_Points / CloudsRenderSettings.MaxCellHeight, 0..1)`
- If `isInverted == true`, thickness becomes negative so the bottom side is below zero.

This is why clouds look volumetric: the "top surface" is at `+thickness`, and the optional "bottom surface" is at `-thickness`.

5.4.5 Cloud scale ("pillow" effect): how the hex radius is modified

Clouds also change the **radius** of the generated hex surface per cell:

- `CalculateCloudScale(cellMetrics)`:
    - `clamp(cellHeightPoints / CloudScaleTreshold, 0..1)` (or 1 if threshold is 0)

That scale affects:

- `platoSize` (top hex size)
- cell corner positions used for bridges
- whether two adjacent cloud cells are treated as "connected"

In `DrawCellBridges`, a neighbor is treated as connected when:

- neighbor is not transparent
- neighbor scale >= 1
- current cell scale >= 1

If a neighbor is **not** connected, the job intentionally collapses some bridge/corner heights to `0f`. This creates the characteristic "puffy edge" instead of a hard bridge to an absent neighbor.

5.4.6 Plato generation: "fan triangulation" around the center

`DrawCellPlato` creates a simple triangulated hex disk:

1. Add a **center vertex** (position at `centerPos`, Y at `cloudThickness`)
2. Loop 6 corners:

    - compute corner positions with `HexMath.HexCornerPosition(centerPos, platoSize, cornerIndex, ...)`
    - add two corner vertices per side
    - add one triangle connecting `center → cornerRight → cornerLeft`

The important parts shared with the surface/water/rain jobs:

- Plato is always a **center + 6 triangle fan**.
- UVs are computed per-vertex via helper functions:
    - `CalculateUvCell / CalculateUvChunk / CalculateUvTerrain / CalculateUvCellCoord`
- Submesh selection follows atlas vs non-atlas rule:
    - `submeshIndexCenter = RenderSettings.IsUseBiomeAtlas ? 0 : cellMetrics.Biome`

Cloud-specific part: **inverted triangle winding**

When `isInverted == true`, the job flips the triangle index order so normals point downward. This is the simplest, cheapest way to render a back-facing "bottom surface" without special shader tricks.

5.4.7 Bridge generation: connecting two cells (or fading out)

Bridges are the "skirts" connecting cell platos, and they are also what hides gaps between different heights.

In clouds, `DrawCellBridges`:

- computes corner positions on the current cell
- queries adjacent cell metrics using `CalcAdjacentCellData(...)`
- computes "target" points in the neighbor cell to bridge toward
- creates triangles for:
    - the bridge face between the two cells
    - the corner triangles between bridges (to fill the diagonal gaps)

Cloud-specific behavior:

- If `isNeighbourConnected == false`, some bridge points have their Y forced to `0f`, which prevents a full bridge and produces a soft edge.
- Plato size on neighbor can differ (neighbor scale affects its corner positions), so the bridge aligns with a scaled neighbor plato.

Shared behavior with `GenerateSurfaceChunkMeshJob` and `GenerateWaterChunkMeshJob`:

- Ground/border selection uses the same logic pattern:
    - `CalculateGroundType_Bridge(...)`
    - `CalculateGroundType_BridgeCorner(...)`
- Those functions use:
    - `RenderSettings.IsColoredBorders`
    - `RenderSettings.ColoredSlopeTreshold`
    - `RenderSettings.BorderSubmeshIndex`
    - biome equality checks
- Submesh index is either:
    - `0` (atlas)
    - or biome-derived index (non-atlas)

Also note: outside-of-terrain adjacency is handled in `GetAdjacentCellMetrics(...)`. When a neighbor cell is outside the terrain, it is treated as a "missing" cell with very low height, which prevents accidental high bridges across terrain borders.

5.4.8 UV atlas: how biomes map into `ChunkMeshBiomeUVConfig`

All jobs that support atlas mode use the same idea:

- `ChunkMeshBiomeUVConfig.Value` is a `float4`:
  - `xy = minUV`
  - `zw = maxUV`

`CalculateUvCell(...)` maps a vertex position into a normalized 0..1 cell space, then remaps into the biome's UV rect:

- `uv = normalized * (max-min) + min`

This mechanism is shared across `GenerateSurfaceChunkMeshJob`, `GenerateWaterChunkMeshJob`, `GenerateRainChunkMeshJob`, and `GenerateCloudsChunkMeshJob`.

5.5 How the other generators compare (quick mental model)

Use this mapping when reading other mesh jobs:

- `GenerateSurfaceChunkMeshJob`
  - **Y**: `cellMetrics.CellHeight_Units`
  - **Radius**: standard hex radius (no scale)
  - **Skip**: transparent cells
  - **Bridges**: connect to neighbor heights; optional colored borders.
- `GenerateWaterChunkMeshJob`
  - **Y**: based on water cell height, but bridges are generated even when one side is transparent.
  - **Key difference**: special handling of transparent neighbors to avoid holes at the shoreline.
- `GenerateRainChunkMeshJob`
  - **Y**: constant `RainRenderSettings.RainHeight` (a plane)
  - **Radius**: scaled by `CalculateRainThickness(cell)` to vary coverage
  - **Skip**: transparent cells
  - **Bridges**: used to connect partial rain coverage; includes logic to skip when neighbor also has "full" rain thickness.
- `GenerateRoadsChunkMeshJob`
  - **Submeshes**: explicitly uses submesh `0` for base and `1` for roads (not biome-driven)
  - **UV0**: not a biome atlas rect; it encodes road-specific tiling coordinates
  - **Connectivity rules**: avoids drawing road segments to transparent neighbors and can enforce a height threshold.

---

6 Biome Color Map: how it is generated

Biome "color map" here refers to `BiomeMap.ColorMap` (a `Color32` per cell) used for visualization/materials that sample a texture-like map.

`ChunkMeshLayer.CalculateColorMap(dependency)` does:

- `HeightMap.CalculateColorMap(...)`
- `CalculateBiomeColorMap(...)`

6.1 Default biome color map (`CalcDefaultChunkMeshColorMapJob`)

Scheduled by `CalculateDefaultBiomeColorMap(...)` when:

- `CellMetricsMap.IsDirty == true`
- and `BiomeMap.ColorMap.ColorPalette` is created

Job behavior:

- Only processes cells in dirty chunks (`ChunkDirtyGrid`)
- If `HexCellMetrics.IsTransparent` → writes `DefaultColor`
- Else → writes `ColorPalette[HexCellMetrics.Biome]` (if within palette bounds)

6.2 Overlapping biome color map (`CalcOverlappingChunkMeshColorMapJob`)

Scheduled by `CalculateOverlapBiomeColorMap(overlappingLayer, ...)` when:

- current layer or overlapping layer `CellMetricsMap` is dirty
- overlapping is enabled via `ChunkMeshLayerSettings.HasOverlappingLayer` + reference

Job behavior:

- Only paints a cell if:
  - current cell is not transparent
  - AND `myCellMetrics.CellHeight_Points > otherCellMetrics.CellHeight_Points`
- Otherwise the cell becomes "not painted" (default color), effectively hiding this layer where another layer is higher.

This is used to stack surfaces (e.g., water under terrain, or secondary overlays).

---

7 Configuration assets (what to create in Unity)

7.1 Base config: `ChunkMeshLayerConfigAsset`

Creates `ChunkMeshLayer` (data-only) and configures:

- init args for:
  - `HeightMapArgs`
  - `BiomeMapArgs`
  - `TransparencyMapArgs`
- `IsAutoPaintBiomes` + `AutoPaintBiomesLevels`
- `ChunkMeshLayerSettings`
- sync references (`SyncHeightMapLayerReference` , etc.)

7.2 Visual config: `VisualChunkMeshLayerConfigAsset`

Creates `VisualChunkMeshLayer` and adds:

- `IsVisible`
- `RenderSettings : ChunkMeshRenderSettings`

- `RenderConfigs` (per view mode) + `DefaultRenderConfig`
- `BiomeUvs : List<Sprite>`
  Converted to `BiomeUVConfigs` (sprite UV rects) used when `RenderSettings.IsUseBiomeAtlas` is enabled.

7.3 Specialized visual configs

- `CloudsChunkMeshLayerConfigAsset` → creates `CloudsChunkMeshLayer`
- `RoadsChunkMeshLayerConfigAsset` → creates `RoadsChunkMeshLayer`
- `RainChunkMeshLayerConfigAsset` → creates `RainChunkMeshLayer`
- `WaterChunkMeshLayerConfigAsset` → creates `WaterChunkMeshLayer`

---

8 Creating a new `VisualChunkMeshLayer` (Clouds-based example)

This is the standard extension model in this codebase:

1. **Create render settings** (a struct) for your feature.
2. **Create a mesh generation job** (`IJob`) that writes to `MeshSource` .
3. **Create a new layer** deriving from `VisualChunkMeshLayer` and override `ScheduleGenerateMeshJob` .
4. **Create a config interface + ScriptableObject asset** that supplies settings and returns the layer from `CreateTerrainLayer()` .

8.1 Minimal layer implementation pattern

Use `CloudsChunkMeshLayer` as the template:

- Store settings (e.g., `MyEffectRenderSettings`)
- In `Init<TInitArgs>` , read them from a config interface
- In `ScheduleGenerateMeshJob` , create and schedule your `GenerateMyEffectChunkMeshJob`

Example shape (documentation snippet only):

```
public class MyEffectChunkMeshLayer : VisualChunkMeshLayer
{
    public MyEffectRenderSettings MySettings = MyEffectRenderSettings.Default;

    public override void Init<TInitArgs>(HexTerrainSettings settings, TInitArgs initArgs)
    {
        base.Init(settings, initArgs);

        if (initArgs is IMyEffectChunkMeshLayerConfig cfg)
        {
            MySettings = cfg.MyEffectRenderSettings;
        }
    }

    public override JobHandle ScheduleGenerateMeshJob(
        ChunkMetricsDataLayer chunkMetrics,
        ChunkMeshSourcesDataLayer chunkMeshSources,
        CellMetricsDataLayer cellMetrics,
        int chunkIndex,
        JobHandle dependency
    )
    {
        var terrainSettings = Settings;
        if (RenderSettings.IsOverrideSlopeSize)
        {
            terrainSettings.SlopeSize = RenderSettings.OverrideSlopeSize;
        }

        var job = new GenerateMyEffectChunkMeshJob
        {
            ChunkIndex = chunkIndex,
            ChunkMetrics = chunkMetrics.Data[chunkIndex],
            CellMetrics = cellMetrics.Data,
            MeshData = chunkMeshSources.Data[chunkIndex],
            TerrainMetrics = TerrainMetrics,
            TerrainSettings = terrainSettings,
            RenderSettings = RenderSettings,
            BiomeUVs = BiomeUvConfigs,
            UvChannelCell = 0,
            UvChannelChunk = 1,
            UvChannelTerrain = 2,
            UvChannelCoord = 3,
            MySettings = MySettings,
        };

        return job.Schedule(dependency);
    }
}
```

8.2 Job implementation guidance

A new mesh job should:

- Call `MeshData.Clear()`
- Iterate chunk cells using `ChunkMetrics.ChunkSize_Cells` and `ChunkMetrics.ChunkPosition_Cells`
- Skip cells based on:
  - transparency
  - height thresholds
  - your feature's settings (like Clouds scale threshold)
- Use `MeshData.AddVertex(...)` and `MeshData.AddTriangle(submesh, i0, i1, i2)`
- Compute UV sets using the same semantics as existing jobs to keep materials consistent

The Clouds job is a good reference because it shows:

- feature-specific thickness/scale calculations
- optional inverted rendering (bottom side)
- how to keep atlas-compatible UV generation

8.3 Hooking it into the terrain

To actually use the new layer:

- Create a config asset (derive from `VisualChunkMeshLayerConfigAsset`)
- Override `CreateTerrainLayer()` to return `new MyEffectChunkMeshLayer()`
- Add that config to whatever system builds a `ChunkMeshLayerGroup` (via `CreateChunkMeshLayerRequest` pipeline)

---

9 Practical notes and common pitfalls

- **Dirty propagation is mandatory**: if your simulation modifies `HeightMap / BiomeMap / TransparencyMap`, ensure the relevant data layers are marked dirty so `CalcCellMetrics` and mesh regeneration runs.
- **Mesh generation runs only for dirty + visible chunks** by default. If you "changed something" and see no update, verify:
  - chunk is visible (`ChunksGridLayer.VisibleChunks`)
  - chunk was marked dirty in `ChunkMeshSources.DirtyChunks`
- **Atlas mode** (`RenderSettings.IsUseBiomeAtlas`) changes submesh usage. Make sure your materials and `HexSurfaceRenderConfigAsset` match the chosen strategy.
- **Roads are special**: they intentionally use dedicated submeshes for base/roads; don't assume `submesh == biome`.

Quick reference: Provided chunk mesh layers and their generator jobs

Layer class	Job	Extra settings
`VisualChunkMeshLayer` (Surface)	`GenerateSurfaceChunkMeshJob`	`ChunkMeshRenderSettings`

Layer class	Job	Extra settings
`WaterChunkMeshLayer`	`GenerateWaterChunkMeshJob`	`ChunkMeshRenderSettings`
`CloudsChunkMeshLayer`	`GenerateCloudsChunkMeshJob`	`CloudsRenderSettings`
`RainChunkMeshLayer`	`GenerateRainChunkMeshJob`	`RainRenderSettings`
`RoadsChunkMeshLayer`	`GenerateRoadsChunkMeshJob`	`HexRoadsRenderSettings`

Biome color map jobs:

- Default: `CalcDefaultChunkMeshColorMapJob`
- Overlap: `CalcOverlappingChunkMeshColorMapJob`
- Metrics: `CalcHexCellMetricsJob`
- MeshData upload: `FillMeshDataJob`

Serialization & Deserialization

This document explains how **saving/loading** works in HexTerrains, how **terrain layers** participate in serialization via `ISerializableTerrainLayer`, how `HexTerrainSerializer` writes/reads the binary format, and how to add serialization support to a **custom user terrain layer**.

---

1) High-level model

HexTerrains stores runtime terrain state on a **terrain entity** (Unity Entities / ECS):

- The terrain entity always contains `HexTerrain` (`IComponentData`) which holds `HexTerrainSettings`.
- Terrain "layers" are typically stored as **managed components** (component objects) on the same entity.
- Any managed component that implements `ISerializableTerrainLayer` can be saved/loaded by `HexTerrainSerializer`.

Key interface:

- `ISerializableTerrainLayer`
  - `bool SerializeLayer(BinaryWriter writer)`
  - `bool DeserializeLayer(BinaryReader reader, HexTerrainSettings terrainSettings)`

---

2) `HexTerrainSerializer` overview

`HexTerrainSerializer` is a static utility that serializes a **single terrain entity** to/from a binary stream or file.

Entry points

Saving:

- `HexTerrainSerializer.SaveTerrainToFile(filePath, terrainEntity, entityManager, customSerializer)`
- `HexTerrainSerializer.Serialize(stream, terrainEntity, entityManager, customSerializer)`
- `HexTerrainSerializer.Serialize(writer, terrainEntity, entityManager, customSerializer)`

Loading:

- `HexTerrainSerializer.LoadTerrainFromFile(filePath, terrainEntity, entityManager, customDeserializer)`
- `HexTerrainSerializer.LoadTerrainFromFile(byte[] fileContent, terrainEntity, entityManager, customDeserializer)`
- `HexTerrainSerializer.Deserialize(stream, terrainEntity, entityManager, customDeserializer)`
- `HexTerrainSerializer.Deserialize(reader, terrainEntity, entityManager, customDeserializer)`

Serializer versioning

`HexTerrainSerializer.Version` is written first (currently `2`).

- Version `1+`: introduces the managed-component layer serialization loop (type hash + layer payload).

- Version `2+` : adds `HexTerrainSettings.SlopeSize` to the saved settings payload.

When adding new fields to `HexTerrainSettings` , the serializer version should be bumped and `ReadTerrainSettings(...)` should be updated with version guards.

---

3) Binary format (what actually gets written)

At a high level the file is:

1. **Serializer version** (`int`)
2. **Terrain settings** (`HexTerrainSettings` fields written in a stable order)
3. **Zero or more layer/component records** until end-of-stream:
    - `StableTypeHash` (`ulong`)
    - component/layer payload written by `ISerializableTerrainLayer.SerializeLayer(...)`

There is **no component count** stored for the layer records: the reader continues until `reader.BaseStream.Position == reader.BaseStream.Length` .

Type identity: Stable Type Hash

For each serialized managed component, `HexTerrainSerializer` writes:

- `TypeHash.CalculateStableTypeHash(componentType.GetManagedType())`

During load:

- `TypeManager.GetTypeIndexFromStableTypeHash(typeHash)`
- `TypeManager.GetType(typeIndex)`

This means save files depend on the **stable identity** of component types. Renaming/moving a type (or changing assemblies) may break loading unless handled via a custom deserializer.

---

4) How terrain layers participate

4.1 Managed components are scanned automatically

During save (`WriteVersion1`), the serializer enumerates all component types on the terrain entity and selects:

- **managed components**
- non-zero-sized

For each managed component object:

1. If `customSerializer` is provided and returns `true` , the serializer assumes the component was handled and skips default logic.
2. Otherwise, if `componentObj is ISerializableTerrainLayer` , it writes the type hash and calls `SerializeLayer(writer)` .

4.2 Deserialization expects components to exist

During load (`ReadVersion1`):

1. It reads the type hash and resolves `componentType`.
2. If `customDeserializer` is provided and returns `true`, it assumes you handled that record (including reading bytes) and continues.
3. Otherwise it does:
    - `entityManager.GetComponentObject<object>(terrainEntity, componentType)`
    - if it implements `ISerializableTerrainLayer`, calls `DeserializeLayer(reader, terrainSettings)`.

Implication: **the terrain entity must already have the corresponding managed component attached** for the default path to work (because `GetComponentObject` retrieves an existing component instance). If you need to create/attach missing components dynamically, do it in `customDeserializer`.

---

5) Layer groups (`HexTerrainLayerGroup`) are serializable too

`HexTerrainLayerGroup : HexTerrainLayer, ISerializableTerrainLayer` provides a standard way to serialize a *collection of layers*:

- Writes `Layers.Count` (`int`)
- For each index:
    - `hasLayer` (`bool`)
    - `layer.Name` (`string`)
    - delegates to the layer's `SerializeLayer(...) / DeserializeLayer(...)` if it implements `ISerializableTerrainLayer`

This means:

- If your terrain uses a group component (for example, a mesh-layer group), the group can be a single serializable managed component that recursively serializes its children.

---

6) Example: `ChunkMeshLayer` serialization

`ChunkMeshLayer : HexTerrainLayer, ISerializableTerrainLayer` demonstrates best practices:

- Calls `CompleteAllJobs()` before reading/writing (important for job-driven data layers).
- Serializes only the **authoritative** data:
    - `HeightMap.Serialize(writer)`
    - `BiomeMap.Serialize(writer)`
    - `TransparencyMap.Serialize(writer)`
- It does **not** serialize derived/runtime-only data like `CellMetricsMap` (those are recalculated).
- On load:
    - calls `Init(terrainSettings)` to allocate data layers to the correct size
    - deserializes maps
    - calls `SetAllDirty(true)` so downstream systems recompute dependent data (metrics, meshes, color maps, etc.)

---

7) Custom hooks: `customSerializer` / `customDeserializer`

`HexTerrainSerializer` supports two optional callbacks:

`CustomSerializerCallback(BinaryWriter writer, object componentObject)`

Use this when:

- a component does **not** implement `ISerializableTerrainLayer`
- you want a different format than `SerializeLayer`
- you want to skip a component entirely

Return `true` to tell the serializer "I handled it".

`CustomDeserializerCallback(BinaryReader reader, Type componentType, HexTerrainSettings terrainSettings)`

Use this when:

- a component type hash no longer maps to a known type
- you want to **create/add** a missing managed component before deserializing
- you want to apply migrations/rename mappings
- you want to read custom payloads

Return `true` only if you fully consumed the corresponding payload bytes from the stream.

---

8) Adding serialization support to a new custom terrain layer

Step-by-step checklist

1. **Store your runtime state** inside the layer (often via data layers like `CellValueDataLayer<T>`, `ColorMapCellValueDataLayer<T>`, etc.).
2. Implement `ISerializableTerrainLayer` on your layer component.
3. In `SerializeLayer`:
    - call `CompleteAllJobs()` (if the layer uses jobs/native containers)
    - write a **layer-local version** (`int`) if you expect the layer format to evolve
    - write your data in a strict order
4. In `DeserializeLayer`:
    - call `CompleteAllJobs()`
    - call `Init(terrainSettings)` (or equivalent) so containers match the saved terrain size
    - read version and data in the exact order you wrote it
    - mark derived outputs dirty (or recompute) so visuals and dependent layers update

Minimal example

Below is a simple managed layer that stores one integer per cell and persists it.

```
using System.IO;
using Fwt.HexTerrains.Data;
using Fwt.HexTerrains.DataLayers;

namespace Fwt.HexTerrains
{
    public sealed class MyCustomIntLayer : HexTerrainLayer, ISerializableTerrainLayer
    {
        private const int LayerFormatVersion = 1;

        public CellValueDataLayer<int> Values;

        public override void Init(HexTerrainSettings settings)
        {
            Values ??= new CellValueDataLayer<int>();
            Values.Init(settings.CellsCount, settings);

            base.Init(settings);
        }

        public override void CompleteAllJobs()
        {
            Values?.CompleteAllJobs();
            base.CompleteAllJobs();
        }

        public override void Cleanup()
        {
            Values?.Cleanup();
        }

        public override Unity.Jobs.JobHandle CleanupAsync(Unity.Jobs.JobHandle dependency)
        {
            return Values?.CleanupAsync(dependency) ?? dependency;
        }

        public override void SetAllDirty(bool isDirty)
        {
            Values?.SetAllChunksDirty(isDirty);
        }

        public bool SerializeLayer(BinaryWriter writer)
        {
            if (writer == null)
            {
                return false;
            }

            CompleteAllJobs();

            writer.Write(LayerFormatVersion);
```

```
            // Persist the data layer. NativeListChunkedDataLayer.Serialize writes:
            // - a presence flag (bool)
            // - the NativeArray payload
            return Values?.Serialize(writer) ?? false;
        }

        public bool DeserializeLayer(BinaryReader reader, HexTerrainSettings terrainSetting
s)
        {
            if (reader == null)
            {
                return false;
            }

            CompleteAllJobs();
            Init(terrainSettings);

            var version = reader.ReadInt32();
            if (version > LayerFormatVersion)
            {
                // Optionally handle forward-compat here (or fail).
            }

            var ok = Values?.Deserialize(reader) ?? false;

            // Ensure dependent systems recompute anything derived from Values
            SetAllDirty(true);

            return ok;
        }

        public override void Dispose()
        {
            Values?.Dispose();
            base.Dispose();
        }
    }
}
```

Step: make sure the layer is on the terrain entity

For the default deserialization path to work, the layer instance must already exist on the terrain entity as a **managed component**.

If you need to add it on demand (or migrate), use `customDeserializer` to attach/create it before calling its `DeserializeLayer(...)`.

---

9) Common pitfalls

- **Forgetting `CompleteAllJobs()`** before serialization: can lead to partial/invalid data.

- **Serializing derived caches** (like metrics or meshes): prefer serializing the minimal authoritative inputs, then recompute.
- **Type rename breaks stable hashes**: use `customDeserializer` to map old types or handle missing components.
- **Changing field order** in `SerializeLayer` without a version guard: always version your layer payload if it might evolve.

UserTools

`UserTools` is the HexTerrains runtime “tooling” layer: a set of interactive states (tools) driven by a state machine. A tool is represented by a `UserToolStateBase` (or a derived class) and is typically selected by a `UserToolTypes` id.

At runtime you:

1. Build a `UserToolsStateMachine`
2. Initialize it with a list of tool configs (`IUserToolStateConfig` , usually `UserToolStateConfigAsset` ScriptableObjects)
3. Initialize it with the current `IHexTerrainAPI`
4. Switch active tool by `UserToolTypes`
5. Call `Tick()` each frame

Key types overview

`UserToolTypes`

`UserToolTypes` is the canonical list of tool ids (ints) used to switch tools without relying on state indices.

File: `Runtime\UserTools\UserToolTypes.cs`

Example:

- `UserToolTypes.Pointer`
- `UserToolTypes.PaintCellObjects` (value `306`)

`UserToolStateBase`

File: `Runtime\UserTools\SM\UserToolStateBase.cs`

`UserToolStateBase` is the root base class for tool states. It inherits `SmState` (from `Fwt.Core.StateMachines`) and adds:

- `UserToolType` (abstract `int`): the id used by `UserToolsStateMachine` to find and switch the tool.
- `Settings` (`UserToolStateSettings`): basic tool metadata (currently only `Name`).
- `Init(UserToolStateSettings)` and `Init<TInitArgs>(TInitArgs)` :
  - If `TInitArgs` implements `IUserToolStateConfig` , the tool is initialized with `IUserToolStateConfig.ToolSettings` .
  - Otherwise, default settings (`UserToolStateSettings.Default`) are used.

Notes:

- `Tick()` is virtual and empty here. Actual tool behavior is implemented in derived states.

Tool state families (derived from `UserToolStateBase`)

`HexTerrainUserToolStateBase`

File: `Runtime\UserTools\SM\HexTerrainUserToolStateBase.cs`

Adds HexTerrains integration to a tool state:

- `TerrainAPI` (`IHexTerrainAPI`): the runtime API entry point for terrain operations and UI.
- `HexTerrainUserToolSettings` (`HexTerrainUserToolStateSettings`): currently controls view mode behavior:
  - `IsChangeViewMode`
  - `RequiredViewMode`
- Lifecycle integration:
  - `PrepareToRun()` stores current view mode and optionally switches to the required view mode.
  - `PrepareToStop()` optionally restores the previous view mode.
- UI helpers:
  - `GetUIScreen<TScreen>()` via `TerrainAPI.GetOrCreateUIScreen<TScreen>()`
  - `ShowSettingsScreen(...)` / `HideSettingsScreen<TScreen>()`
- Cursor helpers:
  - `SetupTerrainCursor(...)` variants for visibility, size, color, resizability.

This is the typical base for any tool that needs terrain data, cursor, and/or UI.

Brush tools: `BrushUserToolState<...>`

File: `Runtime\UserTools\SM\BrushUserToolState.cs`

Brush tools are interactive tools that apply an operation to one or many cells based on:

- mouse input
- raycast hit (`TerrainAPI.GetRaycastData()`)
- brush size / opacity (`TerrainAPI.GetBrushSize()`, `TerrainAPI.GetBrushOpacity()`)

`BrushUserToolState<TBrushTarget>`

Core brush behavior:

- Input and interaction settings:
  - `TimeTreshold` (rate limit between brush applies)
  - `CanChangeBrushSize`
  - `AllowRightMouseButton`
  - hotkeys: `ChangeModeKey`, `ChangeBrushSizeKey`, `ChangeOpacityKey`, plus `+/-` keys
  - `CursorColor`
- `Tick()`:
  1. updates cursor and brush parameters (`UpdateBrush()`)
  2. attempts to apply brush (`TryApplyBrush()`)
- Brush application rules:
  - brush mode is `0` by default, `1` when `ChangeModeKey` is held
  - calls one of:
    - `ApplyBrushToAllCells(...)` (if `IsAppliedToEveryCellOnMap(brushMode)` is true)
    - `ApplyToSingleCell(...)` (single cell or non-resizable brush)
    - `ApplyBrushToAllBrushPoints(...)` (area brush using `HexMath.GetCellsRange_Offset(...)`)
- The tool-specific operation is implemented by overriding:

- `ApplyBrush(int brushMode, int2 cellCoord, int cellIndex, TBrushTarget brushTarget, int mouseButton)`

Brush tools with persistent settings (data source + PlayerPrefs)

Brush tools often use the additional generic layers:

- `BrushUserToolState<TBrushTarget, TSettings>`
  - adds `DataSource` (usually a UI-friendly settings model)
  - loads settings on start (`PrepareToRun()` → `InitSettings()` → `LoadSettings()`)
  - saves settings on stop (`PrepareToStop()` → `SaveSettings()`)
  - includes helpers to store values in `PlayerPrefs` :
    - `SaveSettingsValue<TValue>(...)`
    - `LoadSettingsValueOrDefault<TValue>(...)`
- `BrushUserToolState<TBrushTarget, TSettings, TSettingsScreen>`
  - shows/hides a settings UI screen while the tool is active
  - uses `ShowSettingsScreen(...)` from `HexTerrainUserToolStateBase`
- `BrushUserToolState<TBrushTarget, TSettings, TSettingsScreen, TTerrainLayer>`
  - adds a standard way to resolve the active terrain layer by index and/or name
  - supports per-layer view mode mapping (`ViewModeByLayerIndex`)

Cell-item painting tool states

A representative chain (simplified):

- `UserToolStateBase`
  - `HexTerrainUserToolStateBase`
    - `BrushUserToolState<...>`
      - `PaintCellItemUserToolStateBase<...>`
        - `PaintCellItemAllDataUserToolStateBase<...>`
          - `PaintCellObjectsUserToolState`

`PaintCellItemUserToolStateBase<...>`

File: `Runtime\UserTools\SM\States\CellItems\PaintCellItemUserToolStateBase.cs`

Builds a standard “paint a cell item layer” tool:

- Uses `UserToolSettingsDataSource + UniversalToolSettingsScreen`
- Adds common parameters to `DataSource` :
  - `TerrainLayerIndex`
  - `ChunkMeshLayerIndex`
  - optional filters:
    - `IsCheckCellHeight` , `MinCellHeight` , `MaxCellHeight`
    - `IsCheckBiome` , `MinBiomeIndex` , `MaxBiomeIndex`
- Supports “read value under cursor instead of painting”:
  - `ReadCellKey` (default `LeftAlt` , configurable)

`PaintCellItemAllDataUserToolStateBase<...>`

File: `Runtime\UserTools\SM\States\CellItems\PaintCellItemAllDataUserToolStateBase.cs`

Adds parameters for painting all common item fields:

- `ItemIndex`, `ItemState`
- `ItemPosition`, `ItemRotation`, `ItemScale`

Implements `ApplyBrush(...)`:

- If `ReadCellKey` is held: reads the cell values from the layer and updates the tool parameters (no painting).
- Brush mode `0`: paints only the targeted cell (subject to optional filters).
- Brush mode `1`: paints every cell (or filtered subset), depending on filter flags.

Tool configuration: `UserToolStateConfigAsset` and derived types

`IUserToolStateConfig`

File: `Runtime\UserTools\SM\Data\IUserToolStateConfig.cs`

A tool config must provide:

- `ToolSettings` (`UserToolStateSettings`)
- `CreateUserToolState(ISmState parent)` (factory method)

`UserToolStateConfigAsset`

File: `Runtime\UserTools\SM\Configs\UserToolStateConfigAsset.cs`

A Unity `ScriptableObject` implementation of `IUserToolStateConfig`:

- serialized `UserToolStateSettings _toolSettings`
- abstract `CreateUserToolState(...)`

This is the most common way to declare tools in the Unity Editor and initialize them in a consistent way.

HexTerrain-specific config layers

These add more config surfaces without changing the state machine contract:

- `HexTerrainUserToolStateConfigAsset`
  - adds `HexTerrainUserToolStateSettings TerrainToolSettings`
- `HexTerrainBrushUserToolStateConfigAsset`
  - adds brush behavior fields (time threshold, keys, cursor color, etc.)
  - matches what `BrushUserToolState<TBrushTarget>.Init<TInitArgs>()` consumes via `IHexTerrainBrushUserToolStateConfig`
- `HexTerrainLayerGroupBrushUserToolStateConfigAsset`
  - adds `ViewModeByLayerIndex`
- `PaintCellItemUserToolStateConfigAsset`
  - adds `ReadValueKey` (key used to read cell value under cursor)

Tool-specific config assets usually derive from one of these and implement `CreateUserToolState(...)`.

Initialization flow (UserTools + state machine)

`UserToolsStateMachine`

File: `Runtime\UserTools\SM\UserToolsStateMachine.cs`

`UserToolsStateMachine` is a specialized `SmStateMachine` that:

- stores states in the base `States` list (state-machine order)
- additionally indexes tool states by `UserToolType` (`StatesByType`) for fast selection

Key methods:

- `InitStates<TInitArgs>(IEnumerable<TInitArgs> requests)` where `TInitArgs : IUserToolStateConfig`
  - creates/replaces states based on the config sequence
  - calls `state.Init(request)` for each created/reused state
- `InitStates(IHexTerrainAPI terrainAPI)`
  - for each state in `States`, if it is a `HexTerrainUserToolStateBase`, calls `userToolState.Init(terrainAPI)`
- `SwitchStateByUserToolType(int userToolType)`
  - finds a tool in `StatesByType` and switches the state machine to it (calls `PrepareToStop()` on the previous state and `PrepareToRun()` on the new state via `SmStateMachine.SwitchState(...)`)

How tools are typically initialized

A common initialization sequence is:

1. Prepare a list of configs (usually `UserToolStateConfigAsset` references from the Inspector)
2. Create `UserToolsStateMachine`
3. Call:
   - `userToolsSm.Init(configs, terrainAPI)`
     (this calls `Init(configs)` then `Init(terrainAPI)`)

In update loop:

- call `userToolsSm.Tick()` every frame

On shutdown:

- call `userToolsSm.Dispose()` (disposes disposable states)
- optionally `ClearStates()` if you need to rebuild the tool set

```
graph TD;
    A["Unity bootstrap (MonoBehaviour / system)"] --> B["Create UserToolsStateMachine"]
    B --> C["Init(configAssets) → create states + apply config"]
    C --> D["Init(terrainAPI) → inject IHexTerrainAPI into hex-terrain states"]
    D --> E["Select tool: SwitchStateByUserToolType((int)UserToolTypes.X)"]
    E --> F["Each frame: Tick() → tool logic"]
    E --> G["State transition: old PrepareToStop() / new PrepareToRun()"]
```

How to use UserTools

1) Provide tool configs

Create (or reference) a set of `UserToolStateConfigAsset` ScriptableObjects. Each config must implement `CreateUserToolState(...)`.

2) Initialize state machine

Call `UserToolsStateMachine.Init(...)` with:

- configs (`IEnumerable<IUserToolStateConfig>`)
- `IHexTerrainAPI`

3) Switch tools

Switch by type id (`UserToolTypes`) using:

- `SwitchStateByUserToolType((int)UserToolTypes.PaintCellObjects)`

4) Tick

Call `Tick()` each frame from your update loop. Brush tools apply operations during `Tick()` based on input + raycast hit.

Creating a new UserTool (example: `PaintCellObjectsUserToolState`)

`PaintCellObjectsUserToolState` is a minimal "concrete tool state" because all behavior is inherited from `PaintCellItemAllDataUserToolStateBase<...>`.

File: `Runtime\UserTools\SM\States\CellObjects\PaintCellObjectsUserToolState.cs`

```
public class PaintCellObjectsUserToolState : PaintCellItemAllDataUserToolStateBase<CellObjec
tLayerGroup, CellObjectLayer, ChunkMeshLayerGroup, ChunkMeshLayer>
{
    public override int UserToolType => (int)UserToolTypes.PaintCellObjects;
    public PaintCellObjectsUserToolState(ISmState parent) : base(parent) { }
}
```

Step-by-step: adding a similar tool

Step 1: Add/choose a `UserToolTypes` id

If you are adding a brand new tool type, add a new enum value to `UserToolTypes` (or reuse an existing one).

Step 2: Implement the tool state

Option A (simple): inherit an existing base (like `PaintCellItemAllDataUserToolStateBase`) and only override `UserToolType`.

Option B (custom): inherit `UserToolStateBase` / `HexTerrainUserToolStateBase` / `BrushUserToolState<TBrushTarget>` and implement:

- `ApplyBrush(...)` (for brush tools)
- optional UI settings (via the `BrushUserToolState<..., TSettings, TSettingsScreen>` layers)

Step 3: Create a config asset for the tool (recommended)

Create a ScriptableObject config asset so the tool can be instantiated and configured from the editor.

Below is an example config asset for `PaintCellObjectsUserToolState`. Place it near other tool configs (for consistency, under `Runtime\UserTools\SM\Configs\States\...`).

com.freewebtime.hexterrains\Runtime\UserTools\SM\Configs\States\CellObjects\PaintCellObjectsUserToolStateConfigAsse

```
using Fwt.Core.StateMachines;
using Fwt.HexTerrains.UserTools.SM;
using UnityEngine;

namespace Fwt.HexTerrains.UserTools.SM.Configs
{
    [CreateAssetMenu( fileName = "PaintCellObjectsUserToolStateConfig", menuName = "Fwt/HexT
errains/UserTools/Paint Cell Objects Tool Config" )]
    public class PaintCellObjectsUserToolStateConfigAsset : PaintCellItemUserToolStateConfig
Asset
    {
        public override UserToolStateBase CreateUserToolState(ISmState parent)
        {
            return new States.CellObjects.PaintCellObjectsUserToolState(parent);
        }
    }
}
```

Step 4: Register the config in your tool initialization list

Add the created config asset to the collection passed into `UserToolsStateMachine.Init(configs, terrainAPI)`.

Step 5: Switch to the tool at runtime

```
userToolsSm.SwitchStateByUserToolType((int)UserToolTypes.PaintCellObjects);
```

Notes / gotchas

- `UserToolsStateMachine` indexes tools by `UserToolStateBase.UserToolType`. If two states return the same `UserToolType`, the last one added wins in `StatesByType`.
- Brush tools intentionally rate-limit changes via `TimeTreshold`.
- Many brush tools persist user parameters via `PlayerPrefs` (see `BrushUserToolState<TBrushTarget, TSettings>`). If you change setting key names, existing saved values may no longer load.

User Interface (UI) in Hex Terrains Framework

Hex Terrains Framework UI is built on **Unity UI Toolkit** and a lightweight MVVM-style pattern:

- **View**: `UIDocument` + UXML/USS (UI Toolkit)
- **View host / screen**: `UIScreen` (and derived types) that own the `UIDocument` and control show/hide
- **View model / data**: plain C# objects (usually derived from `UIDataSource`) assigned to `rootVisualElement.dataSource`
- **Binding**: UI Toolkit **data binding** (binding paths + change/version tracking)

This document explains the core pieces and how to use them together.

UI Toolkit usage in the framework

What UI Toolkit provides

UI Toolkit is Unity's retained-mode UI system:

- Layout + styling via **UXML** and **USS**
- A runtime UI root via `UIDocument`
- A binding system that can **bind VisualElement properties** to properties on a C# `dataSource` object

Hex Terrains Framework leans into this by:

- Using `UIDocument` for screens (`UIScreen` and friends)
- Putting all runtime "state" into data sources (typically `UIDataSource`)
- Using UI Toolkit binding paths to connect UXML elements to that data

The binding root: `VisualElement.dataSource`

A screen assigns its view-data to the document root:

- `UIDocument.rootVisualElement.dataSource = viewData;`

Once set, UI elements in that document can bind to properties on that object using **binding paths**.

This is the single most important rule: **if the root `dataSource` is not set, bindings won't resolve**.

Screens: `UIScreen` and screen management

`UIScreen`: a UI Toolkit screen prefab

File: `com.freewebtime.core/Runtime/UI/UIScreen.cs`

`UIScreen` is a `MonoBehaviour` that hosts a `UIDocument` (via `UIDocumentView`) and implements `IUIScreen`:

- `Show()` → `gameObject.SetActive(true)`
- `Hide()` → `gameObject.SetActive(false)`
- `Name` → used for lookup/creation by name

A `UIScreen` is typically used as a **prefab**:

- prefab contains `UIScreen + UIDocument`
- `UIDocument` references a UXML asset (and optional panel settings, USS, etc.)

`UIDocumentView` and `UIDocumentView<TViewData>`

File: `com.freewebtime.core/Runtime/UI/UIDocumentView.cs`

`UIDocumentView` contains common helpers:

- Auto-finds `UIDocument` on `Start() / OnValidate()`
- Calls `InitUI()` on enable and `DisposeUI()` on disable
- Utility methods like `TryGetRootElement(out root)`

`UIDocumentView<TViewData>` implements the standard "bind view-data to the document" flow:

- Stores a private `_viewData`
- On `InitUI()` sets `rootVisualElement.dataSource = _viewData`
- Exposes `SetViewData(TViewData viewData)`

`UIScreen<TViewData>`

File: `com.freewebtime.core/Runtime/UI/UIScreen.cs`

`UIScreen<TViewData>` is the common base for "screen + view-data" prefabs:

- It has a serialized `_viewData` field (so you can assign a default instance in prefab/scene)
- It sets `rootVisualElement.dataSource = _viewData` in `InitUI()`
- It exposes `SetViewData(viewData)` to update the binding target at runtime

Screen API: `IUISystemAPI`

File: `com.freewebtime.core/Runtime/UI/IUISystemAPI.cs`

The framework standardizes screen operations behind:

- `CreateScreen<TScreen>()`
- `GetOrCreateScreen<TScreen>()`
- `GetScreen<TScreen>()`
- `DestroyScreen<TScreen>() / DestroyScreen(UIScreen screen)`

`UIManager` : MonoBehaviour screen manager

File: `com.freewebtime.core/Runtime/UI/UIManager.cs`

`UIManager` is a `MonoBehaviour` implementation of `IUISystemAPI` that:

- Stores screen **prefabs**:
  - `ScreenPrefabs`
  - `ScreenPrefabByName`

    - `ScreenPrefabByType`
- Stores created/registered **instances**:
    - `AllScreens`
    - `ScreensByName`
    - `ScreensByType`
- Instantiates screens under a `UIRoot` transform (`UIScreenRoot`)
- Optionally collects screens already present in the scene:
    - `IsCollectUIScreensFromSceneOnStart`
    - `CollectUIScreensFromScene()`
- Provides `IsCursorOverUI()` (EventSystem-based)

In practice:

1. Put `UIManager` in your scene (bootstrap).
2. Provide it a list of screen prefabs (via `ApplyConfig(...)` or by calling `AddScreenPrefab(...)`).
3. Request screens with `GetOrCreateScreen<TScreen>()` .

`UISystemBase`: ECS screen manager

File: `com.freewebtime.core.ecs/Runtime/UI/UISystemBase.cs`

If you run UI creation from an Entities `SystemBase` , use `UISystemBase` (also implements `IUISystemAPI`).

It mirrors `UIManager` 's logic (prefabs + instances + dictionaries), but is designed to live in the ECS world.

Convenience via `IHexTerrainAPI`

File: `com.freewebtime.hexterrains/Runtime/IHexTerrainAPI.cs` and `HexTerrainAPI.cs`

`HexTerrainAPI` exposes UI helpers that forward to its `UISystemAPI` :

- `CreateUIScreen<TScreen>()`
- `GetOrCreateUIScreen<TScreen>()`
- `GetUIScreen<TScreen>()`
- `DestroyUIScreen<TScreen>() / DestroyUIScreen(UIScreen screen)`

This is what tools/states typically use to show tool-related screens.

Data sources and change tracking (`UIDataSource`)

`UIDataSource`: base class for bindable view-models

File: `com.freewebtime.uitoolkit/Runtime/DataSources/UIDataSource.cs`

`UIDataSource` is the standard base for view-data in the framework:

- Holds a monotonically increasing `Version`
- Implements `CommitChanges()` → increments `Version`
- Implements view-hash via `GetViewHashCode()` returning `Version`

Conceptually:

- **Every meaningful change** to UI-facing data should call `CommitChanges()`.
- The binding system can use the view-hash/version to decide when to re-read values.

Recommended pattern for properties:

- Mark bindable properties with `[CreateProperty]`
- In the setter, call `CommitChanges()` when the value changes

```
// Typical pattern (used throughout the framework)
[CreateProperty]
public int BrushSize
{
        get => _brushSize;
        set
        {
                if (_brushSize == value) return;
                _brushSize = value; CommitChanges();
        }
}
```

`[CreateProperty]` and what it means

Many data-source properties in this framework use `Unity.Properties`:

- `[CreateProperty]` marks a property as discoverable by Unity's property/binding infrastructure.
- In practice: **if you want to bind to it, make it a property and mark it**.

> Tip: Avoid binding directly to public fields. Prefer properties with `[CreateProperty]`.

Data binding: binding paths + `dataSource`

Binding model in one diagram

```
graph TD;
A["UIScreen / UIDocumentView"] --> B["UIDocument.rootVisualElement"];
B --> C["rootVisualElement.dataSource = viewData"];
C --> D["UXML elements with binding paths"];
D --> E["UI Toolkit reads properties via paths"];
F["UIDataSource.CommitChanges()"] --> G["Version++ / view hash changes"];
G --> E;
```

Two binding styles you'll see

1 UXML declarative bindings (recommended for simple cases)

In UI Toolkit you typically bind element properties using binding paths set in UXML (exact attribute names vary by element/property).

General idea:

- Set the screen root `dataSource` to some view-data object.
- Bind UI fields to `PropertyName` on that object.

Example (conceptual):

```
<ui:Label binding-path="Title" />
<ui:SliderInt binding-path="BrushSize" />
```

2 Code-driven bindings (used by custom controls)

Some framework elements set up bindings in C# when attached to panel.

Example: `UIButton` binds its own `OnClick` property using `SetBinding(...)` and a data-source path.

`Fwt.UIToolkit` controls

`UIButton`: bindable click target

File: `com.freewebtime.uitoolkit/Runtime/UIButton.cs`

`UIButton` is a UI Toolkit `Button` with an additional bindable property:

- `OnClick` (type `object`) — can receive:
  - `Action`
  - `UnityEvent`
  - `UIButtonDataSource` (see below)
- `OnClickPath` (`string`, UXML attribute `onClick`) — property path used to bind `OnClick` at runtime

How it works:

- On `AttachToPanelEvent`, if `OnClickPath` is not empty:
  - `UIButton` calls `SetBinding(...)` to bind its `OnClick` property from `dataSourcePath = OnClickPath`
- On click / navigation submit:
  - It calls `ProcessOnClick()` and invokes whatever was bound into `OnClick`

This enables an MVVM-like flow where the view binds to a view-model "command" without requiring hard-coded callbacks.

UXML usage

Conceptual UXML (namespace setup depends on your project's UXML settings):

```
<fwt:UIButton name="ApplyButton" onClick="Apply" text="Apply" />
```

And your view-data object would expose `Apply` as one of:

- an `Action`
- a `UnityEvent`
- a `UIButtonDataSource` (with `OnClick`)

`UIButton` binding gotcha

`OnClickPath` defaults to `"OnClick"`. If your data source uses a different property name, set `onClick="YourPropertyName"`.

Built-in data sources

`UIButtonDataSource`

File: `com.freewebtime.uitoolkit/Runtime/DataSources/UIButtonDataSource.cs`

`UIButtonDataSource : UIDataSource` is a reusable model for "button-like" UI:

- `Text` (`string`) — calls `CommitChanges()` on set
- `OnClick` (`Action`) — calls `CommitChanges()` on set
- `IconTexture` (`Texture2D`) — calls `CommitChanges()` on change
- `IconSprite` (`Sprite`) — calls `CommitChanges()` on change

`UIButton.ProcessOnClick()` explicitly supports this type:

- if `OnClick` is a `UIButtonDataSource`, it invokes `buttonDataSource.OnClick`

So you can either:

- bind `UIButton.OnClick` to an `Action` property, **or**
- bind it to a `UIButtonDataSource` and keep button data grouped.

`ValueEditorDataSource` and `ValueEditorDataSource<TValue>`

File: `com.freewebtime.uitoolkit/Runtime/ValueEditors/DataSources/ValueEditorDataSource.cs`

This is the base abstraction for "editable setting/value" UI models.

Key members:

- `Name` (display name / label)
- `IsVisible` → exposed as `[CreateProperty] DisplayStyle` (Flex/None)
- `IsClampValue` + `ClampValue(...)` helper
- `ValueChanged` event hook (invoked from `CommitChanges()`)

`ValueEditorDataSource<TValue>` adds the actual value:

- `[CreateProperty] TValue Value` with change detection and `CommitChanges()`

- `ValueType` returns `typeof(TValue)`

This makes it possible to build generic “value editor” UIs where:

- the UI displays a list of `ValueEditorDataSource`
- the concrete editor type is chosen by `ValueType` (int/float/bool/enum/etc.)
- visibility is driven by `DisplayStyle`

> Note: The framework’s UI layer typically consumes these via tool settings screens (see below).

Tool settings UI in Hex Terrains

Hex Terrains tools commonly display a settings screen while active (see `UserTools` documentation for the tool lifecycle).

`ToolSettingsScreen` and `ToolSettingsScreen<TSettings>`

File: `com.freewebtime.hexterrains/Runtime/UserTools/UI/ToolSettingsScreen.cs`

This is a specialization of `UIScreen` meant for tool settings.

`ToolSettingsScreen<TSettings>` :

- Implements `IViewDataReceiver<TSettings>`
- Sets `rootVisualElement.dataSource = viewData` on init / set

`UniversalToolSettingsScreen`

File: `com.freewebtime.hexterrains/Runtime/UserTools/UI/UniversalToolSettingsScreen.cs`

A convenience concrete type:

- `UniversalToolSettingsScreen : ToolSettingsScreen<UserToolSettingsDataSource>`

This is the “default” settings screen many tools use.

`UserToolSettingsDataSource`

File: `com.freewebtime.hexterrains/Runtime/UserTools/DataSources/UserToolSettingsDataSource.cs`

This is the standard view-data for tool settings UI:

- `Description + ErrorMessage` are `[CreateProperty]`
- Also exposes `[CreateProperty] DescriptionDisplayStyle / ErrorMessageDisplayStyle`
  - automatically hides UI blocks if strings are empty
- `Values : List<ValueEditorDataSource>`
  - collection of setting entries shown by the UI (typically via a `ListView`)

Common usage pattern:

1. A tool creates/owns a `UserToolSettingsDataSource` .

2. The tool fills `Values` with `ValueEditorDataSource<T>` entries.
3. The tool shows `UniversalToolSettingsScreen` and calls `SetViewData(dataSource)`.

Putting it together (typical runtime flow)

1 Configure a UI system

Pick one:

- `UIManager` (MonoBehaviour)
- `UISystemBase` (Entities `SystemBase`)

Register screen prefabs using `ApplyConfig(...)` or `AddScreenPrefab(...)`.

2 Create or get a screen

Via `IUISystemAPI`:

- `var screen = uiSystemApi.GetOrCreateScreen<MyScreen>();`

Or via `IHexTerrainAPI`:

- `var screen = terrainApi.GetOrCreateUIScreen<UniversalToolSettingsScreen>();`

3 Provide view-data

- Use `UIScreen<TViewData>` / `ToolSettingsScreen<TSettings>` patterns where possible.
- Call `SetViewData(...)` with a view-data instance (often a `UIDataSource`).

4 Bind in UXML and/or via custom elements

- In UXML: bind label/text/values by binding paths.
- For command-like actions: use `Fwt.UIToolkit.UIButton` with `onClick="SomeProperty"`.

5 When data changes, call `CommitChanges()`

- Data sources derived from `UIDataSource` should call `CommitChanges()` in setters.
- This increments `Version` so the UI binding system can refresh.

Practical guidelines / gotchas

- **Keep the view-data instance stable** while the screen is shown.
  - Prefer mutating properties + `CommitChanges()` rather than replacing the whole data source repeatedly.
- **Use `[CreateProperty]`** on anything you want to bind to.
- **Always call `CommitChanges()`** when a bound value changes (or `MarkDirty()` if you intentionally only signal "something changed").
- **Binding paths must match property names** exactly (case-sensitive in most binding contexts).

- **Prefer `UIScreen<TViewData>`** (or `ToolSettingsScreen<TSettings>`) for consistent "assign `dataSource`" behavior.
- When using `UIButton`:
  - Set `onClick` to the correct property path
  - Provide `Action` / `UnityEvent` / `UIButtonDataSource` at that path

Related docs

- User Tools (UserTools.html) (how tools show/hide settings screens during tool activation)